CODEBASE ANALYSIS & LEARNING DOCUMENT

DemandIQ

Automated Demand Forecasting & ML Experimentation Platform

PATH C:\Users\utkar\Videos\Placement-Projects\DemandIQ

GENERATED August 18, 2026

SCOPE Tech stack · architecture · file-by-file walkthrough · build sequence · execution guide

Table of Contents

Part 1: Technology Stack & Tools Analysis

Languages

Python

WHAT IS IT?

Python is a high-level, dynamically typed, interpreted general-purpose programming language emphasizing readability, with an enormous ecosystem for data science and ML.

PURPOSE IN THIS PROJECT

Python is the language for the entire backend: data loading (`src/data_loading/`), validation, preprocessing, feature engineering, all 7 model implementations (`src/models/`), the walk-forward CV/training orchestrator (`src/training/train_pipeline.py`), MLflow tracking, drift/performance monitoring, retraining orchestration, the forecast API, and the FastAPI dashboard server (`dashboard/server.py`). `setup.py` packages `src` as an installable package requiring Python >=3.10.

KEY FEATURES USED

- Type-agnostic scripting glue between pandas/numpy and ML frameworks (sklearn, lightgbm, xgboost, torch, statsmodels)
- `pathlib.Path` for cross-platform file/artifact paths (seen in `setup.py` , `downloader.py`)
- Dynamic/deferred imports (e.g., `model_factory.py` imports `LSTMModel` /torch lazily so the rest of the pipeline runs if torch is unavailable)
- Dataclass/dict-driven config plumbing via `config.yaml` + `PyYAML`
- Exception handling for optional dependencies (Evidently API version fallback, torch import guard)

DETAILED DESCRIPTION

Python's role here is as the substrate that every other backend technology sits on top of. The project deliberately structures itself as a package (`src/`) with clear submodules per pipeline stage — this mirrors how real ML platforms separate concerns (data, features, models, training, monitoring) while staying importable both as a CLI entrypoint (`main.py`) and as library code consumed by the FastAPI dashboard server and pytest suite. Python's dynamic typing and duck-typing let the `BaseModel` abstraction (`src/models/base_model.py`) unify wildly different model families — a `MovingAverageModel` , an `ARIMAModel` , and a PyTorch `LSTMModel` — behind one `fit` / `predict` interface, which is what lets `train_pipeline.py` iterate over all 7 models identically inside the walk-forward CV loop. Its interpreted nature and REPL-friendliness also power the `notebooks/` exploration workflow, and its packaging tools (`setup.py` , `find_packages`) make `src` reusable from both `main.py` and `dashboard/server.py` without path hacks.

ALTERNATIVES

1. **R** - stronger built-in statistical/time-series primitives (e.g., native ARIMA, forecast package) but a much thinner ML-engineering ecosystem (no LightGBM-grade tooling parity, weaker deployment story); this project chose Python for its unified ML + web-API + data ecosystem.
2. **Julia** - faster numerics for custom model code, but immature library ecosystem for MLflow-equivalents, drift detection, and web frameworks; not worth the tooling gap for this scope.
3. **Scala/JVM** - would fit if this were a Spark-scale pipeline (M5's full 58M rows), but the project explicitly caps series count to stay in Python/pandas comfortably.
4. **Node.js** (full-stack JS) - would unify frontend/backend language, but forfeits the scientific-Python ecosystem (sklearn, statsmodels, lightgbm) with no equivalent maturity in JS.

PROS

- Single language spans data loading, ML, and API serving, reducing context switching
- Massive ecosystem of maintained ML/data libraries directly used here (pandas, sklearn, lightgbm, xgboost, torch, statsmodels)
- Readable for a demonstrable/teaching platform like DemandIQ
- Straightforward interop between FastAPI, MLflow, and pandas-based artifacts
- `pip` / `venv` tooling is simple enough for a local-only, no-infra project

CONS

- GIL limits true multi-core parallelism (mitigated here since LightGBM/XGBoost/sklearn use native multithreading internally)
- Slower raw execution than compiled languages for the manual feature-engineering loops
- Dependency/version drift across a large requirements.txt (numpy pinned `<2.0` for compatibility) requires care

RESOURCES

- https://docs.python.org/3/

JavaScript / JSX

WHAT IS IT?

JavaScript is the dynamic scripting language of the web; JSX is a syntax extension (compiled by Babel/SWC) that lets React component markup be written inline with JavaScript logic.

PURPOSE IN THIS PROJECT

JSX/JavaScript (ES modules, `"type": "module"` in `frontend/package.json`) powers the entire React dashboard: `frontend/src/App.jsx` wires up routing, and page components (`Overview.jsx` , `DataOverview.jsx` , `Eda.jsx` , `ModelComparison.jsx` , `Forecasting.jsx` , `MlflowTracking.jsx` , `DriftMonitoring.jsx`) render charts and tables from the

FastAPI backend's JSON.

KEY FEATURES USED

- ES module syntax (`import` / `export`) throughout `frontend/src`
- JSX component syntax combining markup and logic (e.g., `ModelComparison.jsx` importing Recharts primitives directly into render logic)
- Async/await + fetch-style API calls via a custom `useApi` hook wrapping axios (`frontend/src/api.js`)
- Array destructuring/functional composition for chart data shaping (`imp.head(15).rename(...)` -style patterns mirrored client-side)
- Modern React 19 function components and hooks (no class components)

DETAILED DESCRIPTION

JSX is used exclusively rather than plain JS+templating, which is idiomatic for React 19 and lets each page co-locate its data-fetching, loading/error states, and chart configuration in one file (e.g., `Loading.jsx` exports both `Loading` and `ErrorBox` for reuse across pages). The frontend is a small, focused set of ~14 JSX files rather than a heavyweight SPA architecture — no Redux, no global state library — reflecting the project's "dashboard is a pure reader" design principle from `ARCHITECTURE.md` . Vite (see Build Tools) compiles this JSX to browser-runnable JS with esbuild/SWC-backed fast refresh during `npm run dev` . Because the project has no TypeScript compiler in play for `.jsx` files, JSDoc/PropTypes are absent; correctness relies on component simplicity and the small page count, keeping the amount of JS logic proportional to what a dashboard-only frontend needs.

ALTERNATIVES

1. **TypeScript/TSX** - adds static typing and catches prop/shape mismatches against the FastAPI JSON responses at compile time; this project uses `@types/react` / `@types/react-dom` as devDependencies but authors in plain `.jsx` , likely to keep the demo lightweight.
2. **Vue (with `.vue` SFCs)** - comparable component model with less JSX boilerplate, but the team already standardized on React/Recharts.
3. **Svelte** - smaller bundle/no virtual DOM, attractive for a simple dashboard, but smaller ecosystem for chart libraries as mature as Recharts.
4. **Plain HTML + vanilla JS** - would suffice for a handful of static charts, but the multi-page routed dashboard (7 routes) benefits from component reuse (`ChartCard` , `MetricCard` , `StatusBadge`).

PROS

- Ubiquitous browser support, no compile-to-native step needed beyond bundling
- Naturally pairs with Vite's HMR for fast iteration on dashboard pages
- JSX keeps chart/markup logic co-located and readable per page
- Huge ecosystem (Recharts, axios, react-router-dom) available via npm
- Low ceremony for a small, read-only dashboard app

CONS

- No compile-time type safety (despite `@types/*` present, code is `.jsx` not `.tsx`)
- Runtime errors (e.g., malformed API response) surface only when hit, not at build time
- JSX requires a build step (Vite/Babel) rather than running natively in browsers

RESOURCES

- https://react.dev/learn/writing-markup-with-jsx

Frameworks & Libraries

pandas

WHAT IS IT?

pandas is the standard Python library for labeled, tabular data manipulation, providing the `DataFrame` / `Series` structures used throughout data science workflows.

PURPOSE IN THIS PROJECT

pandas is the backbone data structure for the entire pipeline — loading M5/CSV/synthetic data, validation, preprocessing (frequency detection, gap reindexing, imputation), the ~40-feature engineering pipeline, model input/output framing, and the dashboard server's artifact reads (`clean.parquet` , `features.parquet` , `model_comparison.csv` , `forecast.csv`).

KEY FEATURES USED

- `DataFrame` /groupby operations for per-series (`series_id`) aggregation across the long-format canonical schema
- `.shift(1)` before rolling/expanding windows to guarantee leakage-safe lag/rolling features (explicitly called out in `ARCHITECTURE.md`)
- `to_parquet` / `read_parquet` and `to_csv` / `read_csv` I/O (used in `feature_engineering.py` , `preprocessor.py` , `loader.py` , `downloader.py` , `train_pipeline.py` , `forecast_api.py` , `drift_detector.py`)
- Date/time indexing and reindexing for gap-filling irregular series
- `seasonal_decompose` , `acf` / `pacf` integration points (via statsmodels) consuming pandas Series in `dashboard/server.py` 's `/api/eda/{series_id}`

DETAILED DESCRIPTION

pandas is the single most heavily used library in the codebase because DemandIQ's canonical data contract (`date, series_id, item_id, store_id, sales, sell_price, promotion_flag`) is a long-format DataFrame that every stage — load, validate, clean, feature-engineer, train, forecast, monitor — both consumes and produces. The choice to standardize everything downstream of the loader on this one shape (per `ARCHITECTURE.md`) means any new data source only needs to emit a pandas DataFrame in that shape. Its groupby/window-function API is what makes the leakage-safe feature design tractable: lag features, rolling means, and hierarchical (store/item) averages are all computed with per-series `groupby(...).shift(1).rolling(...)` chains rather than hand-rolled loops. pandas also functions as the dashboard's serialization layer — DataFrames are read back from parquet/CSV artifacts by the FastAPI server and converted to JSON for the React frontend, making pandas the bridge between the ML pipeline and the UI.

ALTERNATIVES

1. **Polars** - faster (Rust-backed, multi-threaded) DataFrame library with a similar but stricter API; would speed up the ~40-feature pipeline, but pandas has broader interop (statsmodels, sklearn, MLflow all expect pandas natively) so switching would add friction for marginal gain at M5's top-N scale.
2. **Dask DataFrame** - would help if scaling to full M5 (58M rows) across cores/machines, but the project explicitly keeps `n_series` small to stay in-memory and demonstrable in <15 minutes, so Dask's complexity isn't justified.
3. **Spark (PySpark)** - appropriate for true big-data scale, massive overkill and infra burden for a local, zero-infra demo platform.
4. **NumPy structured arrays only** - lower-level, would work but loses labeled indexing/groupby ergonomics that make the per-series feature logic readable.

PROS

- Universal support across every other library in the stack (sklearn, statsmodels, mlflow, evidently all accept pandas natively)
- Expressive groupby/window API fits the leakage-safe, per-series feature design well
- Mature parquet/CSV I/O used consistently for all pipeline artifacts
- Readable code for validation/quality-scoring logic
- Good enough performance at the project's intentionally capped scale (top-N series)

CONS

- Single-machine, largely single-threaded for row-wise operations — would need Dask/Polars if scaled to full M5
- Memory-hungry for wide feature sets (~40 features × many series)
- `numpy<2.0` pin in requirements.txt suggests care needed around pandas/numpy version compatibility

RESOURCES

- https://pandas.pydata.org/docs/

numpy

WHAT IS IT?

NumPy is the foundational Python library for fast, vectorized numerical arrays and linear algebra operations.

PURPOSE IN THIS PROJECT

NumPy underlies nearly every numeric computation — it's the array engine beneath pandas, scikit-learn, LightGBM/XGBoost, statsmodels, and PyTorch tensors' interop, and is used directly for metric calculations (MAPE, RMSE, MAE, R^2, SMAPE in `src/evaluation/metrics.py`) and array manipulation in feature engineering and forecasting.

KEY FEATURES USED

- Vectorized arithmetic for the 5 evaluation metrics (avoiding Python loops over prediction arrays)
- `np.nan`/masking for skipping zero-sales rows in MAPE calculation (per `ARCHITECTURE.md`)
- Array reshaping/windowing for LSTM sequence construction (60-step input windows)
- Random seed control (`random_seed: 42` in `config.yaml`) via `np.random.seed` for reproducibility across models
- Linear algebra primitives underlying scikit-learn's `LinearRegression`/`RandomForest`

DETAILED DESCRIPTION

NumPy is largely an invisible dependency here — the project code rarely imports it directly for business logic, but it is the array substrate every other numeric library (pandas, sklearn, statsmodels, torch tensors via `.numpy()`) is built on. Its explicit direct use is concentrated in `src/evaluation/metrics.py`, where vectorized formulas compute MAPE/RMSE/MAE/R^2/SMAPE across arrays of predictions vs. actuals for every fold and every model. The pin `numpy>=1.24,<2.0` in `requirements.txt` is a deliberate compatibility guard — NumPy 2.0 introduced breaking ABI changes that several ML libraries (older LightGBM/XGBoost/torch builds) hadn't yet universally supported at the time these dependencies were selected, so capping below 2.0 avoids subtle breakage across the whole dependency graph. Reproducibility across the walk-forward CV (`random_seed: 42`) also depends on NumPy's RNG being seeded consistently before model training, ensuring the 7-model comparison is deterministic run to run.

ALTERNATIVES

1. **CuPy** - GPU-accelerated NumPy-compatible arrays; would help if the pipeline needed GPU-bound numeric work outside PyTorch's own tensor ops, but nothing here needs it beyond LSTM training (which torch already handles).
2. **JAX** - offers autodiff + JIT compilation; irrelevant here since no custom differentiable models are built by hand (LSTM uses plain torch).
3. **Plain Python lists/math module** - would work for the metrics module alone but loses vectorized performance and array broadcasting used across every stage.

PROS

- Universal, zero-friction interop with every ML library in the stack
- Vectorized metric computation avoids slow per-row Python loops across large series

- Deterministic seeding supports reproducible experiments (a core MLflow-tracking goal)
- Minimal direct code footprint yet critical foundation

CONS

- Version-sensitive (hence the `<2.0` pin) — upgrading is a coordinated effort across the whole requirements.txt
- No native labeled-axis support (that's pandas' job) — using raw NumPy for tabular logic would be error-prone

RESOURCES

- https://numpy.org/doc/stable/

scikit-learn

WHAT IS IT?

scikit-learn is Python's general-purpose machine learning library, providing standardized estimators, preprocessing utilities, and model-selection tools.

PURPOSE IN THIS PROJECT

scikit-learn supplies two of the 7 trained models — `LinearRegressionModel` and `RandomForestModel` (`src/models/baseline_models.py` , `src/models/tree_models.py`) — both wrapped behind the shared `BaseModel` interface and trained/evaluated identically to LightGBM, XGBoost, ARIMA, and LSTM inside the walk-forward CV.

KEY FEATURES USED

- `LinearRegression` as an interpretable baseline model
- `RandomForestRegressor` (config: `n_estimators: 100` , `max_depth: 15` , `min_samples_split: 5`) as a tree-ensemble baseline distinct from LightGBM/XGBoost
- Consistent `fit` / `predict` estimator API, which `BaseModel` mirrors so all 7 models are interchangeable in `train_pipeline.py`
- Likely metric/utility helpers (e.g., preprocessing scalers) supporting non-tree models

DETAILED DESCRIPTION

scikit-learn's role is to provide dependable, well-understood baseline and ensemble models against which the more specialized gradient-boosting (LightGBM/XGBoost) and deep-learning (LSTM) models are benchmarked. Because `config.yaml` enables `linear_regression` and `random_forest` alongside the boosted-tree models, the project can show whether the added complexity of LightGBM/XGBoost/LSTM actually earns its keep in MAPE terms — a linear model and a vanilla random forest set a floor. Its estimator API convention (`.fit(X, y)` / `.predict(X)`) is effectively the design template the project's own `BaseModel` abstraction follows, letting `model_factory.py` build a uniform list of models regardless of underlying library (sklearn, lightgbm, xgboost, statsmodels, or torch). scikit-learn's ubiquity and stability also make it the safe default when a model needs to "just work" without hyperparameter fuss, which is exactly the role Linear Regression plays as a sanity-check baseline in a 7-model comparison.

ALTERNATIVES

1. **statsmodels' OLS** - could replace `LinearRegression` for inference-focused stats (p-values, confidence intervals), but sklearn's simpler predict-only API suits a pure-forecasting baseline better.
2. **Custom gradient descent** - would be pure YAGNI here; sklearn's implementation is well-tested and there's no researched need to reinvent it.
3. **H2O.ai / Spark MLlib** - overkill distributed ML frameworks unnecessary at this project's scale.

4. **LightGBM's own RandomForest boosting mode** - could replace sklearn's RandomForest, but the project already uses LightGBM for gradient boosting proper, so keeping sklearn's RF as a genuinely independent baseline (different library, different bias) is more informative.

PROS

- Extremely stable, well-documented, minimal-surprise API used as the design template for the project's own model abstraction
- Provides a fast, interpretable linear baseline and an independent random forest to contextualize the gradient-boosted models
- Zero GPU/special infra requirements
- Easy to fold into the walk-forward CV loop identically to other models
- Battle-tested numerical correctness

CONS

- Random Forest here is slower to train than LightGBM/XGBoost at comparable accuracy for tabular demand data
- Fewer built-in regularization/tuning knobs than LightGBM/XGBoost for this use case
- Not GPU-accelerated (irrelevant at this project's scale, but a limitation generally)

RESOURCES

- https://scikit-learn.org/stable/

LightGBM

WHAT IS IT?

LightGBM is Microsoft's gradient-boosted decision tree framework optimized for speed and memory efficiency via histogram-based splitting and leaf-wise tree growth.

PURPOSE IN THIS PROJECT

LightGBM (`LightGBMModel` in `src/models/tree_models.py`) is one of the 7 models trained per walk-forward fold in `src/training/train_pipeline.py` , configured in `config.yaml` with `learning_rate: 0.05` , `num_leaves: 31` , `max_depth: 7` , `n_estimators: 500` , `subsample: 0.8` , `colsample_bytree: 0.8` . Its feature importances also feed `reports/feature_importance.csv` and the top-5 features monitored for drift.

KEY FEATURES USED

- Gradient-boosted regression trees (`LGBMRegressor` -style estimator) trained on the leakage-safe ~40-feature set
- `min_child_samples: 20` and subsampling/column-sampling (`subsample` , `colsample_bytree`) for regularization against overfitting on limited top-N series data
- Leaf-wise growth via `num_leaves: 31` for efficient use of `max_depth: 7`
- Feature importance extraction, used both for reporting and to select the top-5 drift-monitored features
- One global model trained across all series (per `ARCHITECTURE.md` 's "cross-series learning" design decision), relying on series/store/item categorical encodings + per-series lag features for series-specific context

DETAILED DESCRIPTION

LightGBM is likely the primary/best-performing model candidate in this platform, given it's a modern industry-standard choice for tabular demand forecasting competitions (including M5 itself, where LightGBM-based solutions dominated the Kaggle leaderboard). It's trained as one global model across all series rather than per-series, which `ARCHITECTURE.md` explains as deliberate: cross-series learning plus a single artifact to manage, with per-series context supplied through categorical encodings and lag/rolling features. Its speed (histogram binning, leaf-wise growth) is what makes 3-fold walk-forward CV across 7 models tractable within the project's stated <15 minute training budget on a top-N subset of M5's otherwise ~58M-row dataset. Feature

importances produced by LightGBM double as the signal for the drift-detection module — the "top-5 most important features" that Evidently/KS testing monitors are derived from whichever model wins the model-selection step, and LightGBM's importance metric is a natural, cheap byproduct of training already computed during CV.

ALTERNATIVES

1. **XGBoost** - directly compared in the same 7-model roster; XGBoost uses level-wise growth and has traditionally been more conservative/regularized by default, LightGBM chosen for speed at this data volume, XGBoost kept as ensemble diversity/comparison.
2. **CatBoost** - handles categorical features natively without manual encoding, could simplify the store/item categorical handling, but wasn't included, likely to keep the dependency list smaller since two boosted-tree libraries already provide comparison.
3. **sklearn GradientBoostingRegressor** - much slower, no leaf-wise growth or native categorical/histogram optimizations; not competitive at this dataset size.
4. **Prophet / NeuralProphet** - purpose-built time-series forecasters, but the project's design deliberately treats forecasting as a supervised tabular regression problem (with lag/rolling features) rather than a decomposition-based time-series model, so tree-based tabular learners fit better here.

PROS

- Fast training enabling multiple models × multiple folds within a demonstrable time budget
- Strong tabular/regression performance on demand-forecasting-style feature sets (proven on M5 itself)
- Feature importance output reused for drift monitoring, avoiding duplicate computation
- Handles the ~40 engineered features (temporal, lag, rolling, decomposition, business, hierarchical) well without heavy preprocessing
- Regularization knobs (subsample, colsample, min_child_samples) directly tunable via `config.yaml`

CONS

- Leaf-wise growth can overfit on small/noisy series without careful regularization (mitigated by `min_child_samples`/subsampling here)
- Requires manual recursive forecasting logic (per `ARCHITECTURE.md`, tree models predict one step and must regenerate features each step) — more complex than LSTM's direct multi-horizon output
- Sensitive to hyperparameter choices; the config's fixed values aren't tuned via search in this project

RESOURCES

- https://lightgbm.readthedocs.io/

XGBoost

WHAT IS IT?

XGBoost is a widely used gradient-boosted decision tree library known for regularized, level-wise tree growth and strong out-of-the-box performance on tabular data.

PURPOSE IN THIS PROJECT

XGBoost (`XGBoostModel` in `src/models/tree_models.py`) is another of the 7 models compared per fold, configured with `learning_rate: 0.1`, `max_depth: 6`, `n_estimators: 500`, `subsample: 0.8`, `colsample_bytree: 0.8` in `config.yaml`, providing a second gradient-boosting perspective alongside LightGBM.

KEY FEATURES USED

- `XGBRegressor`-style gradient boosting on the same leakage-safe feature set as LightGBM

- Row/column subsampling (`subsample` , `colsample_bytree`) for regularization
- Level-wise tree growth with `max_depth: 6` (contrasting LightGBM's leaf-wise `num_leaves` -controlled growth)
- Same one-global-model-across-series design and recursive multi-step forecasting as LightGBM

DETAILED DESCRIPTION

XGBoost serves as a deliberate point of comparison against LightGBM — both are gradient-boosted tree ensembles trained on identical folds and features, but with different tree-growth strategies (level-wise vs. leaf-wise) and historically different default regularization behavior. Including both in the 7-model roster lets the walk-forward CV's model-selection step (lowest mean MAPE, RMSE tiebreak) empirically decide which boosting implementation generalizes better on this particular M5 subset, rather than assuming one is universally superior. Like LightGBM, XGBoost is trained as one global model with series/store/item categorical context baked into features, and its forecasts for the horizon are generated recursively (one-step prediction, then feature regeneration) exactly as described in `ARCHITECTURE.md` . XGBoost's long track record in forecasting/Kaggle competitions (including earlier M5-adjacent competitions) makes it a natural, low-risk second gradient-boosting entry that most practitioners will recognize and trust.

ALTERNATIVES

1. **LightGBM** - the direct comparison already in this project; LightGBM is generally faster on large datasets, XGBoost sometimes more robust on smaller/noisier ones — having both provides empirical coverage rather than guessing.
2. **CatBoost** - would add native categorical handling, but wasn't chosen, likely redundant given two boosting libraries already exist.
3. **sklearn's HistGradientBoostingRegressor** - a lighter-weight, no-extra-dependency histogram-based booster; could replace XGBoost for a smaller footprint, but lacks XGBoost's maturity/tuning ecosystem and wide familiarity.
4. **LinearGAM / pyGAM** - interpretable additive models, an alternative baseline, but not chosen since Linear Regression already covers the "simple, interpretable" baseline role.

PROS

- Well-regularized, competition-proven performance on tabular regression tasks
- Provides genuine architectural diversity vs. LightGBM (level-wise vs leaf-wise growth) for the model-selection comparison to be meaningful
- Mature, stable Python API, easy to wrap in `BaseModel`
- Handles the same ~40 feature set without additional preprocessing
- Subsampling config mirrors LightGBM's, keeping the comparison apples-to-apples

CONS

- Typically slower to train than LightGBM on larger datasets (less of an issue at this project's capped top-N scale)
- Also relies on manual recursive forecasting (no native multi-horizon output, unlike the LSTM)
- Another hyperparameter set to maintain in `config.yaml` alongside LightGBM's

RESOURCES

- https://xgboost.readthedocs.io/

statsmodels (ARIMA)

WHAT IS IT?

statsmodels is a Python library for statistical modeling and testing, including classical time-series methods like ARIMA, seasonal decomposition, and ACF/PACF analysis.

PURPOSE IN THIS PROJECT

statsmodels backs `ARIMAModel` in `src/models/statistical_models.py`, one of the 7 compared models, configured with `auto_order: true` (small AIC grid search) and `seasonal: false` in `config.yaml`. It's also used directly in `dashboard/server.py`'s `/api/eda/{series_id}` endpoint via `statsmodels.tsa.seasonal.seasonal_decompose` and `statsmodels.tsa.stattools.acf` / `pacf` for the EDA page's decomposition and autocorrelation charts.

KEY FEATURES USED

- ARIMA model fitting per series (each series gets its own fit, per `ARCHITECTURE.md`, since statistical models are scale-sensitive)
- ADF (Augmented Dickey-Fuller) test to determine differencing order `d`
- Small AIC grid search over candidate `(p,d,q)` orders instead of a full auto-ARIMA search (explicit design tradeoff vs. pmdarima, documented in `ARCHITECTURE.md`)
- `seasonal_decompose` for trend/seasonal/residual decomposition surfaced on the EDA dashboard page
- `acf` / `pacf` for autocorrelation plots on the EDA page

DETAILED DESCRIPTION

statsmodels provides the project's one genuinely "classical" time-series model, ARIMA, contrasted against the tabular-regression approach used by the tree models and the sequence model used by LSTM. Because ARIMA is fit per-series rather than globally, and is a true multi-step forecaster (unlike the tree models' recursive one-step approach), it represents a fundamentally different forecasting paradigm in the 7-model comparison, testing whether classical statistical methods still compete with ML approaches on this data. The project explicitly avoids `pmdarima` (a common auto-ARIMA-order-search library) in favor of a hand-rolled ADF + small AIC grid, a documented tradeoff in `ARCHITECTURE.md`: "ADF picks d; 4 candidate orders by AIC. pmdarima adds a fragile dependency for marginal gain at this scale" — trading away wide hyperparameter search flexibility for one fewer, more stable dependency. Beyond forecasting, statsmodels also directly powers exploratory data analysis in the dashboard: the `/api/eda/{series_id}` endpoint decomposes each series into trend/seasonal/residual components and computes ACF/PACF for the frontend `Eda.jsx` page to visualize.

ALTERNATIVES

1. **pmdarima (auto_arima)** - would automate ARIMA order search more thoroughly, explicitly rejected here as "a fragile dependency for marginal gain at this scale" per `ARCHITECTURE.md`.
2. **Facebook Prophet** - handles seasonality/holidays natively with less manual tuning, but the project already engineers holiday/seasonal features manually (via `holidays` library and decomposition features) for the ML models, so adding Prophet would duplicate that logic in a different paradigm.
3. **Exponential Smoothing (Holt-Winters)** - actually present in the codebase as `ExpSmoothingModel` but disabled (`exponential_smoothing.enabled: false` in `config.yaml`), likely because ARIMA + tree models + LSTM already give sufficient coverage.
4. **sktime / Darts** - unified time-series ML frameworks that wrap ARIMA, LSTM, and boosting under one API; would reduce custom glue code, but the project's `BaseModel` abstraction already provides that unification without an extra heavyweight dependency.

PROS

- True multi-step statistical forecaster, complementing the tree models' recursive approach and LSTM's direct multi-horizon approach
- ADF+AIC grid search is fast and dependency-light compared to full auto-ARIMA search
- Reused for EDA (decomposition, ACF/PACF) beyond just forecasting, avoiding a second library for those stats
- Well-established, interpretable statistical foundation
- Per-series fitting is straightforward to reason about (scale-sensitivity handled naturally)

CONS

- Per-series fitting doesn't scale as gracefully as the tree models' one-global-model approach if `n_series` grows large
- Limited hyperparameter search (only 4 AIC candidate orders) vs. what pmdarima would explore
- `seasonal: false` in config means seasonal ARIMA (SARIMA) patterns aren't modeled directly, relying instead on engineered seasonal features elsewhere

RESOURCES

- https://www.statsmodels.org/stable/tsa.html

PyTorch

WHAT IS IT?

PyTorch is an open-source deep learning framework providing tensor computation with GPU acceleration and automatic differentiation for building neural networks.

PURPOSE IN THIS PROJECT

PyTorch implements `LSTMModel` (`src/models/lstm_model.py`), the 7th model in the comparison — a sequence-to-sequence LSTM (`_Seq2Seq(nn.Module)` with `nn.ModuleList` layers) configured in `config.yaml` with `input_sequence_length: 60` , `lstm_units: [64, 32]` , `dropout: 0.2` , `batch_size: 32` , `epochs: 30` , `learning_rate: 0.001` . Its import is deferred in `model_factory.py` so the rest of the pipeline still runs if torch isn't installed.

KEY FEATURES USED

- `nn.Module` / `nn.ModuleList` for a custom stacked LSTM architecture (64 then 32 units)
- Dropout regularization (`dropout: 0.2`) between LSTM layers
- Direct multi-horizon output design (60-step input → 30-step output in one forward pass, per `ARCHITECTURE.md` , avoiding compounding recursive error)
- Mini-batch training (`batch_size: 32`) over `epochs: 30` with Adam-style optimization at `learning_rate: 0.001`
- Per-series min-max scaling (scale-sensitive, per `ARCHITECTURE.md` 's design notes) before sequence construction

DETAILED DESCRIPTION

PyTorch provides the project's only deep-learning / neural-sequence model, contrasted against the classical statistical (ARIMA), tabular-tree (LightGBM/XGBoost/RandomForest), and simple-baseline (Moving Average/Linear Regression) approaches — giving the 7-model roster genuine methodological diversity. Its "direct multi-horizon" design (predicting all 30 forecast steps in a single forward pass from a 60-step lookback window) is explicitly chosen over recursive step-by-step prediction to avoid compounding error within the horizon, rolling forward in 30-step chunks only for windows longer than the direct horizon. Because torch is a heavier, optional dependency (large install, potential GPU/CPU build mismatches), the codebase treats it defensively: `model_factory.py` wraps the LSTM import in a try/except and logs a warning ("LSTM skipped (torch unavailable)") rather than crashing the whole pipeline, letting the other 6 models still run. Per `ARCHITECTURE.md` , LSTM (like ARIMA) is fit per-series with its own min-max scaling range, since neural network training is highly scale-sensitive — unlike the tree models' single global model with categorical series encodings, each series gets an individually normalized input space for the LSTM.

ALTERNATIVES

1. **TensorFlow/Keras** - equally capable for LSTM sequence modeling; PyTorch likely chosen for its more Pythonic, imperative API which suits building a small custom `_Seq2Seq` module directly.
2. **statsmodels' state-space / Kalman-filter models** - classical alternative to neural sequence modeling, already represented by ARIMA in this project, so LSTM's inclusion targets nonlinear pattern capture ARIMA can't.

3. **Temporal Fusion Transformer / N-BEATS (via pytorch-forecasting or Darts)** - more sophisticated architectures, but a hand-rolled small LSTM is simpler to reason about, faster to train in the <15-minute budget, and doesn't add a heavyweight forecasting-framework dependency.
4. **Skip deep learning entirely** - a legitimate simplification (torch is genuinely optional here, degrading gracefully to 6 models), but keeping it demonstrates DL competitiveness against classical/tree approaches, which is valuable for a platform meant to showcase the full lifecycle.

PROS

- Provides direct multi-horizon forecasting, structurally different from every other model's step-wise approach
- Graceful optional-dependency handling means the whole pipeline doesn't break without torch/GPU
- Custom, small architecture (2 LSTM layers) trains quickly enough to fit the project's time budget on CPU
- Per-series scaling handles heterogeneous demand magnitudes well
- Demonstrates deep learning integration in an otherwise classical-ML-heavy platform

CONS

- Heaviest dependency in requirements.txt (large install size), justifying the defensive optional-import pattern
- Per-series training (rather than global) means training cost scales with `n_series`
- Requires more careful data prep (scaling, sequence windowing) than the tree models
- Less interpretable than tree feature importances or ARIMA coefficients

RESOURCES

- https://pytorch.org/docs/stable/

MLflow

WHAT IS IT?

MLflow is an open-source platform for managing the ML lifecycle: experiment tracking, model packaging, and a model registry with stage transitions (Staging/Production).

PURPOSE IN THIS PROJECT

MLflow (`src/mlflow_utils/tracker.py`) tracks every fold of every model's training run (params/metrics/timing) against a SQLite backend (`sqlite:///mlflow.db` , `config.yaml`), registers the best model in the `DemandIQ_Demand_Forecast_Model` registry, and promotes it to Production only if it beats the incumbent by >2% test MAPE (`promotion_improvement_threshold: 0.02`).

KEY FEATURES USED

- `mlflow.set_tracking_uri` / `mlflow.set_experiment` pointing at the `DemandIQ-M5` experiment on a SQLite store
- `mlflow.start_run(run_name=f"{model_name}_fold{fold}")` per model per CV fold, logging params/metrics/tags/artifacts (`mlflow.log_params` , `mlflow.log_metrics` , `mlflow.set_tag` , `mlflow.log_artifact`)
- `MlflowClient` (`from mlflow.tracking import MlflowClient`) for registry operations (staging/production promotion)
- Run tags (`environment: development` , `project: demand_forecasting`) applied per run for filtering in the MLflow UI
- Registration + promotion gate logic comparing new candidate's test MAPE against the current Production version

DETAILED DESCRIPTION

MLflow is the experiment-tracking and model-registry backbone that makes the 7-models × 3-folds walk-forward comparison auditable — every fold's parameters, metrics, and timing are logged as a distinct run (`{model_name}_fold{fold}`), so the model-selection step (lowest mean MAPE, RMSE tiebreak) is fully traceable after the fact via `mlflow ui` . The registry then formalizes

the promotion decision: a new best model is only promoted to Production if it beats the current Production version's test MAPE by more than the configured 2% threshold, explicitly avoiding "churn from noise-level improvements" (per `ARCHITECTURE.md`), with losing candidates parked in Staging rather than discarded. The SQLite backend (`mlflow.db`) is a deliberate zero-infra choice — per `ARCHITECTURE.md`, "Zero-infra local tracking; swap `mlflow.tracking_uri` for a server later" — meaning the same tracking code would work against a hosted MLflow server (e.g., Postgres-backed) just by changing one config string, without any code changes. The dashboard's `/api/mlflow` endpoint and `MlflowTracking.jsx` page surface this tracking data (via a direct `import mlflow` in `dashboard/server.py`) so users can inspect experiment history without leaving the React UI, and `retraining/orchestrator.py` re-invokes the same tracker functions whenever automated retraining occurs.

ALTERNATIVES

1. **Weights & Biases (wandb)** - richer hosted dashboards/visualization, but requires external account/network dependency; MLflow's local SQLite mode keeps this project fully offline-capable.
2. **Neptune.ai / Comet ML** - similar hosted experiment-tracking SaaS tools; same offline/dependency tradeoff as wandb, not chosen for the same reason.
3. **DVC (Data Version Control) experiments** - git-based experiment tracking, would fit a git-centric workflow but doesn't offer MLflow's model registry/stage-promotion semantics used here.
4. **Custom JSON/CSV logging** - simplest possible alternative (and partially present via `reports/model_comparison.csv`), but lacks a registry, run comparison UI, and promotion workflow that MLflow provides out of the box.

PROS

- Zero-infra SQLite backend keeps the whole platform runnable locally with no external services
- Registry + promotion-threshold logic directly implements a real MLOps practice (avoiding noise-driven redeployments)
- Every fold of every model is individually inspectable via `mlflow ui`
- Same tracking code path is reused by both the initial pipeline and automated retraining
- Swappable backend (`tracking_uri`) means the design scales to a real server later without code changes

CONS

- SQLite backend isn't suitable for concurrent/multi-user production tracking (acceptable here since this is single-user local dev)
- Adds another artifact file (`mlflow.db`) to the project's contract that must stay in sync with `reports/` / `models/` artifacts
- MLflow's own dependency footprint and occasional breaking changes across versions require the `>=2.10` floor

RESOURCES

- https://mlflow.org/docs/latest/index.html

Evidently

WHAT IS IT?

Evidently is an open-source Python library for evaluating, testing, and monitoring ML models and data, best known for its data drift and data quality HTML reports.

PURPOSE IN THIS PROJECT

Evidently (`src/monitoring/drift_detector.py`) optionally renders an HTML drift report (`reports/drift_report.html`) as a supplement to the KS-test-based drift core; the code tries both its ≥0.7 API (`from evidently import Report`, `from evidently.presets import DataDriftPreset`) and legacy API (`from evidently.report import Report`, `from evidently.metric_preset import DataDriftPreset`) for version resilience.

KEY FEATURES USED

- `Report` / `DataDriftPreset` for generating a comparative reference-vs-current data drift HTML report
- Dual-API compatibility handling (new `evidently.presets` vs legacy `evidently.report` / `evidently.metric_preset`) to tolerate library version differences
- Applied to the top-N most important features (from LightGBM/whichever model wins) plus the target column
- Rendered and served via `dashboard/server.py`'s `/api/drift/html` endpoint (`HTMLResponse`) to the `DriftMonitoring.jsx` page

DETAILED DESCRIPTION

Evidently is explicitly positioned as an optional enhancement layer, not the core drift-detection mechanism — per `ARCHITECTURE.md`, "KS-test drift core, Evidently optional: scipy KS is stable across environments and versions; Evidently renders the pretty HTML report when importable (both its ≥0.7 and legacy APIs are tried)." This means the platform's actual drift-triggering logic (statistical significance, retraining triggers) never depends on Evidently being installed or working — it only affects whether a nicer visual report exists for humans to inspect. This defensive dual-API try/except pattern (catching import errors for both the new and legacy Evidently package layouts) reflects a real pain point with fast-moving ML tooling libraries: Evidently's API changed substantially between major versions, and the project chose to support both rather than pin to one exact version. The generated HTML report is served directly through FastAPI as raw HTML (`HTMLResponse`) and can be viewed either as a downloaded file or embedded/linked from the React dashboard's Drift Monitoring page, giving users Evidently's rich visual drift breakdown without the frontend needing to reimplement any of that visualization logic itself.

ALTERNATIVES

1. **scipy KS-test alone (no Evidently)** - this is literally the fallback path already built in; Evidently exists purely to prettify what the KS core already computes, so skipping it entirely loses only the HTML report, not functionality.
2. **whylogs / Deepchecks** - alternative data-quality/drift libraries with similar HTML/report output; not chosen, likely because Evidently's drift-preset report is a well-known, purpose-fit tool for exactly this KS-style comparison.
3. **Custom matplotlib/plotly drift plots** - would remove the Evidently dependency entirely at the cost of hand-building comparison visualizations; the project already uses matplotlib elsewhere, so this is a real "could simplify" option Evidently's optional nature partially concedes.
4. **Great Expectations** - focused more on data validation/expectations than drift reporting; a different tool for a different job (the project's own `src/validation/validator.py` already covers schema/quality validation).

PROS

- Purely additive/optional — its absence or breakage never blocks the core drift-detection pipeline
- Produces a polished, non-trivial-to-hand-build HTML drift report reused directly in the dashboard
- Dual-API handling makes the integration resilient to Evidently's own breaking changes
- Applied precisely to the features that matter (top-N importance + target), keeping report size manageable

CONS

- Library API instability (hence the dual-import workaround) makes it a maintenance burden relative to its optional value
- Adds a non-trivial dependency (`evidently>=0.4`) purely for a "nice to have" visualization
- HTML report generation could be slower/heavier than the core KS test for large feature sets

RESOURCES

- https://docs.evidentlyai.com/

FastAPI

WHAT IS IT?

FastAPI is a modern, high-performance Python web framework for building APIs, using type hints for request/response validation and automatic OpenAPI docs generation.

PURPOSE IN THIS PROJECT

FastAPI (`dashboard/server.py`) is the local dev API layer behind the React dashboard — it exposes ~15 GET/POST endpoints (`/api/status` , `/api/data-overview` , `/api/series` , `/api/eda/{series_id}` , `/api/model-comparison` , `/api/forecast` , `/api/forecast/{series_id}` , `/api/mlflow` , `/api/drift` , `/api/drift/html` , `/api/retrain-log` , `/api/retrain` , `/api/drift-check` , `/api/performance-check` , `/api/triggers` , `/api/upload`) that read pipeline artifacts and trigger retrain/drift actions, explicitly documented as "not a model-serving layer."

KEY FEATURES USED

- `FastAPI(title="DemandIQ API")` app instance with route decorators (`@app.get` , `@app.post`)
- `CORSMiddleware` configured with `allow_origins=["*"]` / `allow_methods=["*"]` to permit the Vite dev server (localhost:5173) to call the API
- `HTMLResponse` for serving Evidently's rendered drift report directly
- `UploadFile` / `python-multipart` for the `/api/upload` endpoint (custom CSV data upload)
- `HTTPException` for structured error responses (e.g., missing artifacts)

DETAILED DESCRIPTION

FastAPI serves as the thin, read-only bridge between the batch ML pipeline's file-based artifacts (parquet/CSV/JSON under `reports/` , `models/` , `data/processed/`) and the React frontend, per the "dashboard is a pure reader of this contract" design principle in `ARCHITECTURE.md` . Rather than serving live model predictions, nearly every endpoint reads a pre-computed artifact (`model_comparison.csv` , `forecast.csv` , `drift_report.json` , etc.) and reshapes it into JSON — the two exceptions being `/api/retrain` and `/api/drift-check` , which invoke `retrain()` and `detect_drift()` from `src/retraining/orchestrator.py` and `src/monitoring/drift_detector.py` respectively, keeping the pipeline and UI decoupled while still allowing on-demand actions. FastAPI's type-hinted route signatures and automatic request parsing make endpoints like `/api/eda/{series_id}` and `/api/forecast/{series_id}` straightforward path-parameterized lookups, while its built-in OpenAPI/Swagger docs generation (implicit, not shown in the excerpts read) gives free API documentation without extra tooling. The `dashboard/server.py` docstring explicitly labels this "local dev API only," and the wide-open CORS policy (`allow_origins=["*"]`) reinforces that this is not meant to be internet-facing — a reasonable simplification for a project whose scope is a local, single-user demonstration platform.

ALTERNATIVES

1. **Flask** - simpler, more minimal micro-framework; lacks FastAPI's built-in Pydantic validation and automatic docs, less suited to a project already using type hints throughout its Python codebase.
2. **Django REST Framework** - far heavier (ORM, admin panel, auth stack) for what's essentially a read-only artifact server; would be significant over-engineering here.
3. **Node.js/Express backend** - would unify the JS frontend/backend language, but forfeits direct in-process access to the Python ML pipeline (pandas artifacts, MLflow client, drift detector) that FastAPI gets for free by living in the same language/runtime.
4. **Direct static file serving (no API)** - the frontend could fetch static JSON/CSV files directly from a static server, but this project needs POST actions (`/api/retrain` , `/api/drift-check` , `/api/upload`) that require server-side logic, which static files can't provide.

PROS

- Fast to build read-heavy endpoints thanks to type-hint-driven request/response handling
- Native Python means the API layer shares the exact same pandas/MLflow/pipeline code as the batch pipeline with no serialization boundary
- Async-capable (uvicorn ASGI) even though this project's usage is largely synchronous file reads

- Automatic interactive API docs (Swagger UI) come free from route type hints
- CORS middleware trivially unblocks the separate Vite dev server during local development

CONS

- Wide-open CORS (`allow_origins=["*"]`) is explicitly a dev-only shortcut, not production-safe
- No authentication/authorization on any endpoint, consistent with "local dev only" scope but a real gap if ever exposed
- As a pure artifact reader, it duplicates some logic (e.g., re-deriving JSON shapes) that could drift from the pipeline's own report formats if not kept in sync

RESOURCES

- https://fastapi.tiangolo.com/

uvicorn

WHAT IS IT?

uvicorn is a lightning-fast ASGI server implementation for Python, commonly used to run FastAPI (and other ASGI) applications.

PURPOSE IN THIS PROJECT

uvicorn runs the FastAPI dashboard server, per the README's quick-start: `uvicorn dashboard.server:app --port 8000`, serving the API that the React dashboard (on port 5173 via Vite) calls.

KEY FEATURES USED

- ASGI app serving (`dashboard.server:app`) with a configurable port (`--port 8000`)
- Local development server mode (no production process manager like Gunicorn layered on top, matching the "local dev only" framing)

DETAILED DESCRIPTION

uvicorn is the minimal, standard choice for running any FastAPI application locally — it's an ASGI server built on `uvloop` / `httptools` for speed, and pairing it with FastAPI is the framework's own recommended default. In this project it's invoked directly via the `uvicorn` CLI rather than embedded in a Docker/production launch script, consistent with the whole platform's local-dev, zero-infra philosophy (no Kubernetes, no reverse proxy, no process supervisor described anywhere in the read files). Its role is entirely operational — it doesn't touch any ML logic, but without it the FastAPI `app` object in `dashboard/server.py` has no way to actually accept HTTP connections from the browser or from axios calls made by the React frontend.

ALTERNATIVES

1. **Gunicorn + Uvicorn workers** - the standard production pairing (Gunicorn as process manager, Uvicorn workers for ASGI), would add resilience/multi-worker support; unnecessary for a single-user local dev API.
2. **Hypercorn** - another ASGI server supporting HTTP/2; no need here since this is a simple local REST API over HTTP/1.1.
3. **Daphne** - Django Channels' ASGI server; irrelevant since this project doesn't use Django.

PROS

- Simple one-command local server startup matching the project's zero-infra dev philosophy
- Fast enough for the API's actual workload (reading small parquet/CSV/JSON artifacts)
- Standard, well-documented pairing with FastAPI, minimal configuration needed

CONS

- Single-process by default (no multi-worker concurrency) — a non-issue for local single-user dev, but not production-ready as-is
- No built-in HTTPS/reverse-proxy handling (expected to be layered on separately if ever deployed)

RESOURCES

- https://www.uvicorn.org/

React

WHAT IS IT?

React is a component-based JavaScript library (now on major version 19) for building user interfaces through declarative, composable UI components.

PURPOSE IN THIS PROJECT

React 19.2 (`frontend/package.json`) is the entire frontend framework for the DemandIQ dashboard — 7 routed pages (Overview, Data Overview, EDA, Model Comparison, Forecasting, MLflow Tracking, Drift Monitoring) plus shared components (`Sidebar` , `ChartCard` , `MetricCard` , `StatusBadge` , `InfoTooltip` , `Loading` / `ErrorBox`), rendered client-side and consuming the FastAPI backend's JSON via axios.

KEY FEATURES USED

- Function components with hooks (implied by modern React 19 usage; no class components in the file listing)
- Component composition — shared `ChartCard` / `MetricCard` / `StatusBadge` components reused across all 7 pages
- `react-dom` for client-side rendering into the DOM (`main.jsx` entry point)
- Declarative routing integration via `react-router-dom` 's `<Routes>` / `<Route>` inside `App.jsx`
- A custom `useApi` hook (seen imported in `ModelComparison.jsx`) wrapping axios calls with presumable loading/error state

DETAILED DESCRIPTION

React structures the dashboard as a set of independent, routed pages that each fetch their own data and render it via Recharts — this matches the "dashboard is a pure reader" architecture from `ARCHITECTURE.md` , where the UI has no business logic beyond display and two action buttons (retrain, drift-check). The component layout (`Sidebar` for navigation, small reusable `ChartCard` / `MetricCard` / `StatusBadge` / `InfoTooltip` building blocks) keeps each of the 7 page components focused purely on which data to fetch and which chart type to render. Choosing React 19 (very current at the time of this project) rather than an older major version suggests the project wants modern defaults (improved hooks ergonomics, React Compiler-adjacent tooling readiness) without needing any of React 19's more advanced features (Server Components, `use()` for data fetching) since this is a classic client-side SPA talking to a REST API, not an RSC/streaming setup. The overall frontend is intentionally small — no state management library, no CSS-in-JS framework visible, no test files listed under `frontend/src` — matching a project whose primary complexity investment is on the ML/backend side, with the frontend purely a thin visualization layer.

ALTERNATIVES

1. **Vue 3** - comparable component model and reactivity system with a gentler learning curve; not chosen, likely simply team/ecosystem familiarity with React plus Recharts' strong React-first API.
2. **Svelte/SvelteKit** - compiles away the framework at build time for smaller bundles; less relevant for a locally-run dev dashboard where bundle size isn't a real constraint.
3. **Plain HTML + Chart.js** - would work for static charts but loses the componentized, routed, reusable-piece architecture (`ChartCard` , `MetricCard` reused across 7 pages) that React naturally provides.

4. **Streamlit/Dash (Python-native dashboards)** - would avoid a separate JS toolchain entirely and reuse Python throughout; not chosen, likely because the project wants a real, polished, customizable SPA (7 distinct routed pages, custom hooks, Recharts) rather than a quick Python-only dashboard generator.

PROS

- Component reuse (`ChartCard` , `MetricCard` , `StatusBadge`) keeps 7 dashboard pages consistent and DRY
- Huge, mature ecosystem (Recharts, react-router-dom, axios) with first-class React integrations
- Declarative rendering simplifies keeping charts in sync with fetched API state
- React 19 gives current, well-supported hooks-based patterns
- Clean separation from the backend (pure JSON consumer) matches the project's decoupled architecture goal

CONS

- No TypeScript despite `@types/react` present, forfeiting compile-time safety on props/API shapes
- No visible state management or data-caching layer (e.g., no React Query) — each page seems to manage its own fetch/loading/error state via a custom hook
- Client-side-only rendering means slower first paint / no SSR (not a real concern for a local internal dashboard)

RESOURCES

- https://react.dev/

Vite

WHAT IS IT?

Vite is a modern frontend build tool providing an extremely fast dev server (native ES modules + esbuild) and an optimized production bundler (Rollup-based).

PURPOSE IN THIS PROJECT

Vite (`frontend/vite.config.js` , package version `^8.2.0`) is the build tool and dev server for the React dashboard, configured with `@vitejs/plugin-react` for JSX/Fast Refresh support. `npm run dev` serves the app at `http://localhost:5173` (per README), and `npm run build` / `npm run preview` handle production bundling and preview.

KEY FEATURES USED

- `defineConfig({ plugins: [react()] })` — minimal config relying on Vite's sensible defaults
- `@vitejs/plugin-react` for JSX transformation and React Fast Refresh (HMR) during development
- Dev server (`vite`), production build (`vite build`), and preview server (`vite preview`) npm scripts

DETAILED DESCRIPTION

Vite is used here in its simplest, near-default configuration — a single plugin (`@vitejs/plugin-react`) and no custom aliasing, proxying, or environment-variable wiring visible in `vite.config.js` . This matches a project where the frontend's job is small and self-contained: 7 pages, a handful of shared components, and API calls to a separately-run FastAPI server on port 8000 (no dev-server proxy needed since CORS is opened wide on the backend instead). Vite's near-instant HMR (hot module replacement) during `npm run dev` is what makes rapid iteration on chart layouts/dashboard pages practical, especially valuable given how many distinct pages (7) and chart types this dashboard renders. Its production build step (`vite build`) uses Rollup under the hood to tree-shake and bundle the app for eventual static hosting, though this project's README doesn't describe any actual deployment of the built frontend beyond local `preview` .

ALTERNATIVES

1. **Create React App (CRA)** - the previous-generation standard, now effectively deprecated/unmaintained; Vite is the modern default replacement with vastly faster dev-server startup and HMR.
2. **Webpack (custom config)** - far more configurable but requires substantially more setup for the same JSX/HMR functionality Vite provides out of the box.
3. **Next.js** - would add SSR/routing/file-based-routing conventions, none of which this project needs since it's a client-only SPA with its own explicit `react-router-dom` routes.
4. **Parcel** - another zero-config bundler; comparable simplicity to Vite, but Vite has become the de facto standard for React SPAs and has first-party plugin support (`@vitejs/plugin-react`) this project uses directly.

PROS

- Near-instant dev server startup and HMR, ideal for iterating on 7 dashboard pages
- Minimal configuration needed (`plugins: [react()]` is the entire config)
- Modern default for React SPAs, well-supported, actively maintained
- Fast production builds via esbuild (dev) + Rollup (build)
- Pairs cleanly with `oxlint` and npm scripts already defined in `package.json`

CONS

- No SSR out of the box (not needed here, but a limitation if requirements changed)
- Version 8.x is very new/cutting-edge — potential for plugin ecosystem lag vs. more established major versions
- No custom proxy config to the FastAPI backend, relying instead on backend-side CORS (a minor architectural coupling)

RESOURCES

- https://vite.dev/

Recharts

WHAT IS IT?

Recharts is a composable charting library for React, built on D3 primitives but exposing declarative, JSX-native chart components.

PURPOSE IN THIS PROJECT

Recharts (`^3.10.1` , `frontend/package.json`) renders every chart in the dashboard — confirmed directly in `ModelComparison.jsx` , which imports `BarChart, Bar, XAxis, YAxis, CartesianGrid, Tooltip, ResponsiveContainer, Cell, LabelList` from `recharts` to visualize per-model metric comparisons (MAPE/RMSE/MAE/R²/SMAPE across the 7 models).

KEY FEATURES USED

- `BarChart` / `Bar` for model-comparison and feature-importance style bar charts
- `ResponsiveContainer` for charts that resize with the dashboard layout
- `Tooltip` and `CartesianGrid` for interactive, readable chart axes/hover details
- `Cell` for per-bar custom coloring (likely color-coding models/series, paired with a `seriesColors` helper module seen imported in `ModelComparison.jsx`)
- `LabelList` for direct value labels on bars (e.g., displaying exact MAPE values above bars)

DETAILED DESCRIPTION

Recharts is the sole charting library across all 7 dashboard pages, chosen for its idiomatic React/JSX component API — charts are composed declaratively (`<BarChart><Bar/><XAxis/>...</BarChart>`) rather than imperatively configured like Chart.js or raw D3, fitting naturally into React's component model and the project's existing `ChartCard` wrapper component pattern. Given the pages involved — Model Comparison (bar charts of 5 metrics × 7 models), Forecasting (presumably line charts of forecast vs. actual), EDA (decomposition/ACF line charts), Drift Monitoring (distribution comparisons) — Recharts' broad primitive set (Bar, Line, Area, Composed charts) covers all of these needs from one library without mixing charting tools. The presence of a custom `seriesColors` and `metricInfo` / `METRIC_LABEL` helper modules (imported alongside Recharts in `ModelComparison.jsx`) shows the project layers its own semantic color/label mapping on top of Recharts' primitives, keeping domain knowledge (which metric is "primary," which color represents which model) in application code rather than the charting library.

ALTERNATIVES

1. **Chart.js (with react-chartjs-2)** - canvas-based, often faster for very large datasets, but less naturally declarative/composable in JSX than Recharts' SVG-based component model.
2. **D3.js directly** - maximum flexibility and customization, but far more code/complexity for standard bar/line charts than this dashboard's needs justify.
3. **Victory** - another React-native charting library with a similar declarative API; comparable choice to Recharts, likely a toss-up decided by team familiarity.
4. **Plotly.js (react-plotly.js)** - powerful interactive charts with built-in zoom/pan, heavier bundle size than needed for straightforward bar/line dashboard visualizations.

PROS

- Declarative JSX API matches React's component model, easy to compose within `ChartCard` wrappers
- SVG-based rendering gives crisp charts and easy custom styling (`Cell` per-bar coloring)
- Covers all chart types this dashboard needs (bar, presumably line/area) from a single library
- Responsive out of the box via `ResponsiveContainer`
- Actively maintained, React 19-compatible major version (3.x)

CONS

- SVG rendering can lag on very large datasets (not a concern here given top-N series and limited data points)
- Less flexible for highly custom/novel visualizations than raw D3
- Bundle size larger than a minimal canvas-based alternative like Chart.js for simple charts

RESOURCES

- https://recharts.org/

axios

WHAT IS IT?

axios is a promise-based HTTP client for the browser and Node.js, commonly used for making API requests from frontend applications.

PURPOSE IN THIS PROJECT

axios (`^1.19.0`, `frontend/package.json`) handles all HTTP calls from the React dashboard to the FastAPI backend, wrapped in `frontend/src/api.js` and consumed by pages via the custom `useApi` hook (seen imported in `ModelComparison.jsx`).

KEY FEATURES USED

- Centralized API client module (`frontend/src/api.js`) rather than scattering raw `fetch` calls across pages
- Promise-based GET requests to the ~15 FastAPI endpoints (`/api/status` , `/api/model-comparison` , `/api/forecast` , etc.)
- Likely POST requests for the action endpoints (`/api/retrain` , `/api/drift-check` , `/api/upload`)
- Consistent error/response handling surfaced through the `useApi` hook to each page's `Loading` / `ErrorBox` components

DETAILED DESCRIPTION

axios centralizes all backend communication behind one small `api.js` module and a `useApi` hook, so every one of the 7 dashboard pages fetches data through the same consistent interface rather than each page hand-rolling `fetch()` calls with its own error handling. This is a lightweight but meaningful architectural choice — it means changing the backend's base URL, adding auth headers, or adjusting timeout/retry behavior only requires touching one file. Given the FastAPI backend is CORS-open (`allow_origins=["*"]`) and runs on a separate port (8000) from the Vite dev server (5173), axios's straightforward promise API is sufficient without needing any of its more advanced features (interceptors for token refresh, request cancellation) that a more complex authenticated app might require. The pairing of a custom `useApi` hook with axios (rather than a full data-fetching library) reflects the project's minimalist frontend philosophy — enough abstraction to avoid repetition across 7 pages, not so much that it needs a caching/query library.

ALTERNATIVES

1. **Native `fetch` API** - available in all modern browsers with zero dependency cost; axios is used instead likely for its more ergonomic API (automatic JSON parsing, cleaner error handling on non-2xx responses) with minimal added weight.
2. **React Query (TanStack Query)** - would add caching, refetching, and stale-time management on top of axios; not used here, likely because dashboard data is read once per page load rather than needing sophisticated cache invalidation.
3. **SWR** - similar data-fetching/caching library to React Query; same reasoning as above for why it's absent.
4. **XMLHttpRequest directly** - the low-level primitive axios wraps; using it directly would mean reimplementing what axios already provides for no benefit.

PROS

- Simple, well-known promise-based API reduces boilerplate vs. raw `fetch`
- Centralizing calls in `api.js` + `useApi` keeps all 7 pages consistent
- Automatic JSON handling simplifies working with FastAPI's JSON responses
- Small footprint appropriate for this dashboard's modest data-fetching needs
- Mature, stable, widely understood library

CONS

- No built-in caching/deduplication (would need React Query/SWR for that if the dashboard grew more interactive)
- Adds a dependency for functionality native `fetch` could mostly cover
- No visible request cancellation/retry logic described in the read files

RESOURCES

- https://axios-http.com/docs/intro

react-router-dom

WHAT IS IT?

react-router-dom is the standard client-side routing library for React web applications, mapping URL paths to rendered components.

PURPOSE IN THIS PROJECT

react-router-dom (`^7.18.2`) drives navigation across the dashboard's 7 pages — `App.jsx` defines `<Routes>` / `<Route>` mappings for `/` (Overview), `/data` (DataOverview), `/eda` (Eda), `/models` (ModelComparison), `/forecast` (Forecasting), `/mlflow` (MlflowTracking), and `/drift` (DriftMonitoring), alongside the persistent `Sidebar` component.

KEY FEATURES USED

- `Routes` / `Route` components for declarative path-to-component mapping
- Flat, non-nested route structure (7 top-level routes, no nested/index routes visible)
- Presumably `Link` / `NavLink` inside `Sidebar.jsx` for navigation (not directly confirmed but standard usage pattern)
- Client-side navigation without full page reloads between dashboard sections

DETAILED DESCRIPTION

react-router-dom provides the navigational skeleton for the dashboard, letting a single-page React app expose 7 logically distinct views (data overview, EDA, model comparison, forecasting, MLflow tracking, drift monitoring) as if they were separate pages, each with its own URL, while sharing one persistent `Sidebar` for navigation between them. The route structure is intentionally flat and simple — no dynamic route params, no nested layouts beyond the sidebar/content split — matching the dashboard's straightforward, non-hierarchical information architecture. Using version 7.x (a major, API-changed generation from the older v6/v5 React Router) suggests the project adopted current best practices at build time rather than carrying legacy routing patterns forward. Because every route maps 1:1 to a top-level dashboard concern mirrored in `ARCHITECTURE.md` 's pipeline stages (data → EDA → models → forecast → MLflow → drift), the router structure itself documents the platform's conceptual pipeline stages as much as it handles navigation mechanics.

ALTERNATIVES

1. **TanStack Router** - newer, type-safe alternative with first-class TypeScript support; not chosen since the frontend is plain JSX, not TypeScript.
2. **Wouter** - a much smaller, minimal router; would suffice for 7 flat routes, but react-router-dom is the de facto standard with broader familiarity/documentation.
3. **No router (conditional rendering + local state)** - a legitimate simplification for only 7 static views, but loses shareable/bookmarkable URLs per dashboard section, which a real router provides essentially for free.
4. **Next.js file-based routing** - would require adopting the entire Next.js framework (SSR, file conventions) for a benefit (routing) this project gets more simply via a client-only SPA router.

PROS

- Standard, well-documented, minimal-config solution for 7 flat dashboard routes
- Bookmarkable/shareable URLs per dashboard section (e.g., linking directly to `/drift`)
- Clean separation of `Sidebar` navigation from routed page content
- No unnecessary complexity (no nested routes, loaders, or data APIs needed for this simple structure)

CONS

- v7's API has diverged notably from v5/v6, so documentation/examples online can be version-specific
- Overkill in theory for only 7 static routes (a simple `useState` -based view switcher could technically work), but the URL/bookmarking benefit justifies it
- No visible use of v7's newer data-loading APIs (loaders/actions) — router used purely for view switching

RESOURCES

- https://reactrouter.com/

Databases & Data Storage

SQLite (via MLflow — mlflow.db)

WHAT IS IT?

SQLite is a lightweight, serverless, file-based relational database engine, commonly used for local/embedded storage without a separate database server process.

PURPOSE IN THIS PROJECT

SQLite is MLflow's tracking backend in this project, configured as `tracking_uri: 'sqlite:///mlflow.db'` in `config.yaml`. It stores every experiment run's params/metrics/tags plus the model registry (staging/production stages) in a single local file, `mlflow.db`, listed as one of the project's core artifacts in `ARCHITECTURE.md`.

KEY FEATURES USED

- Single-file relational storage (`mlflow.db`) requiring no separate database server process
- MLflow's SQLAlchemy-based schema for runs, params, metrics, tags, and registered model versions
- Accessed both by the training pipeline (writing runs) and `mlflow ui --backend-store-uri sqlite:///mlflow.db` (reading, per README) and the dashboard's `/api/mlflow` endpoint

DETAILED DESCRIPTION

SQLite is used here purely as MLflow's storage engine rather than being interacted with directly by any project code — no raw SQL or `sqlite3` module usage appears anywhere in the read files. Its selection is explicitly justified in `ARCHITECTURE.md` as "Zero-infra local tracking; swap `mlflow.tracking_uri` for a server later," meaning the whole platform can track potentially hundreds of runs (7 models × 3 folds × however many pipeline executions, plus retraining runs) without requiring PostgreSQL/MySQL setup, Docker, or any network service. Because MLflow abstracts the actual database interaction behind its Python client API (`mlflow.start_run` , `mlflow.log_metrics` , `MlflowClient`), the choice of SQLite vs. a server-based DB is a one-line config change (`tracking_uri`) with zero code impact elsewhere — a deliberate design property called out directly in the architecture doc. This mirrors the project's overall "zero external credentials needed" philosophy (synthetic data fallback, no cloud services required) — a fully local, single-file database keeps the entire ML lifecycle runnable offline on a laptop.

ALTERNATIVES

1. **PostgreSQL (as MLflow backend)** - the standard production MLflow backend for multi-user/concurrent-write scenarios; explicitly what `ARCHITECTURE.md` suggests swapping to "later" — not needed for this single-user local project.
2. **MySQL** - another common MLflow-supported backend, same tradeoff as Postgres.
3. **File-store backend (MLflow's default local file tracking, no DB at all)** - MLflow's simplest default; SQLite was chosen instead likely for the registry's stronger relational query support (needed for the staging/production promotion logic) which the plain file store handles less robustly.
4. **A managed MLflow tracking service (Databricks-hosted MLflow)** - would remove all local infra concerns but requires an external account/network dependency, contrary to this project's offline-first design.

PROS

- Zero setup — no database server to install, configure, or keep running
- Single portable file (`mlflow.db`) that's easy to back up, inspect, or delete/regenerate
- Sufficient for the project's actual write concurrency (one local pipeline run at a time)
- Directly swappable for a server-based backend later with no code changes

CONS

- Not safe for concurrent multi-writer access (a real constraint if multiple pipeline runs ever executed simultaneously)
- Not suitable for a genuinely multi-user/production tracking scenario
- File can grow and become a single point of failure/corruption risk without backups

RESOURCES

- https://www.sqlite.org/docs.html

Parquet

WHAT IS IT?

Apache Parquet is a columnar, compressed binary file format optimized for efficient storage and fast analytical reads of tabular data.

PURPOSE IN THIS PROJECT

Parquet is the storage format for the two core intermediate pipeline artifacts: `data/processed/clean.parquet` (output of preprocessing) and `data/processed/features.parquet` (output of the ~40-feature engineering pipeline), per `ARCHITECTURE.md`'s artifacts contract. Both are written via pandas' `to_parquet` (using pyarrow as the engine) and read back by training and by the dashboard server.

KEY FEATURES USED

- Columnar storage for efficient reads of specific feature columns during training/CV
- Compression, reducing on-disk size of the cleaned/featured dataset vs. CSV
- Schema/type preservation (important for consistent dtypes across the ~40 engineered features, e.g., datetime columns, categorical encodings)
- Written/read via `pandas.DataFrame.to_parquet` / `read_parquet`, powered by the `pyarrow` engine

DETAILED DESCRIPTION

Parquet sits at the two most performance-sensitive I/O points in the pipeline — after preprocessing (`clean.parquet`) and after feature engineering (`features.parquet`) — because these are the largest, most feature-rich intermediate datasets that training and cross-validation repeatedly read. Choosing a columnar binary format here (rather than CSV) avoids re-parsing text and re-inferring dtypes on every pipeline run or dashboard read, which matters given the ~40 leakage-safe features (lag, rolling, decomposition, business, hierarchical) computed per series. Parquet's strict schema/type preservation is also functionally important, not just a performance nicety: many of these features are floats derived from `shift`/`rolling` operations that could silently change dtype (e.g., int64 to float64 due to NaNs) if round-tripped through a text format like CSV, which could subtly affect model training reproducibility. The dashboard server reads `features.parquet` and `clean.parquet` directly for endpoints like `/api/data-overview` and `/api/eda/{series_id}`, meaning Parquet's fast columnar reads directly benefit UI responsiveness, not just the offline training pipeline.

ALTERNATIVES

1. **CSV** - the project's own choice for smaller, more human-inspectable outputs (`model_comparison.csv`, `feature_importance.csv`, `forecast.csv`) — Parquet was chosen instead for the two largest/most-reused datasets specifically because CSV lacks compression and reliable type preservation at that scale.
2. **Feather (Arrow IPC format)** - similarly fast, pyarrow-backed, less compressed than Parquet but faster to read/write for pure in-process caching; Parquet chosen likely for its broader ecosystem support and better compression for on-disk artifacts.
3. **HDF5** - another common scientific binary format; more complex API and less naturally integrated with pandas'/pyarrow's modern columnar tooling than Parquet.

4. **A real database table (e.g., SQLite/Postgres)** - would add query capability but far more overhead for what's fundamentally a batch-pipeline intermediate artifact read wholesale by pandas each time.

PROS

- Compact, compressed storage compared to CSV for the largest pipeline datasets
- Preserves exact dtypes/schema across writes/reads, avoiding subtle reproducibility bugs
- Fast columnar reads benefit both training (repeated CV folds) and dashboard responsiveness
- Well-integrated with pandas via pyarrow with no extra code complexity

CONS

- Binary format isn't human-readable/diffable like CSV (project mitigates this by keeping smaller reporting outputs in CSV)
- Requires the `pyarrow` dependency specifically for this I/O path
- Less universally tooling-compatible than CSV for quick manual inspection or spreadsheet import

RESOURCES

- https://parquet.apache.org/docs/

CSV

WHAT IS IT?

CSV (Comma-Separated Values) is a plain-text tabular data format, one of the simplest and most universally supported ways to store structured data.

PURPOSE IN THIS PROJECT

CSV is used for the project's human-facing reporting artifacts — `reports/model_comparison.csv` (per-fold metrics across all 7 models), `reports/feature_importance.csv` (best model's feature importances), and `reports/forecast.csv` (generated forecasts) — per `ARCHITECTURE.md` 's artifacts contract. It's also the expected input format for custom user data (`csv_path` in `config.yaml` , and the `/api/upload` endpoint in `dashboard/server.py`), and optionally the raw M5 source data itself.

KEY FEATURES USED

- pandas `to_csv` / `read_csv` for writing/reading reporting artifacts (`preprocessor.py` , `loader.py` , `downloader.py` , `train_pipeline.py` , `forecast_api.py` , `drift_detector.py` all touch CSV I/O per the grep results)
- Long-format schema convention for user-supplied data (`date, series_id, sales` per `config.yaml` 's comment, matching the canonical `date, series_id, item_id, store_id, sales, sell_price, promotion_flag` shape from `ARCHITECTURE.md`)
- Human-readable, diffable reports that a user can open directly in Excel/a text editor without special tooling
- `UploadFile` handling in FastAPI's `/api/upload` endpoint, presumably parsing an uploaded CSV into the pipeline's expected schema

DETAILED DESCRIPTION

CSV is deliberately reserved for artifacts meant to be read by humans or simple external tools, contrasting with Parquet's role for the larger, machine-read-heavy intermediate datasets. `model_comparison.csv` , `feature_importance.csv` , and `forecast.csv` are exactly the outputs a user would want to open in Excel, inspect casually, or import elsewhere — small enough that CSV's lack of compression and slower parsing don't matter, and valuable enough that plain-text portability does. CSV is also the on-ramp for bringing your own data into the platform: `config.yaml` 's `data.source: 'csv'` option and its `csv_path` comment (`# used when source == csv (long format: date, series_id, sales)`) define exactly the minimal schema a new dataset needs, and the README's "To use your own data" section confirms this is the documented, supported path for non-Kaggle

usage. The FastAPI `/api/upload` endpoint extends this to the dashboard UI itself, letting a user upload a CSV file directly rather than editing `config.yaml` and rerunning the CLI pipeline. Using CSV for these specific artifacts (rather than Parquet everywhere) is a sensible format-per-purpose split: Parquet for size/type-sensitive intermediate data, CSV for small, inspectable, and externally-authored data.

ALTERNATIVES

1. **JSON** - used elsewhere in the project (`validation_report.json` , `drift_report.json` , `retraining_log.jsonl`) for structured/nested reports; CSV is preferred instead for these specifically tabular, flat, spreadsheet-like outputs (model comparison, feature importance, forecasts).
2. **Parquet (for reports too)** - would be more efficient, but sacrifices the human-readability/Excel-openability that's the whole point of these particular report artifacts.
3. **Excel (.xlsx) directly** - would give richer formatting, but adds an unnecessary dependency (openpyxl) for something plain CSV already achieves for tabular data.

PROS

- Universally readable — opens in Excel, any text editor, or any language's CSV parser with zero special tooling
- Simple, well-defined schema (`date, series_id, sales` minimum) lowers the bar for bringing custom datasets
- Diffable in version control, unlike binary Parquet files
- Natural fit for the small, report-sized artifacts it's used for here

CONS

- No compression or strict type preservation — not chosen for the larger `clean.parquet` / `features.parquet` datasets for exactly this reason
- Slower to parse at scale than Parquet (irrelevant for the report-sized files it's actually used for)
- Ambiguity risk (delimiters, encoding, date formats) when accepting arbitrary user-uploaded CSVs via `/api/upload`

RESOURCES

- https://docs.python.org/3/library/csv.html

APIs & External Services

Kaggle API (via kagglehub)

WHAT IS IT?

kagglehub is Kaggle's official lightweight Python client for downloading datasets and competition data directly from Kaggle, without needing the full legacy `kaggle` CLI package.

PURPOSE IN THIS PROJECT

kagglehub (`src/data_loading/downloader.py`) downloads the M5 Forecasting Accuracy competition dataset via `kagglehub.competition_download("m5-forecasting-accuracy")` , gated behind the presence of Kaggle credentials (`KAGGLE_USERNAME` / `KAGGLE_KEY` in `.env.example` , or `~/.kaggle/kaggle.json`). If credentials are missing or the download fails (e.g., competition rules not accepted), the pipeline falls back to synthetic data generation.

KEY FEATURES USED

- `kagglehub.competition_download("m5-forecasting-accuracy")` — one-line competition dataset download returning a local cache path

- Deferred/guarded import (`import kagglehub` inside a try block per the grep result) so the whole pipeline degrades gracefully without Kaggle credentials
- Implicit reliance on Kaggle API auth resolution (`~/.kaggle/kaggle.json` or `KAGGLE_USERNAME` / `KAGGLE_KEY` env vars)

DETAILED DESCRIPTION

kagglehub is the project's real-world data acquisition path — it fetches the actual M5 Forecasting Accuracy dataset (Walmart sales data, the same one used in the M5 forecasting competition) so DemandIQ can demonstrate its full lifecycle against real, messy, large-scale retail demand data rather than only toy/synthetic data. Because Kaggle competition datasets require accepting competition rules and authenticating, the project treats this as optional infrastructure: `config.yaml`'s `data.source: 'kaggle'` default triggers the download attempt, but `README.md` and `ARCHITECTURE.md` both stress that a synthetic fallback (trend + weekly/yearly seasonality + promos + noise) keeps "the platform demonstrable with zero external credentials." This design — real Kaggle API integration with a documented, tested fallback — reflects a common real-world pattern for ML platforms that depend on external data providers: never let an external service's availability (or a user's lack of credentials) block the core pipeline from running end-to-end. `kagglehub` itself is a relatively new, minimal replacement for the older `kaggle` CLI package, favored for its simpler Python-native API (`competition_download` / `dataset_download` calls) versus needing a separate CLI tool and `kaggle.json` shell-out.

ALTERNATIVES

1. **Legacy `kaggle` CLI/Python package** - the older way to fetch Kaggle datasets, requiring subprocess calls or its own API wrapper; kagglehub is the modern, actively promoted replacement with a cleaner in-process API.
2. **Manual download + local CSV** - the project's own `csv_path` config option already covers this as a fallback path if a user wants to sidestep the Kaggle API entirely.
3. **Direct HTTP requests to Kaggle's REST API** - possible but reinvents authentication/caching that kagglehub already handles.
4. **Synthetic data only (skip Kaggle integration)** - the project's own designed fallback; keeping the real kagglehub integration adds authenticity/realism value (real M5 data) that synthetic data alone can't fully replicate for demonstrating a genuine forecasting platform.

PROS

- One-line API for fetching a real, well-known, large-scale competition dataset (M5)
- Graceful degradation (synthetic fallback) means the absence of credentials never blocks the demo
- Simpler modern API than the legacy Kaggle CLI package
- Guarded/deferred import pattern keeps this as a genuinely optional dependency at runtime

CONS

- Requires external account setup + competition rule acceptance for the "real data" path, adding friction for first-time users
- Network-dependent and subject to Kaggle's own API availability/rate limits
- Competition-specific API (`competition_download`) is somewhat narrower than general dataset APIs, tying this integration specifically to M5's competition page

RESOURCES

- https://github.com/Kaggle/kagglehub

Build Tools & Build Systems

Vite (Build System)

WHAT IS IT?

Beyond serving as a dev server, Vite is also the project's frontend build system — bundling the React app for production via Rollup and esbuild.

PURPOSE IN THIS PROJECT

`npm run build` (mapped to `vite build` in `frontend/package.json`) compiles the entire `frontend/src` React app into optimized static assets, and `npm run preview` (`vite preview`) serves that production build locally for verification.

KEY FEATURES USED

- `vite build` production bundling (esbuild for dependency pre-bundling, Rollup for the final bundle)
- `vite preview` for locally verifying a production build before any real deployment
- Zero custom build configuration beyond the single `@vitejs/plugin-react` plugin

DETAILED DESCRIPTION

As a build system (distinct from its dev-server role covered under Frameworks & Libraries), Vite's `build` command handles code-splitting, minification, and asset hashing for the React dashboard with no custom Rollup configuration required — the project relies entirely on Vite's sensible defaults. This is appropriate given the frontend's modest scope (7 pages, a handful of shared components, two runtime dependencies beyond React itself), where a custom Webpack/Rollup pipeline would be pure over-engineering. There's no evidence in the read files of a CI/CD pipeline actually invoking `vite build` for deployment (no Dockerfile, no GitHub Actions workflow surfaced) — the build tooling exists and is npm-script-ready, but actual deployment of the built frontend isn't documented in README.md/ARCHITECTURE.md, which focus entirely on local `npm run dev` usage.

ALTERNATIVES

1. **Webpack** - the long-standing standard bundler; would require significantly more configuration for equivalent JSX/HMR/production-build behavior that Vite provides by default.
2. **esbuild directly (no Vite)** - Vite already uses esbuild internally for dev-time dependency pre-bundling; using it standalone would mean losing Vite's dev-server/HMR/plugin ecosystem for no real gain.
3. **Rollup directly (no Vite)** - Vite's production build is Rollup under the hood; using Rollup directly would require hand-configuring what Vite already wires up automatically (JSX transform, asset handling, code-splitting).

PROS

- Zero-config production build with sensible defaults (code-splitting, minification, asset hashing)
- `vite preview` gives a quick, accurate local check of the production bundle before any real deploy
- Consistent tool for both dev and build reduces tooling surface area

CONS

- No documented actual deployment target/pipeline for the built output in this project's read files
- Defaults may need tuning (base path, chunking strategy) if ever deployed behind a subpath or CDN

RESOURCES

- https://vite.dev/guide/build.html

Testing & QA

pytest

WHAT IS IT?

pytest is Python's most widely used testing framework, offering simple assertion-based test writing, fixtures, and rich plugin support.

PURPOSE IN THIS PROJECT

pytest runs the test suite under `tests/` — `test_metrics.py`, `test_features.py`, `test_validation.py`, `test_cv.py` — invoked via `python -m pytest tests -q` per README.md, and required as `pytest>=7.4` in `requirements.txt`.

KEY FEATURES USED

- Plain `assert`-based test functions (pytest's hallmark simplicity, no `self.assertEqual`-style boilerplate)
- Test files organized by pipeline concern: metrics correctness, feature-engineering leakage-safety, data validation, and cross-validation fold logic
- CLI invocation (`python -m pytest tests -q`) for quiet, CI-friendly output
- Presumably fixtures for shared synthetic/sample DataFrames across `test_features.py` / `test_cv.py` (idiomatic pytest pattern, consistent with testing pandas-heavy pipeline code)

DETAILED DESCRIPTION

pytest tests exactly the areas of the pipeline where silent correctness bugs would be most damaging and hardest to notice visually: `test_metrics.py` (are MAPE/RMSE/MAE/R²/SMAPE computed correctly, including the zero-sales-skip edge case in MAPE), `test_features.py` (per `ARCHITECTURE.md`, "A unit test asserts it" regarding leakage-safety — i.e., verifying every lag/rolling/decomposition feature only ever uses strictly-past values), `test_validation.py` (schema/quality/outlier detection logic), and `test_cv.py` (walk-forward fold construction — expanding windows, correct train/validation/test splits). This is a deliberately narrow, high-value test surface rather than exhaustive coverage of every module — there's no visible test file for the model classes themselves (LightGBM/XGBoost/ARIMA/LSTM wrappers), MLflow tracking, or the FastAPI dashboard endpoints, suggesting the project prioritizes testing the parts of the system where correctness is both non-obvious and consequential (a leaked feature or a wrong metric formula would silently corrupt the entire model-selection process) over integration/API-level testing. pytest's minimal ceremony (no boilerplate test classes required, `assert` statements read like plain Python) fits a project that otherwise favors lean, readable code across the board.

ALTERNATIVES

1. **unittest (stdlib)** - built into Python, no extra dependency, but requires more boilerplate (`TestCase` classes, `self.assertEqual`) than pytest's plain-function/assert style; pytest chosen for ergonomics.
2. **nose2** - an older unittest-extension framework, largely superseded by pytest in the Python ecosystem; no reason to prefer it today.
3. **Hypothesis (property-based testing)** - could strengthen the leakage-safety test (`test_features.py`) by generating many random time series to prove no lag ever peeks forward, rather than checking specific cases; not used here, a reasonable scope-limiting choice for a demo-scale project.
4. **doctest** - lightweight inline testing via docstrings; insufficient for the stateful, DataFrame-heavy assertions this project's tests need.

PROS

- Minimal boilerplate, plain `assert` statements keep tests readable
- Focused on the highest-risk correctness areas (metrics, leakage, CV folds, validation) rather than padding coverage everywhere
- Simple CLI invocation fits into any CI system without extra config
- Rich plugin ecosystem available if the project's testing needs grow (fixtures, parametrization already idiomatic)

CONS

- No visible tests for models, MLflow integration, or the FastAPI dashboard endpoints — a real coverage gap if those layers regress silently
- No visible CI configuration (GitHub Actions, etc.) actually running `pytest` automatically in the read files
- Relies on discipline to keep tests in sync with `config.yaml` changes (e.g., new feature additions needing new leakage assertions)

RESOURCES

- https://docs.pytest.org/

oxlint

WHAT IS IT?

oxlint is a Rust-based, extremely fast JavaScript/TypeScript linter (part of the Oxc toolchain), designed as a much quicker alternative to ESLint for common rule sets.

PURPOSE IN THIS PROJECT

oxlint (`^1.75.0` , `frontend/package.json`) lints the React frontend, configured via `frontend/.oxlintrc.json` with the `react` and `oxc` plugins and two explicit rules: `react/rules-of-hooks: "error"` and `react/only-export-components: ["warn", { allowConstantExport: true }]` . Run via `npm run lint` .

KEY FEATURES USED

- `react` plugin rule `rules-of-hooks` set to `error` — enforces React's hooks call-order/conditional-call rules, critical correctness guard for a hooks-based codebase
- `react/only-export-components` set to `warn` with `allowConstantExport: true` — flags files that mix component exports with non-component exports (important for Vite's Fast Refresh to work reliably), while permitting constant exports like small config objects
- `oxc` plugin for core JS/general linting rules
- Config schema reference (`$schema`) pointing at oxlint's own JSON schema for editor autocomplete/validation

DETAILED DESCRIPTION

oxlint is chosen over the traditional ESLint + plugin stack primarily for speed — being written in Rust, it lints far faster than ESLint's JS-based rule engine, which matters more as a developer-experience nicety than a hard requirement for a project this size, but is a deliberate, modern tooling choice nonetheless. The specific rule configuration is minimal and targeted: `rules-of-hooks` catches a genuinely common and hard-to-debug class of React bugs (hooks called conditionally or in loops), while `only-export-components` (with the constant-export allowance) keeps files compatible with Vite's Fast Refresh, which requires component-only exports per file to reliably hot-reload without full page refreshes. The choice to configure only two explicit rules (rather than a large custom ruleset) reflects the project's minimalist tooling philosophy — relying on oxlint's sensible defaults plus these two React-specific guards rather than building out an extensive lint configuration for a small, 14-file frontend. Because oxlint is still a newer tool relative to ESLint, using it here (version 1.75.0, a mature release) signals the project intentionally adopted current, fast-moving JS tooling rather than defaulting to the ESLint status quo.

ALTERNATIVES

1. **ESLint (+ eslint-plugin-react, eslint-plugin-react-hooks)** - the long-standing standard; oxlint reimplements a compatible subset of its most useful rules in Rust for much faster execution, at the cost of a smaller plugin ecosystem than ESLint's.
2. **Biome** - another Rust-based linter/formatter competing directly with oxlint; comparable speed benefits, oxlint chosen here likely for its more React-focused plugin (`react` plugin covering hooks rules specifically).

3. **No linter** - would be a real simplification for such a small frontend, but the `rules-of-hooks` check specifically catches bugs that are otherwise easy to introduce and hard to spot in code review, making this a worthwhile minimal addition.
4. **TypeScript compiler (`tsc --noEmit`) as a lint substitute** - would catch type errors but not the specific hooks-usage-pattern bugs `rules-of-hooks` targets; the project uses plain JSX, so this isn't in play regardless.

PROS

- Extremely fast linting (Rust-based) compared to ESLint, negligible wait time even as the frontend grows
- `rules-of-hooks` catches a real, common class of React bugs that's easy to introduce and hard to debug
- `only-export-components` keeps Fast Refresh reliable during Vite dev sessions
- Minimal, targeted config avoids lint-rule bloat for a small codebase
- Simple `npm run lint` integration

CONS

- Smaller plugin/rule ecosystem than ESLint — fewer highly specialized rules available if needed later
- Younger tool overall, so community troubleshooting resources are less abundant than ESLint's
- Not integrated into a visible CI pipeline or pre-commit hook in the read files, so enforcement currently depends on manual `npm run lint` runs

RESOURCES

- https://oxc.rs/docs/guide/usage/linter.html

Development Tools

config.yaml (centralized YAML configuration)

WHAT IS IT?

This isn't a third-party tool but the project's own convention: a single root `config.yaml` file acting as the master control surface for every tunable aspect of the pipeline, loaded via PyYAML (see Other Utilities).

PURPOSE IN THIS PROJECT

`config.yaml` centralizes data source selection, forecast horizon, CV folds/strategy, per-model hyperparameters (all 7 models), MLflow settings, drift-detection thresholds, retraining triggers, and logging config — per README.md, "Everything is driven by `config.yaml`."

KEY FEATURES USED

- Nested YAML structure mirroring pipeline stages (`data` , `forecasting` , `features` , `models` , `mlflow` , `drift_detection` , `retraining` , `logging`)
- Per-model `enabled` flags plus hyperparameters, letting any of the 7 models be toggled off without code changes (e.g., `exponential_smoothing.enabled: false`)
- A single `random_seed: 42` for reproducibility across the whole pipeline
- Loaded through `src/utils/config_loader.py` 's `load_config` function, used consistently across `model_factory.py` , `mlflow_utils/tracker.py` , and `dashboard/server.py`

DETAILED DESCRIPTION

This centralized-config pattern is a core development-tooling decision for the project: instead of hardcoding hyperparameters or thresholds across dozens of files, every knob lives in one YAML file that both the CLI pipeline (`main.py`) and the FastAPI dashboard (`dashboard/server.py`) read through the same `load_config` utility. This makes experimentation (e.g., changing `data.n_series` or `models.lightgbm.n_estimators`) a one-line YAML edit rather than a code change, directly supporting the "swap CSV/Kaggle/synthetic," "tune per-model hyperparameters," and "adjust drift/retraining thresholds" workflows the README documents. The config's structure also directly documents the platform's architecture — its top-level keys (`data` , `forecasting` , `features` , `models` , `mlflow` , `drift_detection` , `retraining` , `logging`) map almost one-to-one onto the pipeline stages described in `ARCHITECTURE.md` , making the config file itself a readable map of the system.

ALTERNATIVES

1. **Environment variables only** - `.env.example` already covers secrets (Kaggle credentials); YAML is used instead for structured, nested, non-secret configuration since env vars don't naturally express nested hyperparameter groups.
2. **Python config module (`config.py` with constants)** - would lose the language-agnostic, non-executable nature of YAML and make it harder for non-developers to tweak thresholds safely.
3. **JSON config** - equally structured, but YAML's comments (used throughout `config.yaml` to explain fields like `csv_path` and `n_series`) are a real, used advantage JSON lacks natively.
4. **Hydra / OmegaConf** - a more powerful config-composition framework (overrides, multirun) common in ML research; likely unnecessary overhead for a project with one flat config file and no need for CLI override composition.

PROS

- Single source of truth for every tunable parameter across the whole pipeline and dashboard
- Comments directly in the file document intent (e.g., "top-N series by total sales... keep training <15 min")
- No code changes needed to toggle models, change data sources, or adjust thresholds
- Structure mirrors the architecture, aiding onboarding/readability

CONS

- No schema validation shown — a typo'd key could silently fail to apply (mitigated only by code discipline in `config_loader.py`)
- All config in one file could grow unwieldy if the project scaled to many more models/environments
- No environment-specific override mechanism (e.g., dev vs. prod config) beyond manual editing

RESOURCES

- https://yaml.org/spec/

Deployment & DevOps

Local-only / manual process launch (no containerization or orchestration observed)

WHAT IS IT?

This isn't a specific tool but a deployment posture: the project has no Dockerfile, CI/CD workflow, or process orchestrator visible in any of the read files — deployment is "run these commands on your machine."

PURPOSE IN THIS PROJECT

Per README.md's "Quick Start," the entire platform is launched via four manual local commands: `pip install -r requirements.txt`, `python main.py`, `uvicorn dashboard.server:app --port 8000`, and `cd frontend && npm install && npm run dev` (plus optionally `mlflow ui`). No deployment target, container image, or hosting configuration is documented anywhere in README.md or ARCHITECTURE.md.

KEY FEATURES USED

- Manual multi-terminal local process launch (pipeline, API server, frontend dev server, optional MLflow UI each in a separate terminal/process)
- File-based artifact contract (per `ARCHITECTURE.md`) as the sole "integration" mechanism between pipeline and dashboard, rather than any deployed service boundary
- CORS wide open (`allow_origins=["*"]`) on the FastAPI server, appropriate only for this local, non-networked posture

DETAILED DESCRIPTION

DemandIQ is explicitly scoped as a local, demonstrable platform rather than a deployed production service — every design decision documented in `ARCHITECTURE.md` (SQLite MLflow backend, synthetic data fallback, "local dev only" FastAPI server, wide-open CORS) reinforces that deployment/DevOps concerns (containerization, CI/CD, cloud hosting, secrets management beyond a `.env.example`) are intentionally out of scope for this project's current state. This is a reasonable and honest choice for what the README calls "an internal-ML-platform-style system" meant to demonstrate the full forecasting lifecycle end-to-end, not to be operated as a real production service. If this project were to move toward real deployment, the natural next steps (not currently present) would be: a Dockerfile per component (pipeline runner, FastAPI server, frontend static build), a docker-compose or Kubernetes manifest to wire them together, a CI workflow running `pytest` and `npm run lint` / `vite build` on every push, and swapping the SQLite MLflow backend for a server-based one — exactly the kind of "swap it for a server later" language `ARCHITECTURE.md` already anticipates for MLflow specifically.

ALTERNATIVES

1. **Docker Compose** - would containerize the pipeline, FastAPI server, and frontend build into reproducible, one-command-startup services; a natural next step but not implemented, appropriate for the project's current local-demo scope.
2. **GitHub Actions CI** - would automate running `pytest tests` and `npm run lint` / `vite build` on every commit; absent currently, a reasonable low-cost addition if the project moves toward team collaboration.
3. **Cloud deployment (e.g., a managed FastAPI host + static frontend host + managed MLflow/Postgres)** - the eventual production path `ARCHITECTURE.md` gestures at via the swappable `tracking_uri`, but a significant scope expansion beyond this project's current local-only design.
4. **A Makefile or single startup script** - a lightweight middle ground (not currently present) that could reduce the "four manual terminal commands" friction without adopting full containerization.

PROS

- Zero infrastructure overhead — anyone can clone and run the whole lifecycle with four commands and no cloud account
- No deployment complexity to maintain for what's fundamentally a demonstration/learning platform
- Forces a clean file-based artifact contract between pipeline and dashboard (a genuine architectural benefit, not just a shortcut)
- `.env.example` shows secrets are handled correctly (never committed) even without a fuller DevOps setup

CONS

- No reproducible, one-command environment setup (no Docker) — depends on the user's local Python/Node versions matching expectations
- No CI enforcing tests/lint on changes, relying entirely on manual discipline
- Not deployable as-is to any real hosting environment without additional DevOps work (Dockerfiles, process management, secrets handling beyond `.env`)

- Wide-open CORS and no auth would need to be addressed before any real deployment

RESOURCES

- https://docs.docker.com/ (as the natural next step, referenced for context, not currently used)

Package Managers & Dependency Management

pip / requirements.txt

WHAT IS IT?

pip is Python's standard package installer; `requirements.txt` is the conventional flat-file format listing pinned/range-constrained dependencies for reproducible installs.

PURPOSE IN THIS PROJECT

`requirements.txt` (root) lists all 19 Python dependencies with minimum-version (and one max-version) constraints — pandas, numpy, scikit-learn, lightgbm, xgboost, statsmodels, torch, mlflow, evidently, matplotlib, kagglehub, holidays, PyYAML, pyarrow, scipy, pytest, fastapi, uvicorn, python-multipart — installed via `pip install -r requirements.txt` per README.md, and also consumed programmatically by `setup.py` (`install_requires=Path("requirements.txt").read_text().splitlines()`).

KEY FEATURES USED

- Minimum-version constraints (`>=`) for most packages, ensuring required API surfaces (e.g., `mlflow>=2.10` for registry features used)
- One deliberate upper-bound constraint (`numpy>=1.24,<2.0`) to avoid NumPy 2.0's breaking changes across the dependency graph
- Reuse of `requirements.txt` as `setup.py`'s `install_requires` source, avoiding maintaining two duplicate dependency lists
- Simple `pip install -r requirements.txt` as the entire backend setup step

DETAILED DESCRIPTION

`requirements.txt` is the single source of truth for the Python dependency graph, and the project cleverly avoids the common problem of dependency lists drifting between `requirements.txt` and `setup.py` by having `setup.py` literally read and parse `requirements.txt` at install time (`Path("requirements.txt").read_text().splitlines()`) rather than duplicating the list. This is a lightweight, effective pattern for a project that wants both a pip-installable package (`pip install -e .` via `setup.py`) and a simple flat requirements file for quick environment setup. The version constraints are mostly permissive floors (`>=`) rather than exact pins, which is appropriate for a project not yet concerned with fully reproducible/locked production deployments — the one exception, `numpy<2.0`, reflects a real, specific compatibility concern (several ML libraries' wheels/ABIs lagged NumPy 2.0's release) rather than general caution, showing the constraints were chosen deliberately rather than defaulted.

ALTERNATIVES

1. **Poetry** - would add a lockfile (`poetry.lock`) for fully reproducible installs and better dependency resolution; not used here, likely to keep the setup as simple as `pip install -r requirements.txt` for a demonstration project.
2. **pip-tools (`pip-compile`)** - would generate a fully pinned, hash-locked requirements file from a looser input; not used, consistent with the project's minimal-tooling approach for a non-production-locked dependency set.
3. **Conda / conda-forge environment.yml** - common in ML projects for managing native/binary dependencies (e.g., CUDA for torch); not used here, likely because the project's dependencies (as pip wheels) install fine without conda's binary-package management benefits.

4. **uv** - a much faster modern pip/venv replacement; would speed up installs but wasn't necessarily available/standard when this project's tooling was chosen; plain pip remains the most universally compatible choice.

PROS

- Single dependency list reused by both `pip install -r requirements.txt` and `setup.py`'s `install_requires`, avoiding drift
- Minimum-version floors keep dependencies current without over-constraining
- The one upper bound (`numpy<2.0`) shows deliberate compatibility awareness, not just copy-pasted defaults
- Extremely low barrier to entry — no extra tooling (Poetry, conda) required to get started

CONS

- No lockfile means installs aren't fully reproducible bit-for-bit across time (a new dependency release could silently change behavior)
- No hash-pinning for supply-chain integrity verification
- Doesn't separate dev/test dependencies (`pytest`) from runtime dependencies in the same file

RESOURCES

- https://pip.pypa.io/en/stable/

npm / package-lock.json

WHAT IS IT?

npm is Node.js's default package manager; `package-lock.json` is its auto-generated lockfile pinning exact resolved versions (including transitive dependencies) for reproducible installs.

PURPOSE IN THIS PROJECT

npm manages the frontend's dependencies declared in `frontend/package.json` (axios, react, react-dom, react-router-dom, recharts as runtime deps; @types/react, @types/react-dom, @vitejs/plugin-react, oxlint, vite as dev deps), installed via `npm install` per README.md's Quick Start, with `npm run dev` / `build` / `lint` / `preview` as the defined scripts.

KEY FEATURES USED

- Semver-range dependency declarations (`^1.19.0` , `^19.2.8` , etc.) in `package.json` , allowing compatible minor/patch updates
- `package-lock.json` (implied standard npm artifact, not directly read but standard for any `npm install` -based project) pinning exact resolved dependency trees for reproducible installs across machines
- npm `scripts` block as the standard task-runner interface (`dev` , `build` , `lint` , `preview`)
- Clean separation of `dependencies` (shipped to users: axios, react, react-dom, react-router-dom, recharts) vs. `devDependencies` (build/lint-only: types, plugin-react, oxlint, vite)

DETAILED DESCRIPTION

npm is used in its most standard, unremarkable configuration — no workspaces/monorepo tooling, no custom registry, just a `package.json` with a small, well-curated dependency list and four scripts covering the entire frontend workflow (dev, build, lint, preview). The clear separation between runtime `dependencies` (what actually ships in the built bundle) and `devDependencies` (tooling only needed locally) keeps the eventual production bundle lean — types, the Vite plugin, and oxlint never ship to the browser. The `package-lock.json` (standard output of any `npm install` in this setup) ensures that everyone running `npm`

`install` gets the exact same dependency tree, which matters here because Vite 8.x, React 19.x, and Recharts 3.x are all fairly recent major versions where minor transitive-dependency drift could otherwise cause subtle build or runtime differences between developers' machines.

ALTERNATIVES

1. **Yarn (Classic or Berry)** - comparable lockfile-based package manager; npm chosen likely for being Node's zero-extra-install default, appropriate for a small frontend with no need for Yarn's workspace/PnP features.
2. **pnpm** - faster installs and disk-efficient via content-addressable storage; would benefit larger monorepos more than this single small `frontend/` package.
3. **Bun** - a newer, much faster all-in-one JS runtime/package manager; could replace npm for speed, but npm remains the safest, most universally compatible default for a project not otherwise pushing performance limits in tooling.

PROS

- Zero extra install needed (ships with Node.js) — lowest-friction choice for a small frontend
- `package-lock.json` guarantees reproducible dependency resolution across machines
- Clean dependencies/devDependencies split keeps the production bundle free of build-only tooling
- Scripts (`dev` / `build` / `lint` / `preview`) provide a simple, standard task-running interface

CONS

- Slower install times than pnpm/Bun for larger dependency trees (not a real issue at this project's small dependency count)
- No built-in workspace/monorepo support (not needed here, single `frontend/` package)
- Semver-range (`^`) declarations in `package.json` still allow drift between installs unless `package-lock.json` is strictly respected (which npm does by default with `npm ci` , though the README uses `npm install`)

RESOURCES

- https://docs.npmjs.com/

Other Utilities

PyYAML

WHAT IS IT?

PyYAML is the standard Python library for parsing and emitting YAML, the human-readable data-serialization format.

PURPOSE IN THIS PROJECT

PyYAML (`>=6.0` , `requirements.txt`) parses the root `config.yaml` into a Python dict, consumed via `src/utils/config_loader.py` 's `load_config` function and used throughout the pipeline (`model_factory.py` , `mlflow_utils/tracker.py`) and the FastAPI dashboard server.

KEY FEATURES USED

- `yaml.safe_load` (standard, safe parsing pattern) to turn `config.yaml` into a nested Python dict/list structure
- Support for YAML comments in `config.yaml` (used extensively to document intent, e.g., "top-N series by total sales (M5 has 30k; keep training <15 min)")
- Nested mapping/list parsing for the config's structure (models with sub-keys, lists like `lag_lookbacks: [1, 7, 14, 30, 60]`)

DETAILED DESCRIPTION

PyYAML is a small but foundational utility — it's the mechanism by which the entire project's "everything is driven by config.yaml" philosophy (per README.md) actually works. Every pipeline stage, every model's hyperparameters, every threshold (drift, retraining, promotion) flows through one `load_config()` call backed by PyYAML's parser, making it one of the most-imported utility modules in the codebase despite being conceptually simple. Its use of `safe_load` (the standard safe practice, avoiding arbitrary Python object deserialization that plain `yaml.load` permits) is a sensible default for a config file that, while currently trusted/local, costs nothing to parse safely.

ALTERNATIVES

1. **JSON (via stdlib `json`)** - would avoid the PyYAML dependency entirely (json is builtin), but loses YAML's crucial comment support, which `config.yaml` relies on heavily to document non-obvious choices (e.g., why `n_series` is capped, what `csv_path` expects).
2. **TOML (via stdlib `tomllib` in Python 3.11+)** - also supports comments and is now stdlib, could replace PyYAML for Python 3.11+, but the project targets `python_requires=">=3.10"` where `tomllib` isn't yet available, and YAML is simply more idiomatic for deeply nested ML config than TOML's table-heavy syntax.
3. **ruamel.yaml** - a more feature-rich YAML library (round-trip comment preservation on write), unnecessary here since the project only reads `config.yaml`, never programmatically writes it back.

PROS

- Enables human-readable, comment-annotated configuration that documents non-obvious design choices directly in the config file
- Simple, safe (`safe_load`) parsing pattern, minimal risk for a locally-trusted config file
- Handles the deeply nested structure (models → per-model hyperparameters) cleanly
- De facto standard for Python ML project configuration

CONS

- Extra dependency vs. stdlib `json`, solely for comment support and readability
- YAML's whitespace-sensitive syntax can cause subtle parsing errors if a user hand-edits `config.yaml` incorrectly
- `safe_load` is correctly used here, but YAML's flexibility (anchors, multiple document types) is more powerful than this project actually needs

RESOURCES

- https://pyyaml.org/wiki/PyYAMLDocumentation

scipy

WHAT IS IT?

SciPy is Python's core scientific computing library, providing statistics, optimization, signal processing, and other numerical algorithms on top of NumPy.

PURPOSE IN THIS PROJECT

scipy powers the core (non-optional) drift-detection statistical test: `src/monitoring/drift_detector.py` uses `from scipy.stats import ks_2samp` to run a two-sample Kolmogorov-Smirnov test comparing reference vs. current distributions for the top-N most important features plus the target, per `config.yaml`'s `drift_detection.statistical_test: 'ks'` and `p_value_threshold: 0.05`.

KEY FEATURES USED

- `scipy.stats.ks_2samp(reference[c].dropna(), current[c].dropna())` — the two-sample KS test itself, returning a statistic and p-value per monitored feature
- Comparison against `p_value_threshold: 0.05` (config-driven) to flag statistically significant distribution shift per feature
- Applied per-column across the top-5 most important features (`features_to_monitor: 5` in `config.yaml`) plus the target column

DETAILED DESCRIPTION

scipy's `ks_2samp` is explicitly the "core" drift-detection mechanism, per `ARCHITECTURE.md`: "KS-test drift core, Evidently optional: scipy KS is stable across environments and versions" — meaning this single scipy function is the one piece of drift logic the platform's retraining-trigger decisions actually depend on, with Evidently only providing an optional cosmetic HTML report on top. This is a notably lean design choice: rather than depending on a heavier drift-detection framework for the actual trigger logic, the project uses one well-understood, stable statistical test directly. The KS test's nonparametric nature (no assumption about the underlying distribution shape) suits demand-forecasting data well, since sales distributions are often skewed/non-normal and a parametric test (e.g., a t-test on means) could miss distributional shape changes that don't move the mean. Running it per top-N-important-feature (rather than all ~40 features) keeps the monitoring focused and interpretable — each flagged feature can be individually inspected — while still being cheap enough to run frequently (`check_frequency_days: 7` in the retraining config).

ALTERNATIVES

1. **Evidently's own statistical test suite** - already available as the optional layer in this project; the core logic deliberately uses scipy directly instead, for the stability reason `ARCHITECTURE.md` states explicitly.
2. **Population Stability Index (PSI)** - a common alternative drift metric in industry (especially credit risk/finance); KS test chosen instead, likely for scipy's simplicity and scipy already being a dependency for other reasons.
3. **Chi-squared test** - suited to categorical feature drift; KS test (continuous-distribution-focused) fits this project's mostly numeric engineered features (lags, rolling stats) better.
4. **Wasserstein distance (`scipy.stats.wasserstein_distance`)** - another scipy-provided distribution-comparison metric; KS test chosen likely for its more standard, interpretable p-value/significance framing versus a raw distance metric needing its own threshold calibration.

PROS

- Nonparametric — makes no distribution-shape assumptions, suiting real-world skewed demand data
- Extremely stable, mature, dependency already present for other reasons (LightGBM/sklearn transitively use it)
- Cheap to compute per feature, enabling frequent monitoring checks
- Provides the one drift signal the retraining triggers actually depend on, keeping the critical path simple and robust

CONS

- KS test alone doesn't explain *why* a distribution shifted (just that it did) — Evidently's optional report adds interpretability on top
- Sensitive to sample size (large samples can flag statistically significant but practically trivial shifts) — mitigated somewhat by the `recent_window_days: 30` scoping
- Only applied to top-5 features by default — real drift in a lower-ranked feature could go undetected

RESOURCES

- https://docs.scipy.org/doc/scipy/reference/stats.html

holidays

WHAT IS IT?

`holidays` is a Python library providing programmatically generated public holiday calendars for dozens of countries and subdivisions.

PURPOSE IN THIS PROJECT

`holidays` (`>=0.40` , `requirements.txt`) is imported inside `src/features/feature_engineering.py` (`import holidays as hol`) to generate US holiday dates, driven by `config.yaml` 's `features.holiday_country: 'US'` , feeding into the "temporal"/"business features" portion of the ~40-feature leakage-safe engineering pipeline.

KEY FEATURES USED

- Country-specific holiday calendar generation (`holiday_country: 'US'` config) used to derive a boolean/flag feature (e.g., `is_holiday`) per date
- Likely also used to compute days-to/from-nearest-holiday style business features (`include_business_features: true` in config), a common demand-forecasting signal since sales often spike around holidays
- Deferred/local import inside the feature engineering module rather than a top-level import, keeping the dependency scoped to where it's used

DETAILED DESCRIPTION

Holiday effects are one of the most important signals in retail demand forecasting (the M5 dataset itself, being Walmart sales data, is heavily holiday/promotion-influenced), so the `holidays` library gives the feature pipeline an accurate, well-maintained US holiday calendar without hand-coding date rules (which drift year to year — e.g., Thanksgiving's date changes annually). This directly supports `config.yaml` 's `include_business_features: true` and `include_temporal: true` flags, which together with lag/rolling/decomposition features make up the ~40 leakage-safe features described in `ARCHITECTURE.md` . Because holiday features are static, calendar-based facts (not derived from the target/sales column itself), they don't need the `shift(1)` -before-computation leakage-safety treatment that lag/rolling target-derived features require — holiday features are already "known in advance" for any future date, making them a safe and valuable addition to both the training feature set and the forecast-horizon feature set.

ALTERNATIVES

1. **Hand-coded holiday date lists** - fragile and requires yearly maintenance (holidays like Thanksgiving/Labor Day shift dates each year); `holidays` library removes this maintenance burden entirely.
2. **Prophet's built-in holiday effects** - Prophet has its own holiday-modeling mechanism, but this project doesn't use Prophet (see statsmodels/ARIMA alternatives), so a standalone holiday library is the right fit for feature-engineering-based models.
3. **pandas' `USFederalHolidayCalendar`** - a narrower built-in (via `pandas.tseries.holiday`) covering only US federal holidays; the `holidays` library offers broader country/subdivision coverage and easier extension if the project ever supported non-US data (`holiday_country` is already config-driven for exactly this reason).

PROS

- Accurate, maintenance-free holiday calendars (no manual date-list upkeep)
- Config-driven country selection (`holiday_country: 'US'`) makes the feature pipeline portable to other markets
- Holiday features are safe to compute without special leakage handling, unlike target-derived features
- Directly relevant signal for retail demand data (M5), likely a meaningfully predictive feature

CONS

- Currently hardcoded to `'US'` in the demonstrated config — multi-country support would need per-series country mapping if data spanned multiple markets
- Adds a dependency for what's fundamentally a lookup table (though one that's genuinely hard to maintain by hand correctly)
- Doesn't capture retailer-specific promotional calendars (handled separately via the dataset's own `promotion_flag` column)

RESOURCES

- https://python-holidays.readthedocs.io/

pyarrow

WHAT IS IT?

pyarrow is the Python binding for Apache Arrow, providing the in-memory columnar format and the engine pandas uses under the hood for Parquet I/O.

PURPOSE IN THIS PROJECT

pyarrow (`>=14.0` , `requirements.txt`) is the (implicit) engine behind pandas' `to_parquet` / `read_parquet` calls used for `data/processed/clean.parquet` and `data/processed/features.parquet` , the two core intermediate pipeline artifacts per `ARCHITECTURE.md` .

KEY FEATURES USED

- Backing engine for `pandas.DataFrame.to_parquet(...)` / `pd.read_parquet(...)` calls in `preprocessor.py` and `feature_engineering.py`
- Columnar in-memory representation enabling efficient compressed Parquet writes/reads
- Schema/type fidelity preservation across write/read cycles for the ~40-feature dataset

DETAILED DESCRIPTION

pyarrow is a mostly invisible dependency — never imported directly in the pipeline code (pandas abstracts it away entirely), but functionally required for `pandas.read_parquet` / `to_parquet` to work at all. Its presence in `requirements.txt` is what makes the project's choice of Parquet as the intermediate storage format (over CSV) actually operational; without it, pandas would fall back to needing `fastparquet` or fail outright on parquet I/O calls. Because the feature-engineering output (`features.parquet`) contains ~40 columns with mixed types (floats from lag/rolling computations, categorical-like string/int IDs for series/store/item, boolean holiday flags, datetime index), pyarrow's robust Arrow-based type system is what preserves this schema faithfully across every training run and dashboard read, avoiding the subtle dtype-coercion bugs that a text-based format could introduce.

ALTERNATIVES

1. **fastparquet** - the other common pandas Parquet engine; pyarrow chosen instead, likely for its more complete Arrow ecosystem integration and being the more actively maintained/broader-adopted default in recent pandas versions.
2. **No Parquet support (CSV only)** - would remove the pyarrow dependency entirely but sacrifice compression/type-fidelity benefits already discussed under the Parquet entry.
3. **Direct Arrow API usage (bypassing pandas)** - would give finer control but adds complexity with zero benefit here, since pandas' `to_parquet` / `read_parquet` already covers 100% of this project's parquet needs.

PROS

- Enables Parquet I/O for pandas with zero extra code (transparent engine)
- Preserves schema/type fidelity for the ~40-feature dataset across repeated training runs
- Actively maintained, the modern default parquet engine for pandas
- No direct API surface to learn/maintain — pandas handles the interface entirely

CONS

- Adds a fairly large binary dependency purely to support one file format's I/O
- Version compatibility must be kept in step with pandas' own supported pyarrow range
- Never used directly in project code, making its necessity easy to overlook/misremove from `requirements.txt`

RESOURCES

- https://arrow.apache.org/docs/python/

matplotlib

WHAT IS IT?

matplotlib is Python's foundational 2D plotting library, providing fine-grained control over static chart/figure generation.

PURPOSE IN THIS PROJECT

matplotlib (`>=3.8` , `requirements.txt`) is used in the exploratory `notebooks/01_data_exploration.ipynb` (per the grep results, it's the only file referencing `matplotlib` / `plt.`), for ad-hoc plotting during data exploration rather than anywhere in the production `src/` pipeline or the dashboard (which uses Recharts for all UI charts).

KEY FEATURES USED

- Standard `plt.plot` / `plt.figure` -style static chart generation inside Jupyter notebook cells
- Likely used for quick sanity-check visualizations of raw/processed M5 data during exploration (per the `notebooks/` folder's stated purpose: "exploration / training / evaluation walkthroughs")

DETAILED DESCRIPTION

matplotlib's role here is scoped entirely to the analyst/developer-facing exploration notebooks, not the production pipeline or the React dashboard — a sensible separation, since the dashboard's charts are served to end users via Recharts/React, while matplotlib serves the very different need of a data scientist quickly visualizing distributions, trends, or model diagnostics while developing the pipeline in `notebooks/01_data_exploration.ipynb` (and presumably the training/evaluation notebooks mentioned in README's project structure). Because it's confined to notebooks, matplotlib carries no production runtime weight or dependency risk for the FastAPI server or React app — it's purely a development-time tool, listed in `requirements.txt` mainly so the notebooks execute correctly for anyone reproducing the exploration workflow.

ALTERNATIVES

1. **seaborn** - a higher-level, prettier-by-default wrapper around matplotlib; not included, likely because the notebooks' plotting needs are simple enough that raw matplotlib suffices without an extra dependency.
2. **plotly** - would give interactive notebook charts, but adds weight for what's a one-time exploration task, not an interactive deliverable.
3. **Recharts (in a notebook context)** - not applicable, since Recharts is JS/React-only; matplotlib is the natural Python-notebook equivalent for this exploration use case.

PROS

- Standard, ubiquitous, zero-friction plotting for Jupyter-based data exploration
- Doesn't add any weight/risk to the production pipeline or served application
- Flexible enough for whatever ad-hoc diagnostic plots the exploration/training/evaluation notebooks need

CONS

- Confined to notebooks means no reusable plotting code shared with the dashboard (a deliberate, reasonable non-issue given Recharts serves that role instead)
- Static plots only (no interactivity) compared to what a dashboard user gets via Recharts
- Its notebook-only usage makes it easy to overlook when auditing "what's actually used in the pipeline"

RESOURCES

- https://matplotlib.org/stable/contents.html

Part 2: Architectural Flow & System Design

1. High-Level Architecture Overview

DemandIQ is a single-machine, file-system-mediated ML platform. There is no message queue, no service mesh, no database beyond a local SQLite file — every "integration" in this system is either a Python function call within one process, a subprocess CLI invocation (`python main.py`), or a shared file on disk that a second process later reads. A system diagram of this project would show three columns connected by files rather than network calls:

- **Left column — Batch pipeline** (`main.py` driving `src/`): a strictly linear chain of stages — load → validate → clean → features → train/CV → forecast → drift — each stage a plain function call into the next module, no queue or scheduler between them. This is a script, not a service.
- **Middle column — Artifacts on disk**: parquet files (`data/processed/*.parquet`), CSV/JSON/JSONL reports (`reports/*`), a pickled model (`models/best_model.pkl`), and MLflow's own SQLite DB + artifact store (`mlflow.db` , `mlruns/` or similar). This is the *only* contract between the batch pipeline and the dashboard — ARCHITECTURE.md calls it out explicitly as the "Artifacts contract" (see §6).
- **Right column — Dashboard**: a FastAPI process (`dashboard/server.py`) that reads those files over HTTP GET endpoints, plus two POST endpoints (`/api/retrain` , `/api/drift-check`) that call straight back into the `src/` pipeline code in-process (import + function call, not a subprocess or job queue). A separate Vite dev server serves the React SPA, which talks to FastAPI over `http://localhost:8000/api/*` via axios.

Arrows in this diagram: `main.py` → each `src/*` module in sequence, writing files as it goes; `dashboard/server.py` → reads those same files on every GET request (no caching, no in-memory state); dashboard buttons → `dashboard/server.py` → same `src/*` modules the batch pipeline uses (retraining literally reruns `load_data → run_validation → clean → build_features → run_training` , i.e. the `retrain()` function in `src/retraining/orchestrator.py` re-executes most of `main.py` 's body).

External integrations: - **Kaggle API** via `kagglehub.competition_download(...)` in `src/data_loading/downloader.py:26` — pulls the M5 competition CSVs to a local cache dir. Requires `~/.kaggle/kaggle.json` and accepted competition rules; failure is caught and the pipeline falls back to synthetic data (`src/data_loading/loader.py:77-86`). - **MLflow** via `mlflow` + `mlflow.tracking.MlflowClient` , pointed at `sqlite:///mlflow.db` (`config.yaml:71` , used in `src/mlflow_utils/tracker.py:18`). Every CV fold is logged as a run; the winning model is registered and promoted/staged in the MLflow Model Registry. - **Evidently** (optional) in `src/monitoring/drift_detector.py:78-97` — tries the ≥0.7 API first, falls back to the legacy `evidently.report.Report` API, and swallows the exception entirely if neither import works, logging a warning. The KS-test drift core (scipy) always runs regardless of Evidently's presence.

2. Component Breakdown

Data loading — `src/data_loading/loader.py` , `src/data_loading/downloader.py` - Purpose: get raw data into the canonical long format regardless of source. - Inputs: `config["data"]` (source, csv_path, n_series), Kaggle credentials (optional). - Outputs: an in-memory `DataFrame` with columns `[date, series_id, item_id, store_id, sales, sell_price, promotion_flag]` (no file written here). - Dependencies: `kagglehub` (optional, deferred import), `pandas` . - Key files: `src/data_loading/loader.py` (`load_m5` , `load_csv` , `load_data`), `src/data_loading/downloader.py` (`download_m5` , `generate_synthetic`).

Validation — `src/validation/validator.py` - Purpose: schema check + data-quality scoring before anything is trained on. - Inputs: raw long-format `DataFrame` . - Outputs: `reports/validation_report.json` (quality score, missing %, duplicates, date gaps, outlier count) and a `valid: bool` gate — `main.py:42-43` raises `SystemExit` if schema errors exist. - Dependencies: `pandas` , `numpy` . - Key file: `src/validation/validator.py` .

Preprocessing — `src/preprocessing/preprocessor.py` - Purpose: per-series frequency detection, gap reindexing to a complete date range, imputation (sales gap → 0, price → ffill/bfill, promo → 0), negative-sales clipping. - Inputs: validated raw `DataFrame` + config granularity setting. - Outputs: `data/processed/clean.parquet` (also returned in-memory with `df.attrs["frequency"]` set, which the forecast API later reads). - Dependencies: `pandas`. - Key file: `src/preprocessing/preprocessor.py` (`clean` , `detect_frequency`).

Feature engineering — `src/features/feature_engineering.py` - Purpose: build ~40 leakage-safe features (temporal/cyclical, lag, rolling, decomposition, business/price, hierarchical store/item averages), all target-derived features computed on `shift(1)` - first values. - Inputs: cleaned `DataFrame` + `config["features"]` toggles. - Outputs: `data/processed/features.parquet` ; `feature_columns()` also defines the model input contract used by every downstream model. - Dependencies: `pandas` , `numpy` , optional `holidays` package (best-effort, wrapped in try/except at `src/features/feature_engineering.py:31-44`). - Key file: `src/features/feature_engineering.py` .

Model training / walk-forward CV — `src/training/train_pipeline.py` , `src/training/cross_validator.py` , `src/models/model_factory.py` - Purpose: run every enabled model through identical expanding-window CV folds, pick the best by mean MAPE (RMSE tiebreak), refit on train+val, score once on a held-out test window. - Inputs: features `DataFrame` , `config["forecasting"]` , `config["models"]` . - Outputs: `reports/model_comparison.csv` , `reports/feature_importance.csv` , `models/best_model.pkl` , `models/best_model_meta.json` , MLflow runs. - Dependencies: `scikit-learn` , `lightgbm` , `xgboost` , `statsmodels` (ARIMA), optional `torch` (LSTM, deferred import so its absence doesn't break the rest of the platform — `src/models/model_factory.py:34-39`). - Key files: `src/training/train_pipeline.py` (`run_training`), `src/training/cross_validator.py` (`make_folds`), `src/models/model_factory.py` .

MLflow tracking — `src/mlflow_utils/tracker.py` - Purpose: log every fold's params/metrics, register the winning model, and promote it to Production only if it beats the current Production model's test MAPE by the configured margin (default 2%); otherwise park it in Staging. - Inputs: model name/params/metrics per fold; `config["mlflow"]` . - Outputs: rows/artifacts in `mlflow.db` (SQLite) + registry stage transitions. - Dependencies: `mlflow` package. - Key file: `src/mlflow_utils/tracker.py` (`setup` , `log_run` , `register_best` , `get_model_versions`).

Forecasting API — `src/api/forecast_api.py` - Purpose: produce the next-horizon forecast per series from the persisted best model. Feature/tree models forecast recursively (predict → append → regenerate features); ARIMA/ExpSmoothing/LSTM forecast the whole horizon directly. - Inputs: cleaned `DataFrame` , `models/best_model.pkl` . - Outputs: `reports/forecast.csv` (`date, series_id, forecast`). - Dependencies: whichever model class won (`sklearn` / `lightgbm` / `xgboost` / `statsmodels` / `torch`). - Key file: `src/api/forecast_api.py` (`generate_forecast` , `_future_rows`). Note: despite the "api" module name, this is a plain Python function called by `main.py` and by the dashboard indirectly — it is not itself an HTTP endpoint. The actual HTTP layer is `dashboard/server.py` .

Drift monitoring — `src/monitoring/drift_detector.py` - Purpose: two-sample KS test on the top-N important features + target, reference (training window) vs. current (recent window); optional Evidently HTML report. - Inputs: features `DataFrame` , `reports/feature_importance.csv` , `config["drift_detection"]` . - Outputs: `reports/drift_report.json` , optional `reports/drift_report.html` . - Dependencies: `scipy` , optional `evidently` . - Key file: `src/monitoring/drift_detector.py` (`detect_drift`).

Performance monitoring — `src/monitoring/performance_monitor.py` - Purpose: recompute MAPE on the most recent window and compare vs. the training-time test MAPE baseline; flag degradation beyond threshold (default 15%). - Inputs: features `DataFrame` , `models/best_model.pkl` + meta, `config["retraining"]` . - Outputs: an in-memory result dict (not persisted to its own file; consumed by `/api/performance-check` and by the retraining trigger check). - Key file: `src/monitoring/performance_monitor.py` (`check_performance`).

Retraining orchestrator — `src/retraining/orchestrator.py` - Purpose: evaluate automatic trigger conditions (drift ≥3 features, MAPE degradation) and run a full retraining pass (reload → validate → clean → features → train), appending an event to a JSONL log. - Inputs: features `DataFrame` , `config["retraining"]` . - Outputs: `reports/retraining_log.jsonl` (append-only); side effects are the same artifacts training normally produces. - Key file: `src/retraining/orchestrator.py` (`check_triggers` , `retrain` , `read_retrain_log`). Note the imports of `load_data` / `build_features` / `clean` / `run_training` are deferred inside `retrain()` specifically "to avoid a circular import with main-pipeline modules" (line 37 comment) — a sign the module graph isn't fully acyclic and this is a deliberate workaround rather than a layering guarantee.

Dashboard backend (FastAPI) — `dashboard/server.py` - Purpose: local dev HTTP API — read artifacts as JSON/CSV, expose two action endpoints (`/api/retrain` , `/api/drift-check`) that call straight into `src/` . Explicitly documented in its own docstring as "Not a model serving layer." - Inputs: HTTP requests from the React app; files under `data/` , `reports/` , `models/` . - Outputs: JSON responses; one `text/html` endpoint for the raw Evidently report (`/api/drift/html`). - Dependencies: `fastapi` , `pandas` , `statsmodels` (for on-the-fly seasonal decomposition/ACF/PACF in `/api/eda/{series_id}` — this is computed at request time, not precomputed), CORS wide open (`allow_origins=["*"]`). - Key file: `dashboard/server.py` .

Dashboard frontend (React) — `frontend/` - Purpose: single-page app rendering the artifact contract as charts/tables and firing the two action buttons. - Inputs: JSON/HTML from `http://localhost:8000/api/*` . - Outputs: rendered UI only; no client-side state persistence beyond React component state. - Dependencies: React 19, `react-router-dom` 7 (client-side routing), `axios` (HTTP client, `frontend/src/api.js:3` hardcodes `baseURL: "http://localhost:8000/api"`), `recharts` (charting), Vite 8 (dev server/bundler). - Key files: `frontend/src/App.jsx` (routes), `frontend/src/api.js` (axios instance), `frontend/src/useApi.js` (a small `useEffect` / `useState` GET-fetch hook shared by every page), and the seven route pages under `frontend/src/pages/` : `Overview.jsx` , `DataOverview.jsx` , `Eda.jsx` , `ModelComparison.jsx` , `Forecasting.jsx` , `MlflowTracking.jsx` , `DriftMonitoring.jsx` , plus shared components in `frontend/src/components/` : `Sidebar.jsx` , `MetricCard.jsx` , `ChartCard.jsx` , `StatusBadge.jsx` , `InfoTooltip.jsx` , `Loading.jsx` .

3. Data Flow

Entry point: `python main.py [--config config.yaml]` (`main.py:35-56`).

Stage by stage, with the format at each hand-off:

1. **Raw source → raw long DataFrame** (`src/data_loading/loader.py:70-86`). Source is Kaggle M5 CSVs (`sales_train_validation.csv` , `calendar.csv` , `sell_prices.csv`), a user CSV, or generated synthetic CSV. All three paths converge on the same in-memory shape: `date (datetime), series_id (str), item_id, store_id, sales (numeric), sell_price, promotion_flag` . No file is written for this stage; it's in-memory only until preprocessing.
2. **Raw → validated** (`src/validation/validator.py:62-94`). Same DataFrame, unchanged — validation is a read-only check that writes `reports/validation_report.json` and returns a pass/fail gate. `main.py` aborts the whole run via `SystemExit` if `report["valid"]` is false.
3. **Validated → cleaned parquet** (`src/preprocessing/preprocessor.py:21-58`). Per-series reindex to a complete date range at the detected frequency, gaps filled, negatives clipped. Written to `data/processed/clean.parquet` . Same column shape as raw, now gap-free and typed.
4. **Cleaned → features parquet** (`src/features/feature_engineering.py:107-132`). Adds ~40 columns (temporal/cyclical encodings, `lag_1/7/14/30/60` , `rolling_mean/std/min/max_{7,14,30}` , decomposition proxies, price features, store/item hierarchy averages) and drops warm-up rows that lack the longest lag. Written to `data/processed/features.parquet` .
5. **Features → trained model + comparison artifacts** (`src/training/train_pipeline.py:31-118`). Walk-forward CV across 7 possible models, best one refit and tested. Outputs: `models/best_model.pkl` (a dict `{"model": ..., "feature_cols": [...]}`), `models/best_model_meta.json` , `reports/model_comparison.csv` , `reports/feature_importance.csv` , plus MLflow runs/registry entries.
6. **Cleaned data + best model → forecast CSV** (`src/api/forecast_api.py:41-75`). Recursive or direct multi-step forecast for the configured horizon (default 30 periods). Output: `reports/forecast.csv` with columns `date, series_id, forecast` .
7. **Features → drift report** (`src/monitoring/drift_detector.py:25-97` , only if `drift_detection.enabled`). Output: `reports/drift_report.json` (+ optional `.html`).
8. **Dashboard reads everything above** over HTTP GET, re-shaping each artifact into JSON for the frontend (`dashboard/server.py:57-283`). Nothing here is cached — every request re-reads the file from disk.

4. Control Flow

`main.py` is a strict, synchronous, linear script — there is no DAG scheduler, no retries, no parallelism between stages. Each function call's return value feeds directly into the next call's argument (`main.py:40-51`). The only resilience built in at this level is the Kaggle→synthetic fallback inside `load_data`, and a hard `SystemExit` if validation fails — everything else (a model failing to fit, MLflow being unreachable, Evidently not being installed) is caught locally with `try/except` + a logged warning/error and the pipeline continues (e.g. `src/training/train_pipeline.py:44-68` catches per-model-per-fold exceptions so one bad model doesn't kill the whole run, but raises `RuntimeError` if literally every model/fold combination fails).

`dashboard/server.py` is a second, independent process. Its GET endpoints are pure readers — no calls back into `src/` training code, just `pd.read_parquet` / `pd.read_csv` / `load_json` plus light in-request computation (the `/api/eda/{series_id}` endpoint runs `statsmodels` seasonal decomposition and ACF/PACF on demand, computed synchronously per request rather than precomputed — `dashboard/server.py:99-132`).

The two POST endpoints are where control flow crosses back into the pipeline: - `/api/retrain` → `src/retraining/orchestrator.retrain(config, trigger="manual (react dashboard)")` → **synchronously** reruns load→validate→clean→features→train **in the same request/response cycle** (`dashboard/server.py:240-246`, `src/retraining/orchestrator.py:34-70`). This is a significant control-flow fact: there is no background job/queue, so a retrain that takes minutes will hold the HTTP connection open the whole time (FastAPI's default sync def route runs in a threadpool, so it won't block other requests, but the calling browser request itself just waits). - `/api/drift-check` → `src/monitoring/drift_detector.detect_drift(...)`, also synchronous, reading `features.parquet` fresh each time (`dashboard/server.py:249-256`).

Error handling pattern observed throughout: library-level `try/except Exception` around anything optional or environment-dependent (Kaggle creds, `holidays` package, Evidently, `torch`/LSTM, MLflow registry calls), always followed by a `logger.warning` / `logger.error` and a graceful degraded return rather than a crash. At the API layer, missing-artifact conditions are surfaced as `HTTPException(404, ...)` via the shared `_require()` helper (`dashboard/server.py:51-54`), and retrain failures are wrapped into `HTTPException(500, str(e))` (`dashboard/server.py:245-246`). All endpoints are defined as plain (non-`async def`) functions except `/api/upload`, which is `async def` because it awaits `file.read()` — otherwise the codebase does not use asyncio concurrency at all; FastAPI's threadpool is the only concurrency mechanism in play.

5. Deployment Architecture

This is explicitly a **local development / demo platform**, not a production-hardened service, and both the README and ARCHITECTURE.md are upfront about that:

- Run today as three separate local processes, no orchestration between them: 1. `python main.py` — one-shot batch pipeline run (README.md:14-15). 2. `uvicorn dashboard.server:app --port 8000` — FastAPI dev server, and the module docstring itself says "Run: uvicorn dashboard.server:app --reload --port 8000" (`dashboard/server.py:5`), i.e. even the documented run command uses `--reload`, a dev-only flag. 3. `cd frontend && npm run dev` — Vite dev server on `http://localhost:5173` (README.md:22), not a built/served production bundle. 4. Optionally, `mlflow ui --backend-store-uri sqlite:///mlflow.db` for a fourth local process to browse experiment tracking.
- **MLflow backend is SQLite** (`mlflow.db` file in the project root), per `config.yaml:71` and ARCHITECTURE.md's explicit design note: "Zero-infra local tracking; swap `mlflow.tracking_uri` for a server later." There is no remote tracking server configured anywhere in this repo.
- **Configuration** is entirely centralized in `config.yaml` — data source, series count, CV folds, per-model hyperparameters, MLflow experiment/registry names, drift thresholds, retraining triggers, logging. There is no environment-variable-based config layering (no `.env`, no per-environment config files observed) — one file governs dev and would govern any other environment identically.
- **No containerization exists in this repo.** I searched for `Dockerfile` / `docker-compose*` / `*.dockerfile` across the whole tree and found nothing. There is no container image, no compose file, no k8s manifest. Anyone wanting to containerize this would need to author that from scratch.
- **No authentication/authorization anywhere.** `dashboard/server.py:25-26` sets CORS to `allow_origins=["*"], allow_methods=["*"], allow_headers=["*"]` — wide open, appropriate only for `localhost`-to-`localhost` dev traffic.

There are no API keys, no session/auth middleware, no rate limiting on the retrain endpoint (which is computationally expensive and freely POST-able by anyone who can reach port 8000).

- **Scaling notes**: none of this is designed to scale beyond a single machine. The dashboard holds no application state (a "pure reader" per ARCHITECTURE.md:66-67), so running multiple dashboard replicas behind a load balancer would technically work for GET traffic, but the two POST action endpoints run the actual training/drift code in-process on whichever replica receives the request — there's no shared job queue, so concurrent retrain requests would race on the same `models/best_model.pkl` / `mlflow.db` files with no locking observed in the code. `data.n_series` in `config.yaml:10` is the explicit scale knob for the pipeline itself (M5 full data is ~58M rows; top-N keeps a full run under 15 minutes per ARCHITECTURE.md:39).

Honest summary: this is a well-organized single-developer demo/prototype architecture (file-based artifact contract, synchronous pipeline, SQLite tracking), not a system that has been through any production-hardening pass — no auth, no containers, no queue, no horizontal scaling story, no environment separation beyond one YAML file.

6. Integration Points

Integration	Mechanism	Where
Kaggle API	`kagglehub.competition_download("m5-forecasting-accuracy")`	`src/data_loading/downloader.` `28` , called from `src/data_loading/loader.py:7`
MLflow	Python `mlflow` package + `MlflowClient` , tracking URI `sqlite:///mlflow.db` (`config.yaml:71`)	`src/mlflow_utils/tracker.py` `log_run` , `register_best` , `get_model_versions`); also queri by `dashboard/server.py:197-2` (`/api/mlflow`)
Evidently	`evidently.Report` / `DataDriftPreset` (≥0.7 API), fallback to `evidently.report.Report` / `evidently.metric_preset.DataDriftPreset` (legacy API)	`src/monitoring/drift_detecto` `97`
File system (artifacts contract)	Parquet (`clean.parquet` , `features.parquet`), CSV (`model_comparison.csv` , `feature_importance.csv` , `forecast.csv`), JSON (`validation_report.json` , `best_model_meta.json` , `drift_report.json`), JSONL (`retraining_log.jsonl`), pickle (`best_model.pkl`)	Paths centralized in `src/constants` `25` ; written by the pipeline modules, `dashboard/server.py` via `_json()` / `_csv()` helpers (`dashboard/server.py:29-36`)

Part 3: File Structure & Implementation Sequence

Group 1: Configuration & Entry Point

These files form the "skeleton" every other module in DemandIQ hangs off of: a single YAML file holding every tunable knob, a constants module that turns that file's on-disk assumptions into canonical `Path` objects, a loader that validates the YAML before anything trusts it, a logger that every module imports for consistent output, small serialization helpers used to persist pipeline artifacts, and `main.py`, which wires all of it into one runnable pipeline. `setup.py` and `.env.example` are packaging/secrets scaffolding around that same core.

Read in this order: 1. `config.yaml` — everything downstream reads from this; understand its sections first. 2. `src/constants.py` — defines `CONFIG_PATH` and every other path constant derived from the project root; has no dependency on config.yaml's *contents*, only its location. 3. `src/utils/config_loader.py` — the function that actually opens and validates `config.yaml` (`load_config`), using `CONFIG_PATH` from constants.py as its default. 4. `src/logger.py` — sets up logging using `LOGS_DIR` from constants.py; independent of config_loader but conventionally initialized right after config is loaded. 5. `src/utils/serialization.py` — small, standalone pickle/JSON helpers used later by training/model code; no dependency on the other four files. 6. `main.py` — ties everything together: loads config, seeds RNGs, and calls into the rest of the pipeline in sequence. 7. `setup.py` and `.env.example` are packaging/credentials context, not runtime logic — read last. 8. `src/__init__.py` is included in the file list but is currently **empty** (zero bytes) — it exists only so `src` is importable as a regular package.

FILE: CONFIG.YAML

Type: Config

Purpose: Single master YAML file holding every tunable parameter for the entire pipeline — data source selection, forecasting horizon/CV setup, feature-engineering toggles, per-model hyperparameters, MLflow settings, drift-detection thresholds, retraining triggers, logging setup, and the global random seed.

Why Created (Reasoning): Centralizes configuration so that behavior (which models run, how many series, what forecast horizon, drift thresholds, etc.) can be changed without touching code. This is the standard "one config to rule them all" pattern for ML platforms — it lets `main.py` and every pipeline stage (loading, features, training, drift) pull settings from one validated dict instead of hardcoded constants scattered across modules.

Dependencies: None (it's plain data). It is *read by* `src/utils/config_loader.py` via PyYAML.

Key Concepts: - Top-level sections: `data`, `forecasting`, `features`, `models`, `mlflow`, `drift_detection`, `retraining`, `logging`, plus a bare `random_seed` key. - `data.source` is a mode switch (`kaggle | csv | synthetic`) consumed by the data-loading module. - `models.*.enabled` flags let individual models (moving_average, linear_regression, lightgbm, xgboost, random_forest, arima, exponential_smoothing, lstm) be turned on/off independently; note `exponential_smoothing.enabled: false` — it's defined but off by default. - `mlflow.promotion_improvement_threshold: 0.02` — the 2% MAPE-improvement bar mentioned in the README for promoting a new model to Production. - `logging.log_format` duplicates the format string hardcoded in `src/logger.py` (`_FORMAT`) — see Important Notes.

Detailed Explanation: The file is organized as one YAML block per pipeline stage, in the same order the stages execute: data ingestion, forecasting/CV setup, feature engineering, models, MLflow tracking, drift detection, retraining policy, logging, and finally a global `random_seed: 42` used to make runs reproducible. Each model block under `models:` carries both an `enabled` boolean and its own hyperparameters (e.g., `lightgbm.learning_rate`, `lstm.lstm_units`), so `src/utils/config_loader.py`'s `_REQUIRED_SECTIONS` check only enforces that the *section* exists, not that every individual key is present — downstream model factory code is responsible for reading its own sub-keys with sensible defaults.

Comments embedded in the YAML (e.g., `n_series: 10 # ... M5 has 30k; keep training <15 min`) double as inline documentation explaining *why* a value was chosen, which is useful context for anyone tuning the pipeline. The file is the one artifact every other file in this group either reads directly (`config_loader.py`) or is downstream of (`main.py`, and by extension nearly every `src/*` module passed the resulting `config` dict).

Key Code Sections Explained: N/A — this is a data file, not code. Notable structural elements: - `data:` block — `source`, `dataset_name`, `csv_path` (null unless `source: csv`), `date_column`, `target_column`, `granularity: 'auto'`, `n_series: 10`. - `forecasting:` block — `forecast_horizon: 30`, `train_test_split: [0.8, 0.1, 0.1]`, `cv_folds: 3`, `cv_strategy: 'time_series_walk_forward'`. - `mlflow:` block — `tracking_uri: 'sqlite:///mlflow.db'`, `registry_model_name`, `promotion_improvement_threshold: 0.02`. - `drift_detection:` block — `framework: 'evidently'`, `statistical_test: 'ks'`, `p_value_threshold: 0.05`. - `retraining:` block — `trigger_mode: 'manual'`, `mape_degradation_threshold: 0.15`, `drift_feature_count_trigger: 3`.

How To Use/Call This File: Never imported directly by Python — always accessed through `load_config()` in `src/utils/config_loader.py`, which defaults to reading it via `CONFIG_PATH` (`src/constants.py`). `main.py` calls `load_config(Path(config_path))` with a `--config` CLI argument that defaults to `"config.yaml"`, so a user can point the pipeline at an alternate config file without editing code. Every pipeline function `main.py` calls (`load_data`, `run_validation`, `clean`, `build_features`, `run_training`, `generate_forecast`, `detect_drift`) receives the resulting dict as its `config` parameter.

Important Notes: - `logging.log_format` in this YAML is **not actually consumed** by `src/logger.py` — that module hardcodes its own identical-looking `_FORMAT` string rather than reading `config["logging"]["log_format"]`. The two are kept in sync manually; if someone edits one without the other, they will silently diverge. Likewise `logging.level` and `logging.log_file` in the YAML are not read by `_configure()` in `logger.py` (which hardcodes `"INFO"` and always writes to `LOGS_DIR / "demandiq.log"`). This is a real gap between config intent and code behavior — worth flagging rather than assuming they're wired up. - `csv_path: null` — only meaningful when `data.source == 'csv'`; otherwise ignored. - `_REQUIRED_SECTIONS` in `config_loader.py` does not include `logging` or `random_seed`, so a config missing those two would still pass validation (`main.py` uses `config.get("random_seed", 42)`, tolerating its absence).

FILE: MAIN.PY

Type: Core Logic (Entry Point)

Purpose: The single end-to-end pipeline entry point: load config → seed RNGs → load data → validate → clean → build features → train/CV/MLflow → forecast → drift detection, with logging at each major milestone.

Why Created (Reasoning): Every ML platform needs one script that runs the whole thing reproducibly end-to-end (as opposed to notebooks or ad hoc scripts). This file exists so `python main.py` (optionally with `--config`) is the canonical way to execute the full DemandIQ pipeline, and so that all pipeline stages are composed in the correct order with a clear fail-fast point (schema validation) before expensive work (training) begins.

Dependencies: - stdlib: `argparse`, `random`, `pathlib.Path` - third-party: `numpy` (as `np`) - optional third-party: `torch` (imported lazily inside `set_seeds`, tolerated if absent) - project: `src.api.forecast_api.generate_forecast`, `src.data_loading.loader.load_data`, `src.features.feature_engineering.build_features`, `src.logger.get_logger`, `src.monitoring.drift_detector.detect_drift`, `src.preprocessing.preprocessor.clean`, `src.training.train_pipeline.run_training`, `src.utils.config_loader.load_config`, `src.validation.validator.run_validation`

Key Concepts: - `set_seeds(seed)` — reproducibility helper seeding Python's `random`, `numpy`, and (if installed) `torch`. - `main(config_path="config.yaml")` — the orchestration function; not wrapped in try/except, so any stage's exception propagates and halts the script. - Fail-fast validation gate: `if not report["valid"]: raise SystemExit(...)` — stops the pipeline before wasting time on cleaning/training if the raw data fails schema checks. - `if __name__ == "__main__":` block with `argparse` for the `--config` CLI flag.

Detailed Explanation: `main()` is a straight-line, ten-step orchestration function with no branching except the two `if` guards (validation failure, drift-detection enabled flag). It starts by calling `load_config(Path(config_path))` to get the validated config dict, then `set_seeds(config.get("random_seed", 42))` to make the run deterministic across `random`, `numpy`, and (best-effort) `torch`. From there it calls, in strict sequence: `load_data(config)` → `run_validation(raw)` → (fail-fast check) → `clean(raw, config)` → `build_features(cleaned, config)` → `run_training(features, config)` → `generate_forecast(cleaned, config)` → conditionally `detect_drift(features, config)` if `config["drift_detection"].get("enabled", True)`.

Note that `generate_forecast` is called with `cleaned` (pre-feature-engineering data), not `features` — meaning forecasting works from the cleaned raw series rather than the engineered feature matrix; this is a detail worth confirming against `src/api/forecast_api.py` if the reader needs to understand exactly what forecasting consumes. The function logs a start banner, a completion banner with the best model name and test MAPE (pulled out of `summary["best_model"]` and `summary["test_metrics"]["mape"]`, which is the contract `run_training` must return), and finally a static log line reminding the user how to launch the dashboard (`uvicorn dashboard.server:app --port 8000` + `cd frontend && npm run dev`) — this is a hardcoded operator hint, not an automatic launch.

The `if __name__ == "__main__":` block is the CLI surface: it builds an `argparse.ArgumentParser`, adds a single `--config` option defaulting to `"config.yaml"`, and calls `main(parser.parse_args().config)`. This makes the module both a runnable script and, in principle, an importable function (`from main import main`) for programmatic invocation (e.g., from a notebook or test), though nothing in the read files currently does that.

Key Code Sections Explained: - `set_seeds(seed: int) -> None` — seeds `random.seed(seed)` and `np.random.seed(seed)` unconditionally, then attempts `import torch; torch.manual_seed(seed)` inside a `try/except ImportError: pass`. Important because it's the only mechanism keeping model training (especially the LSTM, which uses PyTorch per the README) reproducible across runs; the silent `except ImportError` means torch seeding fails silently (not loudly) if torch isn't installed, which is intentional since torch/LSTM is one of several optional models. - `main(config_path: str = "config.yaml") -> None` — the orchestration function described above; every stage function receives either `config` alone or `(data, config)`, establishing the convention (also seen in config_loader.py) that `config` is the dict passed everywhere rather than individual keyword arguments. - The validation gate — `report = run_validation(raw); if not report["valid"]: raise SystemExit(f"Data validation failed: {report['schema_errors']}")`. This is the pipeline's one hard stop: it assumes `run_validation` returns a dict with `valid` (bool) and `schema_errors` keys, a contract defined in `src/validation/validator.py` (not read in this batch, but the contract is visible here). - `argparse` CLI block — minimal, single-flag parser; no subcommands, no environment-variable override, so config path must be passed positionally via `--config`.

How To Use/Call This File: Run directly from the repo root: `python main.py` or `python main.py --config path/to/other_config.yaml`. This is the documented Quick-Start command in `README.md`. It is not designed to be imported as a library in the current codebase (no other file in this batch imports from `main`), though its `main()` function is technically importable since it's a plain function, not gated behind `if __name__ == "__main__"`.

Important Notes: - No exception handling around the pipeline stages other than the explicit validation-failure `SystemExit` — any other exception (bad data source, missing MLflow backend, etc.) will produce a raw traceback and abort the script. There's no partial-resume capability; a failure partway through means rerunning from the top. - The `torch` import is deliberately deferred to inside `set_seeds` (not top-level) — this keeps `main.py`'s hard dependencies limited to `numpy`, and makes the LSTM/torch dependency soft at this entry-point level, even though the rest of the pipeline may still require torch to be installed for the LSTM model to actually run. - `generate_forecast(cleaned, config)` uses `cleaned`, not `features` — a detail easy to misread; worth double-checking against the forecast API's expectations if extending this file. - The dashboard/frontend launch commands are just log messages, not `subprocess` calls — nothing in `main.py` starts the dashboard automatically.

FILE: SRC/CONSTANTS.PY

Type: Config / Utility

Purpose: Defines canonical, absolute `Path` objects for every directory and file the pipeline reads or writes (data, models, logs, reports), plus canonical column-name constants, all derived from the project root — and ensures the relevant directories exist at import time.

Why Created (Reasoning): Prevents every module in `src/` from independently guessing relative paths (which breaks depending on current working directory) or hardcoding column names like `"date"` / `"sales"` in multiple places. Centralizing these as constants means a rename or restructuring only requires editing one file, and importing this module has the side effect of guaranteeing the directory tree exists before anything tries to write into it.

Dependencies: stdlib only — `pathlib.Path`. No project imports (this is a leaf module; everything else depends on it, it depends on nothing internal).

Key Concepts: - `PROJECT_ROOT = Path(__file__).resolve().parent.parent` — computed once, relative to this file's own location (`src/constants.py` → parent is `src/`, parent.parent is the repo root), making all subsequent paths independent of the current working directory. - Directory constants: `DATA_DIR`, `RAW_DIR`, `PROCESSED_DIR`, `EXTERNAL_DIR`, `MODELS_DIR`, `LOGS_DIR`, `REPORTS_DIR`. - File-path constants for pipeline artifacts: `CLEAN_DATA_PATH`, `FEATURES_PATH`, `VALIDATION_REPORT_PATH`, `COMPARISON_PATH`, `FORECAST_PATH`, `FEATURE_IMPORTANCE_PATH`, `DRIFT_REPORT_PATH`, `DRIFT_HTML_PATH`, `RETRAIN_LOG_PATH`, `BEST_MODEL_PATH`, `BEST_MODEL_META_PATH`. - Canonical long-format column names: `DATE_COL = "date"`, `SERIES_COL = "series_id"`, `TARGET_COL = "sales"`. - `CONFIG_PATH = PROJECT_ROOT / "config.yaml"` — the default path `load_config()` reads from. - Module-level side effect: a `for` loop at import time (`for _d in (...): _d.mkdir(parents=True, exist_ok=True)`) creates every core directory if missing.

Detailed Explanation: This file has no functions or classes — it's pure module-level constant definitions plus one directory-creation side effect, executed once on first import (and cached by Python's module system on subsequent imports). The path hierarchy models the physical layout of the repo: `data/{raw,processed,external}`, `models/`, `logs/`, `reports/`, all rooted at `PROJECT_ROOT`. Each named artifact path (e.g., `FEATURES_PATH = PROCESSED_DIR / "features.parquet"`) documents, just by existing as a named constant, exactly what artifact each pipeline stage is expected to produce — a reader can infer the pipeline's data-flow contract (raw → clean.parquet → features.parquet → model outputs → reports) purely from this file's constant names, without reading the stages themselves.

The three canonical column-name constants (`DATE_COL`, `SERIES_COL`, `TARGET_COL`) establish the "long format" convention referenced in `config.yaml`'s comment (`# used when source == csv (long format: date, series_id, sales)`) and in the module docstring ("Canonical long-format column names used across the whole pipeline"). Any module that needs to reference the sales column, rather than hardcoding `"sales"`, is expected to import `TARGET_COL` from here — keeping the schema consistent even though `config.yaml` also lets a user rename `date_column` / `target_column` for their own CSV.

The trailing loop that creates directories runs as an import-time side effect — this is a deliberate but slightly unusual pattern: simply `import src.constants` anywhere in the codebase is enough to guarantee `data/raw`, `data/processed`, `data/external`, `models/`, `logs/`, and `reports/` exist on disk, with `mkdir(parents=True, exist_ok=True)` making it idempotent and safe to run repeatedly (e.g., in tests or notebooks).

Key Code Sections Explained: - `PROJECT_ROOT = Path(__file__).resolve().parent.parent` — the anchor for every other path in the file; because it's derived from `__file__` rather than `Path.cwd()`, the pipeline can be invoked from any working directory (e.g., a CI job running from a different folder) and still resolve paths correctly. This is the key reason this module has to be imported before path-dependent code runs. - The directory-creation loop `for _d in (RAW_DIR, PROCESSED_DIR, EXTERNAL_DIR, MODELS_DIR, LOGS_DIR, REPORTS_DIR): _d.mkdir(parents=True, exist_ok=True)` — a module-level side effect (not wrapped in a function), meaning it runs exactly once, automatically, the first time anything imports `src.constants` (directly or transitively). Important because `src/logger.py` depends on `LOGS_DIR` existing before its `RotatingFileHandler` can open a file there — this loop is what guarantees that. - The artifact path constants (`CLEAN_DATA_PATH` through `BEST_MODEL_META_PATH`) — each is a `Path` built by `/` composition on a directory constant; collectively they define the full on-disk contract between pipeline stages (what `preprocessing` writes, what `training` reads/writes, what `monitoring` / `retraining` read/write).

How To Use/Call This File: Imported wherever a canonical path or column name is needed, e.g. `from src.constants import CONFIG_PATH` (used in `src/utils/config_loader.py`) or `from src.constants import LOGS_DIR` (used in `src/logger.py`). Modules needing multiple constants just add them to the same `from src.constants import ...` line. Because of the import-

time directory-creation side effect, no other module needs to call `.mkdir()` itself for these standard directories.

Important Notes: - Because directory creation happens as a bare module-level side effect (not inside a function guarded by `if __name__ == "__main__"` or similar), simply importing this module anywhere — including from a test file or a notebook — will create real directories on disk. This is convenient but means the module has a filesystem side effect on import, which is slightly unusual and worth knowing if writing unit tests that shouldn't touch the filesystem (e.g., in a read-only CI sandbox). - `_d` is a throwaway loop variable prefixed with underscore by convention (not exported/used elsewhere), consistent with `_FORMAT` / `_configured` naming style seen in `src/logger.py`. - The file assumes `src/constants.py` itself never moves relative to the repo root — moving it would silently change what `PROJECT_ROOT` resolves to.

FILE: SRC/UTILS/CONFIG_LOADER.PY

Type: Utility (Config Validation)

Purpose: Loads `config.yaml` from disk with PyYAML and performs a minimal structural check that all required top-level sections are present, raising clear errors if the file is missing or malformed.

Why Created (Reasoning): Rather than letting a missing or incomplete config file fail deep inside some pipeline stage with a confusing `KeyError`, this module centralizes config loading and validation so failures happen immediately, at startup, with an informative message naming exactly which section is missing. This is the standard "load once, pass around as a dict" pattern noted in the module's own docstring.

Dependencies: - stdlib: `pathlib.Path` - third-party: `yaml` (PyYAML), specifically `yaml.safe_load` - project: `src.constants.CONFIG_PATH` (used as the default argument for `path`)

Key Concepts: - `_REQUIRED_SECTIONS` — a module-level tuple of the seven section names that must exist: `("data", "forecasting", "features", "models", "mlflow", "drift_detection", "retraining")`. - `load_config(path: Path = CONFIG_PATH) -> dict` — the single public function; defaults to the project's canonical config path but accepts an override (used by `main.py`'s `--config` flag). - Explicit, typed exceptions: `FileNotFoundError` for a missing file, `ValueError` for missing sections — both documented in the function's docstring under a NumPy-style `Raises` section.

Detailed Explanation: `load_config` does three things in sequence: (1) checks `Path(path).exists()` and raises `FileNotFoundError` with the attempted path in the message if not found; (2) opens the file with explicit `encoding="utf-8"` and parses it via `yaml.safe_load(f)` into a plain Python dict; (3) computes `missing = [s for s in _REQUIRED_SECTIONS if s not in cfg]` and raises `ValueError(f"config.yaml missing sections: {missing}")` if any required section is absent, otherwise returns `cfg` unchanged.

The validation is intentionally shallow — it only checks that the seven named top-level keys exist in the parsed dict; it does not validate the *shape* or *types* of values within each section (e.g., it wouldn't catch a `models.lightgbm.n_estimators` set to a string instead of an int, or a missing `forecast_horizon`). That deeper validation, if it exists at all, is left to whichever downstream module actually consumes each section (e.g., a model factory reading `config["models"]["lightgbm"]`). Notably, `logging` and `random_seed` — both present in `config.yaml` and both used elsewhere (`main.py` reads `random_seed` via `.get()` with a default) — are *not* in `_REQUIRED_SECTIONS`, meaning a config file lacking them would still pass validation.

Key Code Sections Explained: - `_REQUIRED_SECTIONS = ("data", "forecasting", "features", "models", "mlflow", "drift_detection", "retraining")` — module-level constant defining the validation contract; because it's a plain tuple checked with `s not in cfg`, adding a new required section to future config schemas means editing this one line, not scattering checks through the codebase. - `load_config(path: Path = CONFIG_PATH) -> dict` — the function signature itself is important: the default parameter is evaluated once at import time to `CONFIG_PATH` (from `src.constants`), meaning callers who just do `load_config()` with no arguments always get the canonical project config, while `main.py` explicitly passes `Path(config_path)` (from the `--config` CLI flag) to override it. - The `FileNotFoundError` / `ValueError` raises — both include the actual problematic value (the path or the list of missing sections) directly in the message, which is a small but deliberate usability choice: the caller gets an actionable error without needing a debugger.

How To Use/Call This File: `from src.utils.config_loader import load_config` then `config = load_config()` (uses default `config.yaml`) or `config = load_config(Path("custom.yaml"))`. This is exactly the pattern `main.py` uses: `config = load_config(Path(config_path))`. The returned `dict` is then passed by reference into every subsequent pipeline function (`load_data(config)`, `run_validation(raw)` — note this one doesn't take config — `clean(raw, config)`, `build_features(cleaned, config)`, `run_training(features, config)`, `generate_forecast(cleaned, config)`, `detect_drift(features, config)`), establishing the "load once, pass around as a dict" convention named in the file's own module docstring. Other consumers per the earlier grep include `dashboard/server.py` and several of the notebooks.

Important Notes: - `yaml.safe_load` is used rather than `yaml.load` — this is a deliberate security-conscious choice (safe_load won't execute arbitrary Python object constructors embedded in a YAML file), appropriate since this reads a config file that could in principle be edited by less-trusted contributors. - Validation is shape-shallow: presence of the seven listed section keys only, not their internal schema. A `config.yaml` with `mlflow: {}` (empty dict) would pass this validation and only fail later, deep inside whatever module tries to read `config["mlflow"]["tracking_uri"]`. - `_REQUIRED_SECTIONS` omits `logging` and `random_seed`, both of which do appear in the real `config.yaml` and are used elsewhere in the codebase — so this list should be read as "sections without which most of the pipeline cannot function," not "every top-level key that config.yaml is expected to have."

FILE: SRC/LOGGER.PY

Type: Utility (Cross-cutting / Core Logic)

Purpose: Provides a single centralized logging setup — console output plus a rotating log file — configured exactly once no matter how many modules request a logger, via the module-level `get_logger(name)` function.

Why Created (Reasoning): Without centralization, every module would either configure its own handlers (risking duplicate log lines if multiple modules call `logging.basicConfig`) or produce inconsistent formatting. This module solves that by guarding the actual handler setup behind a one-time `_configured` flag and giving every caller a consistently-namespaced logger (`demandiq.<module_name>`), so log output across the whole pipeline (data loading, training, drift detection, dashboard, etc.) is uniform and goes to both the console and a shared rotating file.

Dependencies: - stdlib: `logging`, `logging.handlers` - project: `src.constants.LOGS_DIR` (determines where the rotating log file is written)

Key Concepts: - Module-level state: `_FORMAT` (hardcoded format string) and `_configured` (boolean guard against re-configuring handlers). - `_configure(level: str = "INFO") -> None` — private setup function; idempotent via the `_configured` guard and early return. - `get_logger(name: str) -> logging.Logger` — the public API; calls `_configure()` (a no-op after the first call) then returns `logging.getLogger(f"demandiq.{name}")`. - Handler stack attached to the `"demandiq"` root-ish logger: a `logging.StreamHandler()` (console) and a `logging.handlers.RotatingFileHandler` (file, 5,000,000-byte cap, 3 backups, UTF-8 encoded), both sharing one `Formatter`.

Detailed Explanation: The module defines a private `"demandiq"` logger (retrieved via `logging.getLogger("demandiq")`, not the true Python root logger) that `_configure()` sets up exactly once: it sets the level, builds one `Formatter` from `_FORMAT`, then creates and attaches a `StreamHandler` (console) and a `RotatingFileHandler` pointed at `LOGS_DIR / "demandiq.log"` with `maxBytes=5_000_000` and `backupCount=3`. The `_configured` global flag, checked and set inside `_configure()`, ensures that even if `get_logger()` is called from dozens of different modules across the codebase, handlers are attached only once — avoiding the classic "duplicate log lines" bug that happens when logging setup runs on every import.

`get_logger(name)` is the only function other modules are meant to call. Because it namespaces every logger under `demandiq.<name>` (e.g., `demandiq.src.training.train_pipeline` if called with `get_logger(__name__)`), all loggers created this way are children of the single configured `"demandiq"` logger and inherit its handlers automatically via Python's standard logger hierarchy/propagation — this is *why* only the `"demandiq"` logger itself needs handlers attached; child loggers don't need their own.

This file is a leaf-level cross-cutting utility: virtually every `src/*` module (per the earlier grep: `main.py`, `retraining/orchestrator.py`, `monitoring/performance_monitor.py`, `monitoring/drift_detector.py`, `api/forecast_api.py`, `features/feature_engineering.py`, `training/train_pipeline.py`, `mlflow_utils/tracker.py`,

`training/cross_validator.py`, `models/model_factory.py`, `models/lstm_model.py`, `models/statistical_models.py`, `preprocessing/preprocessor.py`, `validation/validator.py`, `data_loading/loader.py`, `data_loading/downloader.py`) imports `get_logger` at the top and calls `logger = get_logger(__name__)` once at module scope, following the exact pattern seen in `main.py`.

Key Code Sections Explained: - `_configure(level: str = "INFO") -> None` — the guarded setup function. The `if _configured: return` at the top makes repeated calls (one per module import, since every module calls `get_logger` at import time) safe and cheap. Notably, the `level` parameter is **never actually passed a non-default value anywhere in this file** — `get_logger` calls `_configure()` with no arguments, so the level is always `"INFO"` in practice, even though `config.yaml` has a `logging.level` key that looks like it should control this. This is the same gap flagged under `config.yaml`'s Important Notes: the YAML's `logging` section is not currently wired into this function. - `get_logger(name: str) -> logging.Logger` — the public entry point every other module calls. Its docstring says "configures handlers on first call," accurately describing the lazy-initialization pattern: the first call anywhere in the process triggers `_configure()`; every subsequent call across any module is just a cheap `logging.getLogger(...)` lookup. - The `RotatingFileHandler` construction — `logging.handlers.RotatingFileHandler(LOGS_DIR / "demandiq.log", maxBytes=5_000_000, backupCount=3, encoding="utf-8")` — caps the log file at ~5MB before rotating, keeping up to 3 rotated backups (so up to ~20MB of log history total). This depends on `LOGS_DIR` already existing, which is guaranteed by `src/constants.py`'s import-time `mkdir` side effect.

How To Use/Call This File: Standard usage in any module: `from src.logger import get_logger` then, at module scope, `logger = get_logger(__name__)`, followed by calls like `logger.info(...)`, `logger.error(...)` etc. anywhere in that module. This is exactly the pattern in `main.py`: `logger = get_logger(__name__)` at the top, then `logger.info("=== DemandIQ pipeline start ===")` and similar calls throughout `main()`.

Important Notes: - The `level` parameter of `_configure()` is dead in practice — nothing calls `_configure(level=...)` with a real value, so `config.yaml`'s `logging.level: 'INFO'` setting has no live effect on this module; the level is hardcoded to `"INFO"` via the default parameter. Anyone wanting configurable log levels would need to thread `config["logging"]["level"]` into `get_logger` / `_configure`, which isn't done today. - Similarly, `_FORMAT` in this file is a hardcoded duplicate of `config.yaml`'s `logging.log_format` string — they currently match character-for-character, but nothing enforces that; editing one without the other silently desyncs config and behavior. - Because `_configured` is a module-level global (not thread-safe by construction), concurrent first-time imports from multiple threads could theoretically race on handler setup, though this is unlikely to matter for a single-process batch pipeline like this one. - The logger name `"demandiq"` used internally is not the true Python root logger — it's a named logger that all `demandiq.*` child loggers roll up into via propagation, meaning if code elsewhere calls `logging.getLogger()` (the bare root) or configures the actual root logger, it would not automatically share these handlers.

FILE: SRC/UTILS/SERIALIZATION.PY

Type: Utility

Purpose: Four small, generic functions for persisting Python objects to disk — `save_pickle` / `load_pickle` for arbitrary objects (models, preprocessors) and `save_json` / `load_json` for JSON-serializable metadata/reports — each save function auto-creating its parent directory first.

Why Created (Reasoning): Nearly every pipeline stage needs to persist something to disk (a trained model, a fitted preprocessor, a metrics report, a feature-importance table) and reload it later (the dashboard reading pipeline artifacts, retraining comparing against a previous best model). Rather than repeating "open file, dump, handle missing parent dir" logic in every module, this file provides that as four tiny reusable helpers — the minimal amount of code needed, not a bespoke serialization framework.

Dependencies: stdlib only — `json`, `pickle`, `pathlib.Path`, `typing.Any`. No project imports; this is a leaf utility module.

Key Concepts: - `save_pickle(obj: Any, path: Path) -> None` / `load_pickle(path: Path) -> Any` — binary pickle round-trip for arbitrary Python objects (trained model instances, fitted scalers, etc.). - `save_json(obj: Any, path: Path) -> None` / `load_json(path: Path) -> Any` — UTF-8 JSON round-trip for structured/inspectable data (reports, metadata). - Both save

functions call `path.parent.mkdir(parents=True, exist_ok=True)` before writing — callers never need to pre-create the destination directory. - `save_json` uses `json.dump(obj, f, indent=2, default=str)` — pretty-printed with a `str` fallback for non-JSON-native types (e.g., `datetime`, `Path`, `numpy` scalars) instead of raising `TypeError`.

Detailed Explanation: This is a deliberately minimal module — four functions, no classes, no configuration, no error handling beyond what `open()` / `pickle` / `json` raise natively. `save_pickle` and `save_json` both follow the same shape: coerce `path` to a `Path`, ensure the parent directory exists, then write. `load_pickle` and `load_json` are the inverse — just open and deserialize, with no existence check of their own (a missing file raises the standard `FileNotFoundError` from `open()`).

The choice to split pickle vs. JSON serialization reflects the two different kinds of artifacts the pipeline produces: model objects and fitted transformers are inherently Python objects (not JSON-representable) and need pickle, per `constants.py`'s `BEST_MODEL_PATH = MODELS_DIR / "best_model.pkl"`; whereas metadata, reports, and comparison tables are meant to be human-readable and cross-tool-compatible, matching `BEST_MODEL_META_PATH`, `VALIDATION_REPORT_PATH`, and `DRIFT_REPORT_PATH` all having `.json` in their names (per `src/constants.py`). The `default=str` fallback in `save_json` is a small but important pragmatic touch — it means the caller doesn't need to manually convert non-native types (timestamps, `Path` objects, numpy scalars) before serializing; they get stringified automatically rather than raising `TypeError: Object of type X is not JSON serializable`.

Key Code Sections Explained: - `save_pickle(obj: Any, path: Path) -> None` — `path = Path(path); path.parent.mkdir(parents=True, exist_ok=True); with open(path, "wb") as f: pickle.dump(obj, f)`. Important because it's the mechanism by which the "best model" (per `constants.BEST_MODEL_PATH`) gets persisted after training, and later reloaded by the dashboard or retraining orchestrator for comparison/serving. - `load_pickle(path: Path) -> Any` — a direct `pickle.load` with no type checking on what comes back; the caller is trusted to know what kind of object was pickled at that path. No defensive coding here — deliberately minimal. - `save_json(obj: Any, path: Path) -> None` — same directory-creation pattern as `save_pickle`, then `json.dump(obj, f, indent=2, default=str)`. The `indent=2` makes saved reports (e.g., `validation_report.json`, `drift_report.json`) human-readable when opened directly, which matters since these are meant to be inspected by a developer or displayed by the dashboard, not just round-tripped by code. - `load_json(path: Path) -> Any` — mirrors `load_pickle`'s simplicity; a bare `json.load(f)` with UTF-8 decoding, no schema validation.

How To Use/Call This File: `from src.utils.serialization import save_pickle, load_pickle, save_json, load_json`, then e.g. `save_pickle(trained_model, BEST_MODEL_PATH)` (combining with the path constants from `src/constants.py`) or `report = load_json(VALIDATION_REPORT_PATH)`. Per the earlier grep, this pattern is used across `training/train_pipeline.py`, `training/cross_validator.py`, `models/*.py`, `preprocessing/preprocessor.py`, `validation/validator.py`, `data_loading/*.py`, `monitoring/*.py`, `retraining/orchestrator.py`, `api/forecast_api.py`, `features/feature_engineering.py`, `mlflow_utils/tracker.py`, and `dashboard/server.py` — i.e., essentially every stage that produces or consumes a persisted artifact.

Important Notes: - `pickle` is inherently insecure to unpickle from untrusted sources (arbitrary code execution on load) — acceptable here since all pickled files are produced by this same trusted pipeline, but worth flagging if `best_model.pkl` were ever loaded from an external/untrusted source. - No error handling wraps the file operations — a corrupted pickle file, a permissions error, or invalid JSON will raise the underlying library's native exception (`pickle.UnpicklingError`, `json.JSONDecodeError`, `PermissionError`) directly up to the caller; there's no custom exception type or logging here. - `load_pickle` / `load_json` do not call `.mkdir()` (unlike their save counterparts) since loading doesn't need to create directories — this asymmetry is intentional, not an oversight.

FILE: SETUP.PY

Type: Config (Packaging)

Purpose: Standard `setuptools` packaging script that declares DemandIQ as an installable Python package named `demandiq`, discovering all packages under `src/` and reading its install dependencies from `requirements.txt`.

Why Created (Reasoning): Lets the project be installed (e.g., `pip install -e .`) so that `src` and its submodules are importable from anywhere without manual `sys.path` manipulation or always running from the repo root — standard practice for any non-trivial Python project, especially one with nested packages like `src.data_loading`, `src.models`, etc.

Dependencies: stdlib: `pathlib.Path`. Third-party (build-time only): `setuptools` (`find_packages`, `setup`). Reads `requirements.txt` at setup-time via `Path("requirements.txt").read_text().splitlines()`.

Key Concepts: - `setup(name=..., version=..., description=..., packages=..., python_requires=..., install_requires=...)` — the single `setuptools.setup()` call that declares the package. - `find_packages(include=["src", "src.*"])` — auto-discovers `src` and every submodule under it (`src.data_loading`, `src.models`, `src.utils`, etc.) as installable packages, rather than listing them by hand. - `python_requires=">=3.10"` — enforces a minimum Python version. - `install_requires=Path("requirements.txt").read_text().splitlines()` — dependencies are declared once in `requirements.txt` and reused here rather than duplicated inline.

Detailed Explanation: This is a short, conventional `setuptools`-based `setup.py`. It names the package `demandiq`, version `1.0.0`, with a one-line description matching the README's tagline ("Automated demand forecasting & ML experimentation platform"). `packages=find_packages(include=["src", "src.*"])` is the key line: it tells `setuptools` to scan the repo for any directory matching `src` or `src.<anything>` that contains an `__init__.py` (making it a valid Python package) and include it in the built distribution — this is why `src/__init__.py` (even though empty) needs to exist, and why every subpackage under `src/` (data_loading, models, utils, etc.) presumably also has its own `__init__.py`.

`install_requires=Path("requirements.txt").read_text().splitlines()` is a small but notable pattern: rather than hardcoding a dependency list inside `setup.py` (which would then need to be kept in sync with `requirements.txt`), it reads `requirements.txt` directly at setup-time and splits it into a list of requirement strings. This means the file is read relative to wherever `setup.py` is invoked from (i.e., it assumes `pip install .` / `python setup.py` is run from the repo root, where `requirements.txt` lives next to `setup.py`) — it does not use an absolute path via `src.constants.PROJECT_ROOT`, unlike most other path-handling in this codebase.

Key Code Sections Explained: - `packages=find_packages(include=["src", "src.*"])` — the core packaging directive. Important because it determines exactly what gets installed/importable; if a new top-level package were added outside `src/` (e.g., a hypothetical `dashboard` package), it would need its own explicit include pattern or it wouldn't be picked up (note `dashboard/server.py` exists per the README's project structure, and is not covered by this `include` pattern — it appears `dashboard` is treated as a standalone app, not part of the installable `demandiq` package). - `install_requires=Path("requirements.txt").read_text().splitlines()` — ties `setup.py`'s dependency list directly to `requirements.txt` instead of duplicating it; a maintainer only needs to update one file. Note this uses a **relative** path (`Path("requirements.txt")`, not anchored to any project-root constant), so it only works correctly when the working directory is the repo root at setup/build time. - `python_requires=">=3.10"` — a version floor, presumably chosen because of a language feature used elsewhere in the codebase (e.g., structural pattern matching, or a newer typing feature) or because of a dependency's own minimum Python requirement.

How To Use/Call This File: Not imported by any other Python file — it's invoked by `pip` or `setuptools` tooling directly: `pip install -e .` (editable install) or `python setup.py install` / `build`. This is standard packaging plumbing that makes `import src...` work reliably regardless of the caller's current directory, once installed.

Important Notes: - `Path("requirements.txt").read_text()` will raise `FileNotFoundError` if `setup.py` is invoked from a directory other than the repo root (since it's a relative path) — this is a fragility worth knowing if packaging/CI ever runs `setup.py` from an unexpected working directory. - The README's Quick Start does not mention `pip install -e .` or `setup.py` at all — it only shows `pip install -r requirements.txt` and `python main.py`, suggesting `setup.py` may be present for completeness/distribution purposes but isn't part of the primary documented workflow; worth confirming with the maintainer whether it's actively used. - `dashboard/` (a FastAPI app per the README) is not included in `find_packages`'s pattern, implying it's run separately (via `uvicorn dashboard.server:app`) rather than through the installed `demandiq` package.

FILE: .ENV.EXAMPLE

Type: Config (Secrets Template)

Purpose: A template showing the two environment variables (`KAGGLE_USERNAME`, `KAGGLE_KEY`) needed to authenticate with the Kaggle API for downloading the M5 forecasting dataset, meant to be copied to a real `.env` (or equivalent) and filled in with actual credentials.

Why Created (Reasoning): Standard practice for keeping secrets out of version control: rather than documenting required environment variables only in prose, a `.env.example` file gives contributors a ready-to-copy template (`cp .env.example .env`) so they know exactly which variable names the code expects, without ever committing real credentials.

Dependencies: None directly — it's a template, not code. The variables it documents (`KAGGLE_USERNAME`, `KAGGLE_KEY`) are presumably read by the Kaggle API client library (likely inside `src/data_loading/downloader.py`, not read in this batch) either via `os.environ` or via the Kaggle API's own credential resolution.

Key Concepts: - `KAGGLE_USERNAME` / `KAGGLE_KEY` — the two credential fields the official Kaggle API expects (matching the fields normally found in a `~/.kaggle/kaggle.json` file). - The comment on line 1 (`# Kaggle API credentials (or place kaggle.json in ~/.kaggle/)`) documents that this is one of *two* supported ways to authenticate — env vars or the conventional `kaggle.json` file — consistent with `config.yaml`'s `data.source: 'kaggle'` option and the README's "Kaggle setup (optional)" section.

Detailed Explanation: This is a two-line template file, not executable code. It exists purely to document the shape of credentials the pipeline's Kaggle-download path expects, without embedding real secrets in the repo. Per the README's Troubleshooting table ("Kaggle 403 / no credentials | Pipeline auto-falls back to synthetic data..."), these credentials are entirely optional — if absent, `config.yaml`'s `data.source: 'kaggle'` path presumably fails gracefully and the pipeline falls back to synthetic data generation instead of hard-failing. This file, `config.yaml`'s `data.source` option, and the (unread-in-this-batch) `src/data_loading/downloader.py` module together form the Kaggle-integration surface, but this env file specifically only concerns *credentials*, not dataset selection (that's `config.yaml`'s `data.dataset_name`).

Key Code Sections Explained: N/A — no code, just two `KEY=value` template lines with placeholder values (`your_username`, `your_key`).

How To Use/Call This File: A developer copies this to a real `.env` file (e.g., `cp .env.example .env`) and fills in actual Kaggle credentials, or alternatively skips this entirely and places a `kaggle.json` file in `~/.kaggle/` per the Kaggle CLI's own convention (as the inline comment notes). Nothing in the four core files read in this batch (`main.py`, `constants.py`, `config_loader.py`, `logger.py`) actually loads a `.env` file (no `python-dotenv` import seen); that loading, if it happens, would occur inside the Kaggle downloader module itself or be handled by the Kaggle library's own environment-variable lookup — this wasn't confirmed since `src/data_loading/downloader.py` was not part of this reading batch.

Important Notes: - This file is a *template* — presumably `.env` itself (the real, filled-in version) is git-ignored; that was not verified in this batch since no `.gitignore` was read. - It's ambiguous from the files read here whether the actual `.env` is loaded via `python-dotenv`, shell export, or direct `os.environ` reliance — no dotenv-loading code appeared in `main.py` or any of the other core files read. Confirming this would require reading `src/data_loading/downloader.py`. - Only Kaggle credentials are templated here; there's no equivalent entry for MLflow, database, or other service credentials, consistent with the project being a local-first ML pipeline (SQLite-backed MLflow per `config.yaml`'s `tracking_uri: 'sqlite:///mlflow.db'`) with no other external services in this configuration file.

FILE: SRC/INIT.PY

Type: Utility (Package Marker)

Purpose: Marks the `src` directory as a regular, importable Python package.

Why Created (Reasoning): Required so that `import src.constants`, `from src.utils.config_loader import load_config`, etc. work as regular (non-namespace) package imports, and so `setup.py`'s `find_packages(include=["src", "src.*"])` recognizes `src` as a package to install.

Dependencies: None.

Key Concepts: N/A.

Detailed Explanation: This file is currently **empty (0 bytes)** — confirmed by direct read, which returned a "file exists but contents are empty" notice rather than any content. This is entirely normal and by far the most common form of `__init__.py`: it carries no package-level exports, no version string, no re-exports of submodules — it exists solely so that `src/` is treated as a package

by Python's import system and by `setuptools`' `find_packages`.

Key Code Sections Explained: None — the file has no content.

How To Use/Call This File: Never referenced directly; its mere presence is what allows every `from src.xxx import yyy` / `import src.xxx` statement throughout the codebase (seen in every other file in this batch) to work. Every subpackage referenced elsewhere (`src.data_loading`, `src.utils`, `src.models`, `src.features`, `src.training`, `src.monitoring`, `src.retraining`, `src.api`, `src.mlflow_utils`, `src.preprocessing`, `src.validation`, `src.evaluation`) presumably needs its own equivalent `__init__.py`, though only this top-level one was part of this reading batch.

Important Notes: - Being empty means there is no convenience re-export (e.g., no `from src.logger import get_logger` shortcut at the package level) — every import elsewhere in the codebase must use the full submodule path (`from src.logger import get_logger`, not `from src import get_logger`), which is consistent with what was observed in `main.py`'s import block. - If this file were ever deleted, `src` could still work as an implicit "namespace package" under modern Python (3.3+), but `setup.py`'s `find_packages` (as opposed to `find_namespace_packages`) would very likely fail to discover it, breaking the packaging step — this is a subtle but real dependency between this empty file and `setup.py`.

Group 2: Data Loading & Validation

These five files form the first stage of the pipeline shown in ARCHITECTURE.md: `load (Kaggle M5 / CSV / synthetic) → validate (schema, quality, outliers → quality score)`. Nothing downstream (preprocessing, feature engineering, training) can be trusted unless the data first lands in the canonical long format (`date, series_id, item_id, store_id, sales, sell_price, promotion_flag`) and then passes a validation gate that scores its quality. Loading and validation are grouped together because they answer two halves of the same question: "do we have data, and can we trust it?"

Read them in this order: 1. `src/data_loading/downloader.py` — gets raw data onto disk (Kaggle or synthetic). 2. `src/data_loading/loader.py` — turns whatever is on disk into the canonical long-format DataFrame. 3. `src/data_loading/__init__.py` — package marker only. 4. `src/validation/validator.py` — checks the canonical DataFrame's schema/quality/outliers and scores it. 5. `src/validation/__init__.py` — package marker only.

FILE: SRC/DATA_LOADING/INIT.PY

Type: Package marker (empty) **Purpose:** Makes `src/data_loading` an importable Python package so `from src.data_loading import downloader` works elsewhere in the codebase. **Why Created (Reasoning):** Python packages need an `__init__.py` (or namespace-package semantics) for reliable `import` resolution across the project; an explicit empty file is the simplest, most predictable choice. **Dependencies:** None. **Key Concepts:** Empty file; no exports, no `__all__`. **Detailed Explanation:** The file is completely empty. It carries no logic, re-exports nothing, and defines no `__all__`. Its only job is to exist so that `src.data_loading` is a regular package. `src/data_loading/loader.py` imports the sibling module via `from src.data_loading import downloader`, which relies on this file marking the directory as a package. **Key Code Sections Explained:** N/A — no code in the file. **How To Use/Call This File:** Never imported or referenced directly; it's implicit infrastructure. Other files import submodules like `src.data_loading.downloader` or `src.data_loading.loader`. **Important Notes:** Being empty is deliberate and correct here — there is no shared state or convenience re-export that the two submodules need, so adding content would be unrequested abstraction.

FILE: SRC/DATA_LOADING/DOWNLOADER.PY

Type: Core Logic (data acquisition) **Purpose:** Downloads the real M5 competition dataset from Kaggle via `kagglehub`, or generates a synthetic long-format demand dataset when Kaggle access isn't available. **Why Created (Reasoning):** The platform must be runnable with zero external credentials (per README/ARCHITECTURE "Synthetic fallback" design decision) while still supporting the real M5 dataset for a realistic demo. This file isolates all "how do we get raw data onto disk" concerns from "how do we shape it" (that's `loader.py`'s job). **Dependencies:** `pathlib.Path`, `numpy`, `pandas`, `kagglehub` (imported lazily inside `download_m5`), `src.constants.RAW_DIR`, `src.logger.get_logger`. **Key Concepts:** `SYNTHETIC_PATH` constant (`RAW_DIR / "synthetic_sales.csv"`), `download_m5()`, `generate_synthetic()`, lazy import of `kagglehub`, `numpy.random.default_rng`

for reproducible synthetic series, `if __name__ == "__main__"` fallback pattern. **Detailed Explanation:** `download_m5()` lazily imports `kagglehub` (only when called, so the dependency isn't required unless Kaggle is actually used) and calls `kagglehub.competition_download("m5-forecasting-accuracy")`, returning the local path where the M5 files (`sales_train_validation.csv`, `calendar.csv`, `sell_prices.csv`) land. This requires `~/.kaggle/kaggle.json` credentials and accepted competition rules; if either is missing it raises rather than silently failing.

`generate_synthetic(n_series, n_days, seed)` builds a fully synthetic long-format dataset with a seeded `numpy` Generator so runs are reproducible. For each series it composes: a random base level (`rng.uniform(50, 200)`), a linear trend (`np.linspace`), weekly seasonality (`15 * sin(2π·day/7 + phase)`), yearly seasonality (`25 * sin(2π·day/365.25 + phase)`), a Bernoulli promotion flag (5% of days via `rng.random(n_days) < 0.05`), a promo sales bump (`+40%` of base on promo days), and Gaussian noise scaled to `8%` of the base level. The result is clipped to be non-negative (`np.clip(..., 0, None)`), rounded to whole units for `sales`, and `sell_price` is discounted 20% on promo days. Each series gets a synthetic `series_id` like `ITEM_01_STORE_1` (item cycles 1..n, store cycles through 3 stores via `i % 3 + 1`). All series are concatenated and written to `SYNTHETIC_PATH` as CSV.

The `if __name__ == "__main__"` block demonstrates the fallback contract used throughout the pipeline: try `download_m5()`, and on any exception (missing credentials, network error, rules not accepted) log a warning and call `generate_synthetic()` instead. This exact try/except pattern is what `loader.load_data()` reuses at the pipeline level. **Key Code Sections Explained:** - `download_m5() -> Path`: Thin wrapper around `kagglehub.competition_download`; the `import kagglehub` is inside the function body (not at module top) specifically so importing `downloader.py` never fails/requires `kagglehub` installed unless this path is actually exercised. - `generate_synthetic(n_series=10, n_days=1095, seed=42) -> Path`: The synthetic data generator; produces ~3 years of daily data per series with trend + two seasonal components + promo effects + noise, matching the canonical schema columns directly (`date, series_id, item_id, store_id, sales, sell_price, promotion_flag`) so no further reshaping is needed downstream. - `SYNTHETIC_PATH = RAW_DIR / "synthetic_sales.csv"`: Shared constant so `loader.py` can check `downloader.SYNTHETIC_PATH.exists()` before regenerating, avoiding redundant regeneration on repeated pipeline runs. **How To Use/Call This File:** `src/data_loading/loader.py`'s `load_data()` calls `downloader.download_m5()` inside a try/except and, on failure, checks `downloader.SYNTHETIC_PATH.exists()` before calling `downloader.generate_synthetic(n_series=..., seed=...)`. Can also be run standalone: `python -m src.data_loading.downloader`. **Important Notes:** Kaggle fallback is a first-class design decision (see ARCHITECTURE.md "Synthetic fallback" row), not an afterthought — the except clause deliberately catches any `Exception`, since kagglehub failures can be network errors, auth errors, or missing-rules errors, and the response is the same in all cases. The synthetic data is only regenerated if the file doesn't already exist (checked by the caller, not this file) — re-running the pipeline reuses the same synthetic CSV. Because `seed=42` is the default, repeated calls with the same `n_series` produce identical data, which matters for reproducible demos/tests. The 3x IQR-style generous promo/seasonality parameters are hand-tuned to "look realistic" rather than statistically derived from real data.

FILE: SRC/DATA_LOADING/LOADER.PY

Type: Core Logic (data shaping / ingestion entrypoint) **Purpose:** Converts whatever raw data is available (M5 Kaggle files, a user's generic CSV, or the synthetic CSV) into the single canonical long-format DataFrame that every downstream stage consumes. **Why Created (Reasoning):** ARCHITECTURE.md states "Everything downstream of the loader consumes one long format... New data sources only need a loader that emits this shape." This file is the seam that lets the pipeline support three very different data sources (Kaggle wide-format M5, arbitrary user CSVs, synthetic data) while everything after it — validation, preprocessing, features, training — only ever needs to know one schema. **Dependencies:** `pathlib.Path`, `pandas`, `src.constants` (`DATE_COL`, `SERIES_COL`, `TARGET_COL`), `src.data_loading.downloader`, `src.logger.get_logger`. **Key Concepts:** `load_m5()`, `load_csv()`, `load_data()` (the public entrypoint), wide-to-long `melt`, M5's `snap_CA/TX/WI` columns repurposed as a promotion proxy, config-driven source selection (`config["data"]["source"]`), fallback chaining (kaggle → synthetic). **Detailed Explanation:** `load_m5(m5_dir, n_series)` reads M5's wide `sales_train_validation.csv` (one row per item/store, one column per day `d_1..d_1941`) and `calendar.csv` (maps `d_*` codes to real dates and SNAP-benefit flags per state). It computes each series' total volume (`sales[day_cols].sum(axis=1)`) and keeps only the top-N by volume (`totals.nlargest(n_series)`) — because per ARCHITECTURE.md the full M5 melt is ~58M rows, and top-N keeps the demo runnable in under 15 minutes. It then melts the wide table to long format (one row per item/store/day), merges in real dates via the `d` code, and optionally left-merges `sell_prices.csv` on `(store_id, item_id, wm_yr_wk)`. It builds `series_id` as

`item_id + "_" + store_id`, and cleverly reuses M5's SNAP (Supplemental Nutrition Assistance Program) benefit-day flags as a stand-in `promotion_flag` — matching each row's state prefix (`CA`, `TX`, `WI`, parsed from `store_id[:2]`) to the corresponding `snap_CA/TX/WI` column. Finally it selects only canonical columns and sorts by `(series_id, date)`.

`load_csv(path)` handles the generic/user-supplied and synthetic-data path: it parses `date` as datetime on read, requires `date` and `sales` columns to exist (raises `ValueError` otherwise), and if `series_id` is missing it synthesizes one by concatenating whichever of `item_id`, `product_id`, `store_id` are present (joined with `_`), or falls back to the literal string `"series_0"` if none exist. It then sorts by `(series_id, date)` just like `load_m5`.

`load_data(config)` is the single public entrypoint the rest of the pipeline calls. It reads `config["data"]["source"]` (`"csv"` or `"kaggle"`, default `"kaggle"`). If source is `"csv"` and a `csv_path` is configured, it loads that file directly via `load_csv`. If source is `"kaggle"`, it tries `downloader.download_m5()` then `load_m5()`; on any exception it logs a warning and falls through to synthetic. In the synthetic fallback branch, it only calls `downloader.generate_synthetic()` if `downloader.SYNTHETIC_PATH` doesn't already exist (avoiding regenerating data on every run), then loads that CSV via `load_csv`. This gives a three-tier fallback: explicit CSV path → Kaggle → synthetic, matching the Quick Start / Troubleshooting sections of README.md. **Key Code Sections Explained:** - `load_m5(m5_dir: Path, n_series: int = 10) -> pd.DataFrame`: The most complex function — wide-to-long melt, top-N volume filtering, calendar join for real dates, sell-price join, and the SNAP-as-promotion trick. Important because it's the only place that understands M5's specific file layout; everything after it is source-agnostic. - `load_csv(path: Path) -> pd.DataFrame`: The generic-CSV loader; enforces the two hard-required columns (`date`, `sales`) and synthesizes `series_id` when absent. This is also what loads synthetic data, since the synthetic CSV already matches this shape. - `load_data(config: dict) -> pd.DataFrame`: The only function other modules should call. Encodes the full fallback chain (csv config → kaggle → synthetic) and is what `main.py` invokes to kick off the pipeline. **How To Use/Call This File:** Called from the pipeline entrypoint (`main.py`) as `df = load_data(config)`, where `config` is the parsed `config.yaml`. The resulting `df` is what gets passed into `src.validation.validator.run_validation(df)` next, and later into preprocessing/features. To point the pipeline at your own data, README.md shows setting `data.source: 'csv'` and `data.csv_path` in `config.yaml`. **Important Notes:** `load_m5`'s promotion-flag substitution (SNAP benefit days, not actual promotions) is a documented approximation — M5 doesn't ship a direct promo flag, so SNAP eligibility is used as a proxy signal that correlates with purchasing behavior. `load_csv`'s `series_id` synthesis order (`item_id`, `product_id`, `store_id`) means if only `store_id` is present, every row across all items at a store would collapse to the same `series_id` unless other id columns are added — a caller with multi-item CSVs must include an item-level id column. The `sell_price` merge in `load_m5` is a `left` join and only happens if `sell_prices.csv` exists, so `sell_price` may end up `NaN` for missing price rows (this is exactly what `validator.quality_metrics`'s `missing_pct` will surface). `load_data`'s synthetic branch reuses an existing `SYNTHETIC_PATH` file across runs rather than regenerating — so changing `n_series` in config after synthetic data already exists on disk has no effect until the file is deleted.

FILE: SRC/VALIDATION/INIT.PY

Type: Package marker (empty) **Purpose:** Makes `src/validation` an importable Python package. **Why Created (Reasoning):** Same rationale as `src/data_loading/__init__.py` — required for `from src.validation import validator`-style imports to resolve. **Dependencies:** None. **Key Concepts:** Empty file; no exports. **Detailed Explanation:** Empty, identical in role to the data_loading package marker. No shared validation state or re-exports live here. **Key Code Sections Explained:** N/A — no code in the file. **How To Use/Call This File:** Not imported directly; implicit package infrastructure for `src.validation.validator`. **Important Notes:** None beyond noting it's intentionally empty.

FILE: SRC/VALIDATION/VALIDATOR.PY

Type: Core Logic (data quality gate) **Purpose:** Validates a canonical long-format DataFrame's schema, computes data-quality metrics (missingness, duplicates, negatives, date gaps), flags outliers, rolls everything into a single 0–100 quality score, and writes a JSON validation report. **Why Created (Reasoning):** Per ARCHITECTURE.md's pipeline flow, `validate` is a distinct stage between `load` and `clean` — nothing should be fed into preprocessing/feature engineering without first checking it matches the expected shape and isn't pathologically dirty. It also produces `reports/validation_report.json`, one of the dashboard's contracted artifacts (per ARCHITECTURE.md's "Artifacts contract" table), so this file is the sole producer the dashboard depends on for validation info. **Dependencies:** `numpy`, `pandas`, `src.constants` (`DATE_COL`, `SERIES_COL`, `TARGET_COL`,

`VALIDATION_REPORT_PATH`), `src.logger.get_logger`, `src.utils.serialization.save_json`. **Key Concepts:** `REQUIRED_COLUMNS` schema dict, `validate_schema()`, `quality_metrics()`, `detect_outliers()` (IQR or z-score, per-series), `run_validation()` orchestrator, weighted quality-score penalty formula, JSON report via `save_json`. **Detailed Explanation:** `REQUIRED_COLUMNS = {DATE_COL: "datetime", SERIES_COL: "object", TARGET_COL: "numeric"}` declares the three columns every downstream consumer needs and their expected pandas dtype category. `validate_schema(df)` walks this dict and returns a list of human-readable error strings: a missing column, a date column that isn't actually datetime-typed, or a sales column that isn't numeric. An empty list means "schema valid."

`quality_metrics(df)` computes: `missing_pct` — per-column percentage of NaNs (`df.isna().mean() * 100`, rounded to 2 decimals) as a dict; `duplicate_rows` — count of rows sharing the same `(date, series_id)` pair (`df.duplicated(subset=[...])`); `negative_sales` — count of rows where `sales < 0` (guarded by `if TARGET_COL in df else 0`); and `date_gaps` — summed across every series group, `(max_date - min_date).days + 1` (the expected number of daily observations) minus `d.nunique()` (actual distinct dates present), i.e. how many calendar days are missing from each series' date range.

`detect_outliers(df, method="iqr", z_thresh=4.0)` returns a boolean `pd.Series` mask aligned to `df.index`, computed independently per `series_id` group (so one series' scale doesn't distort another's outlier bounds). The default IQR method flags values outside `[Q1 - 3*IQR, Q3 + 3*IQR]` — note the multiplier is 3x, not the conventional 1.5x, because (per the docstring) demand spikes are often genuine signal, not noise, so the bounds are deliberately widened to avoid over-flagging real promotional spikes. The alternate z-score method flags `|z| > z_thresh` (default 4.0), with `+ 1e-9` added to the standard deviation to avoid division by zero for constant series.

`run_validation(df)` is the orchestrator and the only function called from outside the module. It runs all three checks, then computes a `quality_score` starting at 100.0 and subtracting capped weighted penalties: missing target values (`target_missing * 3`, capped at 30), duplicate rows as a fraction of total rows (`dupes/len(df) * 100 * 2`, capped at 20), date gaps as a fraction of total rows (same formula, capped at 20), outlier rate (`outliers.mean() * 100`, capped at 10), and a flat 30-point penalty if any schema errors exist. The score is floored at 0. It assembles a report dict (row/series counts, date range, schema errors, all quality metrics, outlier count/percentage, quality score, and a `valid` boolean equal to `not schema_errors`), writes it to `VALIDATION_REPORT_PATH` via `save_json`, logs success or failure, and returns the dict. **Key Code Sections Explained:** - `validate_schema(df) -> list[str]`: Hard gate on structural correctness (column presence + dtype). Important because it's what makes `report["valid"]` `False` and triggers the flat 30-point score penalty — schema failures are treated as categorically worse than quality issues. - `quality_metrics(df) -> dict`: The "how dirty is this data" measurements feeding the score; `date_gaps`'s per-series expected-vs-actual day-count diff is the one non-obvious metric — it assumes daily frequency and would misreport gaps for weekly/monthly data (a caveat worth knowing before trusting it on non-daily series). - `detect_outliers(df, method="iqr", z_thresh=4.0) -> pd.Series`: Per-series (not global) outlier detection with an intentionally loosened IQR multiplier (3x vs standard 1.5x) — a deliberate domain-specific tuning choice for demand data, not a generic-stats default. - `run_validation(df) -> dict`: The public entrypoint; owns the quality-score formula (capped weighted penalties across four dimensions) and is the single writer of `reports/validation_report.json`, a contracted dashboard artifact. **How To Use/Call This File:** Called from `main.py` right after `load_data()`, as `report = run_validation(df)` (or similar), before the pipeline proceeds to preprocessing. The dashboard (FastAPI server) reads `reports/validation_report.json` directly as one of its read-only artifacts rather than calling into this module. **Important Notes:** All three penalty terms except the schema flat penalty are expressed as `(count / len(df)) * 100 * 2`, meaning they're proportional to how large a fraction of the dataset is affected, scaled up 2x, then capped — so on very large datasets a small number of absolute duplicates/gaps barely moves the score, while on tiny datasets a single duplicate can meaningfully hurt it. `quality_score` can theoretically double-penalize the same rows (e.g., a row with a missing target could also be flagged an outlier if it were dropped/imputed to 0), but the function doesn't try to de-duplicate overlapping penalty sources — it's a simple additive heuristic, not a statistically rigorous score. `detect_outliers` groups by `series_id`, so a series with very few points can produce unstable quartile/z-score estimates. Schema validation only checks presence and coarse dtype (datetime/numeric/object) — it does not validate value ranges, referential integrity between `item_id` / `store_id` / `series_id`, or that `sell_price` / `promotion_flag` exist at all (they're optional in the canonical format per `REQUIRED_COLUMNS`, which covers only `date`, `series_id`, `sales`).

Group 3: Preprocessing & Feature Engineering

These four files form the middle of the pipeline (`load → validate → clean → features → train` , per ARCHITECTURE.md): `src/preprocessing/preprocessor.py` turns a raw long-format frame into a gap-free, imputed series per `series_id` , and `src/features/feature_engineering.py` turns that clean frame into the ~40-column model-input matrix, with every target-derived feature computed leakage-safely (`shift(1)` before any rolling/expanding aggregate). The two `__init__.py` files are empty package markers with no logic. Read in this order: `src/preprocessing/__init__.py` and `src/features/__init__.py` (trivial, skip quickly) → `src/preprocessing/preprocessor.py` (produces the clean frame features.py consumes) → `src/features/feature_engineering.py` (the core of this group) → `tests/test_features.py` (proof that the leakage-safety claim actually holds).

FILE: SRC/PREPROCESSING/INIT.PY AND SRC/FEATURES/INIT.PY

Type: Package marker (empty) **Purpose:** Make `src/preprocessing` and `src/features` importable as Python packages (`from src.preprocessing.preprocessor import clean` , `from src.features.feature_engineering import build_features`). **Why Created (Reasoning):** Standard Python package requirement (or convention pre-PEP 420) so the rest of the codebase can use dotted-module imports consistent with the other `src/*` packages (`data_loading` , `validation` , `models` , etc.). **Dependencies:** None. **Key Concepts:** Empty file, no exports, no re-exporting of submodule symbols — callers import directly from the submodule (e.g., `src.preprocessing.preprocessor` , `src.features.feature_engineering`), not from the package root. **Detailed Explanation:** Both files are empty. They exist solely so `src.preprocessing` and `src.features` are recognized as packages. There is no `__all__` , no convenience re-export, and no package-level initialization logic — every other file in the codebase that needs `clean` , `build_features` , or `feature_columns` imports the submodule path explicitly. **Key Code Sections Explained:** N/A — no code. **How To Use/Call This File:** Never referenced directly; their only effect is enabling `import src.preprocessing.preprocessor` / `import src.features.feature_engineering` to resolve. **Important Notes:** Because they're empty, there's no single "package API surface" to check — read the submodules themselves to find the real functions.

FILE: SRC/PREPROCESSING/PREPROCESSOR.PY

Type: Core Logic (data cleaning pipeline stage) **Purpose:** Convert the raw long-format frame (`date, series_id, item_id, store_id, sales, sell_price, promotion_flag`) into a gap-free, imputed, per-series frame at a consistent frequency, then persist it to `data/processed/clean.parquet` . **Why Created (Reasoning):** Real (and synthetic/M5) retail data has duplicate rows, irregular date gaps, and missing values; downstream feature engineering assumes a complete, regular date index per series (lags/rolling windows need contiguous positions to be meaningful). This module is the single place that guarantees that invariant before features are built. **Dependencies:** `pandas` ; `src.constants` (`CLEAN_DATA_PATH` , `DATE_COL` , `SERIES_COL` , `TARGET_COL`); `src.logger.get_logger` . **Key Concepts:** `detect_frequency()` (modal date-diff inference), `clean()` (drop-dedupe → per-series reindex to `pd.date_range` → group-specific imputation → clip negatives → concat → write parquet), frequency map `{1:"D", 7:"W", 30/31:"MS"}` . **Detailed Explanation:** `detect_frequency` looks at the first series in the frame, sorts its dates, takes the modal (`.mode()`) day-to-day difference, and maps that to a pandas offset alias (daily/weekly/monthly), defaulting to `f"{freq_days}D"` for anything else. `clean` first drops exact duplicate `(series_id, date)` rows, then determines the frequency either via `detect_frequency` (when `config["data"]["granularity"] == "auto"` , the default) or takes it verbatim from config. It then iterates `df.groupby(SERIES_COL)` , and for each series: sets `date` as the index, builds a complete `pd.date_range` spanning that series' own min/max date, and reindexes onto it — this is what introduces the gap rows as NaN. Static identifier columns (`item_id` , `store_id` , if present) are forward/backward filled to plug the gaps. The target (`sales`) is filled with 0 and clipped at 0 (rationale stated in the docstring: "a missing retail day is a no-sale day"). `sell_price` is ffill/bfill'd (last known price persists through a gap), and `promotion_flag` gaps become 0 (no promo). All per-series frames are concatenated back together, the detected/used frequency is stashed in `cleaned.attrs["frequency"]` , and the result is written to `CLEAN_DATA_PATH` as parquet. This module is the "clean" stage in ARCHITECTURE.md's pipeline diagram, sitting directly after validation and before feature engineering — it's the producer of `data/processed/clean.parquet` , one of the contract artifacts the dashboard/other stages read. **Key Code Sections Explained:** - `detect_frequency(df)` : Infers cadence from real data rather than requiring the caller to specify it, so the same code works for daily, weekly, or monthly M5/CSV/synthetic inputs. Uses only the first series (`df[SERIES_COL].iloc[0]`) as a representative sample — assumes all series in the frame share one frequency. - `clean(df, config)` : The single entry point for this module. The `for sid, g in df.groupby(SERIES_COL)` loop is the core: reindexing per series (not globally) is important because different series can have different date ranges (different min/max), so a single global date range would incorrectly extend/truncate individual series. - Imputation branch inside the loop: distinguishes imputation strategy by *column semantics* — sales gaps are business-meaningful zeros, whereas price/promo gaps are "carry the last known state forward" — this is a deliberate, not generic, choice per the inline comment. - `cleaned.attrs["frequency"] =`

`freq`: propagates the detected frequency as frame metadata so downstream code (e.g., forecasting horizon logic) can know the cadence without re-detecting it. **How To Use/Call This File:** Called from the main pipeline (`main.py`) as the "clean" stage, typically `clean(raw_df, config)` where `config` is the loaded `config.yaml` dict (needs `config["data"]["granularity"]`). Returns the cleaned DataFrame and also writes it to `CLEAN_DATA_PATH`; `src/features/feature_engineering.build_features` is normally called next on this returned/loaded frame. **Important Notes:** `detect_frequency` only samples the *first* series — if series have heterogeneous frequencies, this silently applies one frequency to all. Frequency day-count-to-alias mapping is a coarse lookup (`{1,7,30,31}`) with a generic `f"{freq_days}D"` fallback for anything else (e.g., won't correctly detect true calendar-month frequency if day counts vary 28-31 across months, since only the modal diff is used). `clip(lower=0)` on sales means legitimate negative-adjustment rows (returns) would be zeroed out, not preserved — implicit assumption that raw sales are never negative-as-signal.

FILE: SRC/FEATURES/FEATURE_ENGINEERING.PY

Type: Core Logic (feature engineering pipeline stage) **Purpose:** Build the ~40-column, leakage-safe feature matrix (temporal, lag, rolling, decomposition, business, hierarchical groups) from the cleaned long-format frame, and persist it to `data/processed/features.parquet`. **Why Created (Reasoning):** All 7 downstream models train on one shared feature matrix (per ARCHITECTURE.md's "one global model across series" decision); this module is the single place responsible for generating those features correctly and, critically, without leaking future information into any row — a requirement stated explicitly in both README.md ("Leakage-safe features") and ARCHITECTURE.md ("A unit test asserts it"). **Dependencies:** `numpy`, `pandas`; `src.constants` (`DATE_COL`, `FEATURES_PATH`, `SERIES_COL`, `TARGET_COL`); `src.logger.get_logger`; optionally the third-party `holidays` package (imported lazily inside `add_temporal`, guarded by try/except). **Key Concepts:** Six feature-group functions (`add_temporal`, `add_lags`, `add_rolling`, `add_decomposition`, `add_business`, `add_hierarchical`) each independently toggleable via `config["features"]`; the `shift(1)`-before-aggregate pattern used consistently for every target-derived feature; `build_features()` as the orchestrator that also drops warm-up rows and fills remaining NaNs; `feature_columns()` as the canonical "what goes into the model" column selector. **Detailed Explanation:** `build_features` is the entry point. It sorts by `(series_id, date)` (essential — all `shift`/`rolling`/`expanding` calls assume chronological order within each series), then conditionally runs each feature-group function based on `config["features"]` flags (`include_temporal`, `include_decomposition`, `include_business_features`, `include_hierarchical`; lags and rolling always run, parameterized by `lag_lookbacks` and `rolling_windows`). After all groups run, it computes `max_lag = max(lag_lookbacks)` and drops every row where that longest lag's column is still NaN (`df.dropna(subset=[f"lag_{max_lag}"])`) — these are the "warm-up" rows at the start of each series where there isn't enough history yet. Any remaining NaNs in feature columns (e.g., partial rolling windows, early decomposition/hierarchical values) are filled with 0 via `feature_columns(df)`. The result is optionally written to `FEATURES_PATH`. The feature groups break down as: **temporal** (`add_temporal`) — calendar fields (day_of_week, month, quarter, day_of_month, week_of_year, is_weekend) plus cyclical sin/cos encodings of day-of-week and month (so e.g. December and January are "close" to the model), plus holiday flags/day-count via the optional `holidays` library, falling back to all-zero holiday columns if the import or country lookup fails. **Lag** (`add_lags`) — raw `TARGET_COL` shifted by each value in `lookbacks` (e.g., `lag_1`, `lag_7`), computed per-series via `groupby(SERIES_COL)[TARGET_COL].shift(k)`. **Rolling** (`add_rolling`) — mean/std/min/max over each window, plus a median over a 14-day window if 14 is in the configured windows. **Decomposition** (`add_decomposition`) — trend (30-day past-only rolling mean), seasonal (past-only expanding mean per series+day-of-week, minus trend), and residual (shifted value minus trend minus seasonal) — a lightweight, causal proxy for classical trend/seasonal/residual decomposition. **Business** (`add_business`) — price normalized against each series' own mean price, and period-over-period price percent change; ensures a `promotion_flag` column exists (defaulting to 0) if the source data lacks one. **Hierarchical** (`add_hierarchical`) — categorical encodings of `store_id`/`item_id` (via pandas `.cat.codes`), plus past-only expanding-mean average sales per store and per product. This is the "features" stage in ARCHITECTURE.md's pipeline, consuming `clean.parquet` (from `preprocessor.py`) and producing `features.parquet`, which every model in `src/models/` and the training orchestrator in `src/training/` treats as the ground-truth input matrix. **Key Code Sections Explained:** - `add_lags(df, lookbacks)`: `df.groupby(SERIES_COL)[TARGET_COL].shift(k)` — this is inherently leakage-safe by construction: `shift(k)` for k≥1 always reads a value from k rows earlier *within that series' group*, never the current or a future row. `groupby` before `shift` is what keeps series from bleeding into each other's lag history. - `add_rolling(df, windows)`: The comment "shift(1) first: window covers [t-w, t-1], never the current value" states the mechanism directly. `shifted = df.groupby(SERIES_COL)[TARGET_COL].shift(1)` is computed *once* and reused for every window — the raw target series has already had "today" removed before any `.rolling(w)` call runs over it, so the widest a window can reach is `t-1` back to `t-w`; it can never include the value being predicted for. `min_periods=max(2, w // 2)` allows

partial windows early in a series (rather than requiring a completely full window), trading a bit of noise for less data loss. - `add_decomposition(df)` : Reuses the same `shift(1)` -first pattern for `trend_component` (30-day past-only rolling mean). The `seasonal_component` is computed by grouping on a composite key (`series_id + "_" + day_of_week`) and taking a past-only `.expanding().mean()` on the *already-shifted* series, then subtracting trend — so seasonal signal is "the historical past-only average sales on this weekday for this series, adjusted for trend," never touching the current row's actual value. - `add_hierarchical(df)` : `store_avg_sales` / `product_avg_sales` again use `shift(1)` before `.expanding().mean()` grouped by `store_id` / `item_id` — average sales for a store/product as of *before* the current row, avoiding the classic hierarchical-feature leak where a global/group average computed over the whole dataset (including the current and future rows) implicitly encodes the answer. - `build_features(df, config, save)` : Orchestrates group order and config toggles, then the `dropna(subset=[f"lag_{max_lag}"])` step is the leakage-safety complement to the shift pattern — it physically removes rows where the longest-lag feature couldn't be computed (start-of-series warm-up), rather than filling them with a fabricated non-past value. - `feature_columns(df)` : Defines the model-input contract by exclusion (`DATE_COL, SERIES_COL, TARGET_COL, item_id, store_id, sell_price` are excluded) — notably it excludes the *raw* target and raw price, ensuring the model only ever sees derived (lagged/rolled/shifted) versions of those, not the current-row raw values, which is a second, structural line of defense against leakage beyond the per-feature `shift(1)` calls. **How To Use/Call This File:** Called from `main.py` as the "features" pipeline stage: `features_df = build_features(clean_df, config)` . `feature_columns(df)` is used both internally (to know which columns to fillna(0)) and externally by training/model code to select the exact model-input columns from `features.parquet` . Config keys consumed: `features.include_temporal` , `features.lag_lookbacks` , `features.rolling_windows` , `features.include_decomposition` , `features.include_business_features` , `features.include_hierarchical` , `features.holiday_country` . **Important Notes:** The `holidays` import is optional/lazy — if the package isn't installed or the country lookup throws, `is_holiday` / `days_since_last_holiday` silently become all-zero columns rather than failing the pipeline (logged as a warning). `add_business` 's `price_norm` divides by each series' *entire-history* mean price (`transform("mean")` , not shifted) — this uses the whole series' price distribution (past and future) as a normalization constant, which is a mild deviation from strict past-only computation, though it doesn't touch the target and is a normalization denominator rather than a predictive signal per se. The warm-up drop is keyed only on the single longest lag column (`lag_{max_lag}`); if other features (e.g., the 30-day trend rolling window) require more history than the longest configured lag, their NaNs are just zero-filled rather than row-dropped, which is a deliberate but implicit choice enforced by the final `fillna(0)` over all feature columns.

Leakage-safety test coverage (tests/test_features.py): The test file builds a small synthetic two-series frame (`series_id` "A"/"B", `sales` = 0..99 for each) and calls `build_features` directly with `save=False` . Three tests assert the leakage-safety property concretely rather than just trusting the code: - `test_lag_is_exactly_previous_value` : asserts `lag_1` at row t equals `sales` at row t-1, i.e. `a["lag_1"].to_numpy()[1:] == a["sales"].to_numpy()[:-1]` — proves the lag feature is exactly one step in the past, not the current value. - `test_rolling_mean_excludes_current_row` : with `sales` = 0..99, checks that `rolling_mean_7` at the *last* row (t=99, sales=99) equals 95 — the mean of sales 92..98 (the 7 values strictly before 99), not sales 93..99 which would include the current value 99. This is the direct numeric proof that `add_rolling` 's `shift(1)` -before-`rolling` truly excludes the current row from its own aggregate. - `test_no_nans_in_feature_matrix` : asserts no NaNs remain anywhere in `feature_columns(feats)` after `build_features` runs — proving the warm-up drop + fillna(0) steps leave a fully model-ready matrix with no leaked "future-looking" imputation needed to patch it.

Together these tests validate the two things README/ARCHITECTURE claim: (1) target-derived features are computed from strictly-past values (proven with an exact arithmetic check on both a raw lag and a windowed aggregate), and (2) the final matrix handed to models is NaN-free.

Group 4: Model Implementations

INTRO: OVERALL ARCHITECTURE

This group implements DemandIQ's seven forecasting models behind one uniform interface, using a classic **abstract base class + concrete implementations + factory** design:

- **`base_model.py`** defines `BaseModel` , an `ABC` with two abstract methods (`fit` , `predict`) and one optional hook (`feature_importance`). This is the **Strategy pattern**: the training/evaluation/forecasting loop elsewhere in the codebase

(trainer, evaluator, forecast generator) is written once against `BaseModel` and never needs to know whether it is driving a moving average, a gradient-boosted tree, an ARIMA process, or an LSTM.

- **Concrete families** (`baseline_models.py` , `statistical_models.py` , `tree_models.py` , `lstm_model.py`) each implement `BaseModel` for a cluster of related algorithms that share fitting/prediction mechanics (feature-based sklearn-style regressors vs. per-series statistical models vs. a global sequence model).
- `model_factory.py` is the **Factory pattern**: it reads the `models` section of the run config and instantiates only the enabled models, isolating all the "which models are turned on, with what hyperparameters" logic in one place, and deferring the `torch` import for LSTM so the rest of the platform runs without that dependency installed.

This design was chosen because the seven models are fundamentally different beasts — some feature-based and stateless-per-call (LinearRegression, LightGBM/XGBoost/RandomForest), some purely descriptive with no learning (MovingAverage), some per-series statistical fits (ARIMA, ExpSmoothing), and one a global neural sequence model requiring scaling and windowing (LSTM). Without a common interface, the training/evaluation pipeline would need per-model branching everywhere. With `BaseModel.fit` / `predict` , the rest of the system (training loop, backtesting/evaluation, comparison report, live forecasting) is written once and stays model-agnostic.

Recommended reading order: `base_model.py` (the contract) → `baseline_models.py` → `statistical_models.py` → `tree_models.py` → `lstm_model.py` (the concrete families, roughly simplest to most complex) → `model_factory.py` last (ties everything together via config).

Note: `src/models/__init__.py` exists but is **empty** — it has no content and is not documented as its own subsection below beyond this note; it simply marks `src/models` as a package with no re-exports.

FILE: SRC/MODELS/BASE_MODEL.PY

Type: Abstract Base Class (interface contract) **Purpose:** Defines the common `fit` / `predict` / `feature_importance` interface that every one of the seven forecasting models must implement. **Why Created (Reasoning):** The training loop, evaluator, and forecast generator need to treat wildly different model types (statistical, tree-based, neural, naive baselines) identically. Without this shared contract, every caller would need type-specific branches (`if isinstance(model, ARIMAModel): ... elif ...`). Centralizing the contract here means new models can be added later by just implementing this interface, with zero changes to the training/evaluation code. **Dependencies:** `abc` (stdlib: `ABC` , `abstractmethod`), `numpy` , `pandas` . No intra-project dependencies — it is a leaf module that everything else in `src/models` depends on. **Key Concepts:** Strategy pattern via an abstract interface. Class attribute `name: str = "base"` (overridden per subclass, used for logging/reporting/identifying models in comparison output). Abstract methods `fit(train, feature_cols)` and `predict(future, feature_cols, history=None)` . Optional hook `feature_importance()` defaulting to `None` . **Detailed Explanation:** This file is intentionally tiny — it is a pure contract, not logic. `fit` takes a long-format training DataFrame plus the list of feature column names to use, and is expected to mutate internal state (fitted estimator, per-series parameters, network weights, etc.) with no return value. `predict` takes a `future` DataFrame (the rows to forecast, time-ordered per series), the same `feature_cols` , and an optional `history` DataFrame containing all data strictly before `future` . The docstring explicitly notes `history` is required by sequence/statistical models (ARIMA, ExpSmoothing, LSTM need the raw target history to seed their forecast) but ignored by feature-based models (LinearRegression, tree models) since their required lookback is already baked into precomputed feature columns (e.g., rolling means) in `future` / `train` itself.

The `feature_importance()` method returns `None` by default, and is overridden by any model that can produce one (linear coefficients, tree feature importances). This lets an importance-reporting step call it uniformly across all models without checking type first — models that can't produce importances simply return `None` , and downstream code treats that as "nothing to report" for that model.

This class fits into the larger system as the single generic type that `model_factory.create_models()` returns a `list[BaseModel]` of, and that the training/evaluation pipeline (elsewhere in `src/` , likely a `trainer.py` or `evaluate.py`) iterates over polymorphically. **Key Code Sections Explained:** - `class BaseModel(ABC)` — the interface itself; inheriting from `ABC` means Python enforces that subclasses cannot be instantiated unless they implement both abstract methods, catching incomplete implementations at instantiation time rather than at first call. - `fit(self, train, feature_cols) -> None` (abstract) — every concrete model's training entry point; signature is deliberately generic (a DataFrame + column list) so it

accommodates both a single global fit (LSTM, tree models) and per-series loops done internally (ARIMA, ExpSmoothing). - `predict(self, future, feature_cols, history=None) -> np.ndarray` (abstract) — every concrete model's inference entry point; returns a flat `np.ndarray` aligned row-for-row with `future`, which lets the caller assign it directly as a prediction column without per-model reshaping. - `feature_importance(self) -> pd.Series | None` — optional hook, non-abstract so subclasses aren't forced to implement it. **How To Use/Call This File:** Never used directly (it's abstract). All five concrete-model files subclass it. `model_factory.py` imports it purely for the `list[BaseModel]` return type annotation. Any code (training loop, evaluator, forecaster) that needs to run a model calls `.fit(...)` then `.predict(...)` on whatever concrete instance the factory produced, without importing or checking concrete types. **Important Notes:** The `history` parameter's semantics ("required by some, ignored by others") is a slightly leaky abstraction — callers must always pass `history` (or risk a `ValueError` from LSTM, see below) even though most models ignore it. `feature_cols` is passed to `predict` even though several models (ARIMA, ExpSmoothing, LSTM, MovingAverage) never reference it — again, kept for interface uniformity.

FILE: SRC/MODELS/BASELINE_MODELS.PY

Type: Core Logic (concrete `BaseModel` implementations) **Purpose:** Implements two simple baseline forecasters: a parameter-free moving-average ensemble and a standard-scaled linear regression, both used as lower-bound sanity checks against the more sophisticated models. **Why Created (Reasoning):** Every forecasting project needs cheap, interpretable baselines to prove that expensive models (LightGBM, LSTM) are actually adding value over "just average the last N days" or "a linear model on the engineered features." These also serve as fast smoke-tests of the pipeline since `MovingAverageModel` has no training cost at all. **Dependencies:** `numpy`, `pandas`, `sklearn.linear_model.LinearRegression`, `sklearn.pipeline.make_pipeline`, `sklearn.preprocessing.StandardScaler`; internally imports `src.constants.TARGET_COL` and `src.models.base_model.BaseModel`. **Key Concepts:** Strategy pattern implementations (concrete leaves of `BaseModel`). `MovingAverageModel` is stateless (`fit` is a no-op) and reads precomputed `rolling_mean_{7,14,30}` feature columns straight from the `future` frame rather than computing anything itself. `LinearRegressionModel` wraps `StandardScaler + LinearRegression` in an sklearn `Pipeline`, and implements `feature_importance()` using absolute coefficient magnitude. **Detailed Explanation:** `MovingAverageModel.__init__` accepts a `windows` list (default `[7, 14, 30]`). Its `fit` does nothing — there are no parameters to learn since it just averages existing feature columns at predict time. Its `predict` selects whichever `rolling_mean_{w}` columns exist in `future` for the configured windows and returns their row-wise mean as the forecast. This relies entirely on the feature engineering stage upstream having produced leakage-safe rolling means (i.e., means computed only from data before each target date).

`LinearRegressionModel` is a genuinely fitted model: `fit` stores the `feature_cols` list it was trained on (so `predict` can reindex `future` with exactly the same columns/order) and fits a scikit-learn pipeline of `StandardScaler` followed by `LinearRegression` on `train[feature_cols]` against `train[TARGET_COL]`. `predict` simply calls `self.pipe.predict(future[self._cols])`. `feature_importance` pulls `.coef_` off the fitted `LinearRegression` step inside the pipeline, takes absolute value (since sign doesn't matter for "importance" ranking), and returns a sorted `pd.Series` indexed by feature name.

These two models sit at the "baseline" end of the model roster that `model_factory.create_models()` builds; both are enabled by default in config (`enabled: True` fallback) unlike the tree/statistical/LSTM models, which default to disabled or need explicit enabling. **Key Code Sections Explained:** - `MovingAverageModel.predict` — `future[cols].mean(axis=1).to_numpy()`: the entire "model" is a row-wise mean over whichever rolling-mean columns are present; if none of the configured windows exist as columns, `cols` is empty and `.mean(axis=1)` on an empty column selection would produce `NaN`/error-prone behavior — an implicit assumption that feature engineering always produces at least one matching rolling-mean column. - `LinearRegressionModel.__init__` — builds `make_pipeline(StandardScaler(), LinearRegression())` once; scaling matters for linear regression's coefficient interpretability and numerical conditioning, though not strictly required for its predictive accuracy. - `LinearRegressionModel.fit` — stores `self._cols = feature_cols` specifically so `predict` doesn't have to be passed the same list again (defensive against future/train having extra or reordered columns). - `LinearRegressionModel.feature_importance` — `pd.Series(np.abs(coefs), index=self._cols).sort_values(ascending=False)`: gives a ranked, human-readable feature-importance table analogous to what tree models expose, which lets a unified "feature importance" reporting step work across both linear and tree models. **How To Use/Call This File:** Instantiated by `model_factory.create_models()` when `models.moving_average.enabled` / `models.linear_regression.enabled` are true in config (both default to `True` if the config key is missing). Used through the

standard `BaseModel.fit` / `.predict` interface by the training/evaluation pipeline; no other file calls these classes directly. **Important Notes:** `MovingAverageModel` has no memory of the training set at all — it is purely a function of whatever rolling features already exist in the `future` frame, so its accuracy is entirely dependent on upstream feature engineering correctness. `LinearRegressionModel` will raise a `KeyError` at predict time if `future` is missing any column recorded in `self._cols` during `fit`. Neither model uses the `history` parameter of `predict`.

FILE: SRC/MODELS/STATISTICAL_MODELS.PY

Type: Core Logic (concrete `BaseModel` implementations, classical time-series statistics) **Purpose:** Implements two classical per-series time-series models — ARIMA (with automatic differencing-order selection) and Holt-Winters Exponential Smoothing — each fit independently per `series_id`. **Why Created (Reasoning):** Classical statistical forecasters (ARIMA/ETS family) are strong, well-understood baselines for time series that machine-learning models sometimes fail to match, especially on short or seasonal series with limited feature engineering. Including them lets the model comparison stage empirically decide whether "real" ML models beat traditional statistics on this dataset. **Dependencies:** `warnings` (stdlib), `numpy`, `pandas`, `statsmodels.tsa.arima.model.ARIMA`, `statsmodels.tsa.holtwinters.ExponentialSmoothing`, `statsmodels.tsa.stattools.adfuller`; internally imports `src.constants.{DATE_COL, SERIES_COL, TARGET_COL}`, `src.logger.get_logger`, `src.models.base_model.BaseModel`. **Key Concepts:** Strategy pattern implementations. Per-series model fitting (a *dictionary of fitted statsmodels objects*, keyed by `series_id`, rather than one global model). A shared helper `_multi_step_predict` maps each series' h-step-ahead forecast onto the corresponding rows of `future` by rank/position. ARIMA order selection combines an **Augmented Dickey-Fuller (ADF) stationarity test** to pick the differencing term `d`, with a small **AIC-based grid search** over `(p, d, q)` candidates to pick the final order — explicitly called out in the task as needing explanation (see below). `warnings.filterwarnings("ignore")` is set module-wide to suppress statsmodels convergence chatter. **Detailed Explanation:** Both models operate on the "one model per series" principle — since ARIMA and Exponential Smoothing are univariate time-series methods with no natural way to share statistical strength across unrelated products/stores, `fit` loops over `train.groupby(SERIES_COL)` and fits/stores one model object per series ID into a `dict`. This differs fundamentally from the tree models (one global model, all series encoded as features) and is a deliberate trade-off: statistically well-founded per series, but with fitting cost that scales linearly with the number of series and no cross-series generalization.

The shared `_multi_step_predict(forecast_fn, future)` helper handles turning per-series forecasts into a flat prediction array aligned with `future`'s row order: for each series' rows (a group `g`) in `future`, it calls `forecast_fn(sid, h)` where `h = len(g)` is the number of forecast steps needed for that series, and assigns the returned array's first `h` values to those rows' positions in the output. This assumes `future` rows for a given series are contiguous *in relative rank* — i.e., the first row for a series corresponds to forecast step 1, second to step 2, etc. (the docstring states this explicitly: "rank of each date within its series = forecast step").

`ARIMAModel.fit` chooses the differencing order `d` via ADF: if the p-value from `adfuller(y)` is below 0.05, the series is judged stationary and `d=0`; otherwise `d=1` (only ever one level of differencing is considered — no second-order differencing path). It then builds a small candidate grid of `(p, d, q)` orders — `[(1,d,1), (2,d,2), (0,d,1), (1,d,0)]` if `auto_order=True`, else fixed `(1,1,1)` — fits an `ARIMA` model for each candidate, and keeps whichever has the lowest AIC (Akaike Information Criterion, a standard goodness-of-fit-vs-complexity trade-off metric), wrapping each fit attempt in a `try/except` so a candidate that fails to converge is silently skipped rather than crashing the whole fit. If literally no candidate order fits, it raises `RuntimeError`. A ponytail comment in the source explicitly flags this 4-candidate AIC grid as a deliberate simplification versus a full `pmdarima.auto_arima` search, with an explicit upgrade path noted if search quality becomes a problem.

`ExpSmoothingModel.fit` is simpler: for each series it adds a tiny epsilon (`1e-6`) to the series (likely to avoid zero/negative-value issues with multiplicative-style component initialization even though `trend="add", seasonal="add"` is used) and fits `ExponentialSmoothing` with additive trend and additive seasonality at a fixed `seasonal_periods=7` (weekly seasonality assumption, sensible for daily sales data). Both models' `predict` methods simply delegate to `_multi_step_predict`, calling `.forecast(h)` on the stored per-series fitted object. **Key Code Sections Explained:** - `_multi_step_predict(forecast_fn, future)` — the shared row-mapping helper; critical because it decouples "how to produce an h-step forecast for one series" (passed in as a lambda) from "how to lay those forecasts back onto the flat `future` DataFrame," letting both `ARIMAModel` and `ExpSmoothingModel` reuse identical placement logic. - ADF test + AIC order search (`ARIMAModel.fit`, lines ~46-62) — this is the statistically important piece flagged in the task: `adfuller(y)[1] < 0.05` tests the null hypothesis of a unit root (non-

stationarity); rejecting it (p < 0.05) means the series is already stationary so no differencing is needed (`d=0`), otherwise one difference is applied (`d=1`). Then rather than trusting one fixed `(p,d,q)` , four small candidate orders are tried and the one minimizing AIC (balances fit quality against number of parameters, penalizing overfitting) is kept — a lightweight, per-series automatic order selection without the overhead of a full grid/stepwise search library. - `ARIMAModel.orders: dict[str, tuple]` — records the chosen order per series purely for logging/diagnostics (`logger.info(... orders=%s)`), not used in prediction logic itself. - `ExpSmoothingModel.fit` 's `+ 1e-6` epsilon — a defensive numerical nudge; without more context in this file it's not fully explained why additive (not multiplicative) mode would need this, so this detail is somewhat ambiguous/defensive-only rather than mathematically required by additive Holt-Winters. **How To Use/Call This File:** Instantiated by `model_factory.create_models()` only when `models.arima.enabled` / `models.exponential_smoothing.enabled` are explicitly set `True` in config (both default to disabled, unlike the baseline models). Used via the standard `BaseModel.fit` / `.predict(future, feature_cols, history)` interface, though note neither model actually uses `feature_cols` (they operate purely on `TARGET_COL` history) — `history` is also not consulted since these store their own fitted per-series state and use `future` 's span/length purely to know how many steps `h` to forecast. **Important Notes:** Because forecasting is per-series and stateful (`self.fitted[sid]`), predicting for a series never seen during `fit` will raise a `KeyError` on `self.fitted[sid]` — there is no cold-start/unseen-series fallback here (unlike LSTM, which has an explicit unseen-series scaling fallback). The `_multi_step_predict` assumption that `future` rows are contiguous-per-series-in-rank-order is load-bearing and undocumented as an explicit precondition beyond the docstring comment — if `future` is not sorted by series then by date, forecasts will be misassigned. `ARIMAModel` 's `seasonal` constructor parameter is accepted but never actually used anywhere in `fit` / `predict` (dead parameter, based on the code as written).

FILE: SRC/MODELS/TREE_MODELS.PY

Type: Core Logic (concrete `BaseModel` implementations, gradient-boosted/ensemble trees) with a shared internal base class **Purpose:** Implements LightGBM (the platform's primary/preferred model per ARCHITECTURE.md), XGBoost, and Random Forest, all as thin wrappers around scikit-learn-API regressors sharing one fit/predict/importance implementation. **Why Created (Reasoning):** These three tree ensembles all expose an identical scikit-learn-compatible API (`.fit(X, y)` , `.predict(X)` , `.feature_importances_`), so rather than triplicating fit/predict/importance code three times, a private shared base class (`_SklearnLike`) captures the common logic once, and each concrete model class only supplies the specific estimator instance. **Dependencies:** `numpy` , `pandas` ; internally imports `src.constants.TARGET_COL` , `src.models.base_model.BaseModel` . The actual ML libraries (`lightgbm.LGBMRegressor` , `xgboost.XGBRegressor` , `sklearn.ensemble.RandomForestRegressor`) are imported lazily *inside* each subclass's `__init__` , not at module top — mirroring the deferred-import pattern used for `torch` /LSTM in `model_factory.py` , so importing `tree_models` doesn't hard-require all three libraries be installed if only one is used. **Key Concepts: Adapter pattern layered inside the Strategy pattern** — `_SklearnLike(BaseModel)` adapts any scikit-learn-API `estimator` object to the `BaseModel` interface, and `LightGBMModel` / `XGBoostModel` / `RandomForestModel` each just plug a different estimator into that adapter. **Global model across all series**: unlike the statistical models, there is exactly one fitted estimator per model class, trained on the entire `train` DataFrame (all series' rows concatenated) at once, relying on `feature_cols` to encode which series/time/lag context each row belongs to (this is the mechanism explicitly called out in the task to explain, elaborated below). **Detailed Explanation:** `_SklearnLike.__init__` stores an `estimator` (whatever regressor was constructed by the concrete subclass) and an empty `_cols` list. `fit` records `feature_cols` and calls `self.est.fit(train[feature_cols], train[TARGET_COL])` — a single call across the *entire* training frame, not looped per series. `predict` calls `self.est.predict(future[self._cols])` , again across all rows/series at once. `feature_importance` reads `.feature_importances_` off the estimator (an attribute all three tree libraries expose) if present, returning `None` otherwise, and returns a sorted `pd.Series` indexed by feature name — identical shape/contract to `LinearRegressionModel.feature_importance` in `baseline_models.py` .

The three concrete subclasses (`LightGBMModel` , `XGBoostModel` , `RandomForestModel`) each take a `params: dict` (hyperparameters straight from config) and a `seed: int = 42` , construct their respective estimator with `random_state=seed` (or `seed=` where the library expects that name) plus the given `params` spread in, and call `super().__init__(estimator)` . `LGBMRegressor` additionally sets `verbose=-1` (suppress LightGBM's own logging), and `XGBRegressor` sets `verbosity=0` ; `RandomForestRegressor` sets `n_jobs=-1` (use all CPU cores) — small library-specific quality-of-life defaults layered on top of the shared adapter.

This "one global model across all series" design is the core mechanism to understand: rather than fitting a separate LightGBM per product/store combination (which would be expensive and data-hungry per series), the *entire* long-format training frame — all series stacked into one table — is fed to a single `LGBMRegressor.fit()` call. What lets the model still differentiate between series is entirely the *feature engineering* upstream: `feature_cols` presumably includes a series identifier (encoded numerically/categorically) plus per-series lag/rolling-window features, calendar features, etc. The tree model learns a single set of split rules that can condition on those per-series encodings/features to produce series-specific predictions — e.g., a split on a `series_id`-derived feature or on a rolling-mean feature that differs per series lets the tree branch differently for different series without needing a distinct model object per series. This is a standard "global model with entity features" pattern common in demand forecasting (e.g., similar to how M5-competition-winning LightGBM approaches worked) — much more data-efficient than one-model-per-series when there are many series with limited history each, since patterns learned from one series' trees can implicitly transfer to another via shared feature space. **Key Code Sections Explained:** - `_SklearnLike` — the shared adapter class; the single most important abstraction in this file, because it's what keeps `LightGBMModel` / `XGBoostModel` / `RandomForestModel` each down to a 3-line `__init__` (import + construct estimator + `super().__init__`). - `_SklearnLike.fit` — one global `.fit()` call over the whole (all-series-concatenated) `train` frame; this is the exact mechanism behind the "one global model, per-series via encodings" pattern called out above. - `_SklearnLike.feature_importance` — uses `getattr(self.est, "feature_importances_", None)` defensively, since not every possible sklearn-API estimator is guaranteed to expose that attribute, even though all three currently wired-in estimators do. - Lazy `from lightgbm import LGBMRegressor` / `from xgboost import XGBRegressor` / `from sklearn.ensemble import RandomForestRegressor` inside each `__init__` — deferred imports so, e.g., a machine without `xgboost` installed can still use `LightGBMModel` and `RandomForestModel` without an ImportError at module load time (only the actually-instantiated model's import runs). **How To Use/Call This File:** Instantiated by `model_factory.create_models()` for whichever of `lightgbm` / `xgboost` / `random_forest` config keys are enabled (all default to disabled — `enabled: False` fallback), passing the rest of that config sub-dict as `params` and the global `random_seed` as `seed`. Used identically to every other model via `BaseModel.fit` / `.predict`; ARCHITECTURE.md/README describe LightGBM as the primary/preferred model of the seven. **Important Notes:** Because there is one global model, all series must be present together in the `train` frame at fit time — you cannot incrementally fit new series in without refitting the whole model (unlike ARIMA/LSTM which handle unseen series more gracefully at predict time, see their notes). `predict` will raise a `KeyError` if `future` is missing any column that was in `feature_cols` at fit time (same fragility pattern as `LinearRegressionModel`). None of these models use the `history` parameter of `predict` — all necessary lookback must already be encoded into `feature_cols`.

FILE: SRC/MODELS/LSTM_MODEL.PY

Type: Core Logic (concrete `BaseModel` implementation, deep learning / PyTorch neural network) **Purpose:** Implements a global sequence-to-sequence LSTM that maps the last `seq_len` (default 60) days of a series' sales directly to the next `horizon` (default 30) days in one forward pass, trained across all series at once with per-series min-max scaling. **Why Created (Reasoning):** Tree models and statistical models can miss complex temporal/sequential patterns (e.g., longer-range dependencies, non-linear interactions across time steps) that a recurrent neural network can potentially capture; including an LSTM lets the platform's model comparison stage evaluate whether deep learning earns its (much higher) training cost and complexity over simpler approaches. It's also explicitly optional (deferred import in `model_factory.py`) since `torch` is a heavy dependency not everyone running the platform will have installed. **Dependencies:** `numpy`, `pandas`, `torch` (and `torch.nn`); internally imports `src.constants.{DATE_COL, SERIES_COL, TARGET_COL}`, `src.logger.get_logger`, `src.models.base_model.BaseModel`. **Key Concepts: Direct multi-horizon (seq2seq) architecture**: a stack of `nn.LSTM` layers followed by a single `nn.Linear` head that outputs all `horizon` (30) future values in one shot from the final hidden state — this is "direct" multi-step forecasting (all horizons predicted simultaneously by one output layer), contrasted with "recursive" multi-step forecasting (predict 1 step, feed it back in, repeat) which is actually what happens *within* `predict` when the requested prediction span exceeds `horizon` (see below — the model is architecturally direct but is called recursively/iteratively at inference to cover more than 30 days). **Per-series min-max scaling** (`self.scalers: dict[str, tuple[float, float]]`) rather than global scaling, since different series can have wildly different sales magnitudes. Early stopping on a held-out validation split during training. Global model (one shared network across all series, same "encodings implicit in scaled sequence" idea as tree models but via the sequence itself rather than explicit feature columns). **Detailed Explanation:** The `_Seq2Seq` `nn.Module` builds a stack of `nn.LSTM` layers per the `units` list (default `[64, 32]` — first LSTM layer has 64 hidden units taking a single input feature (`in_size=1`, just the scaled sales value) per time step, second LSTM layer has 32 units taking the first layer's 64-dim output per

time step), with dropout applied after each LSTM layer. The `head` is a single `nn.Linear(units[-1], horizon)` that takes only the *final* time step's hidden state (`x[:, -1, :]`) from the last LSTM layer and projects it directly to a `horizon` -length vector — i.e., the entire 30-day forecast is produced in one linear projection from one hidden vector, not step-by-step. This is the "direct multi-horizon" design explicitly called out in the task.

`LSTMModel.fit` builds training examples per series: for each series, it min-max-scales the full target history to `[0, 1]` (storing `(min, max)` in `self.scalers[sid]`), then slides a window across the scaled series producing `(seq_len, horizon)` input/output pairs — every possible offset `i` where `X = ys[i:i+seq_len]` (last 60 scaled days) and `Y = ys[i+seq_len : i+seq_len+horizon]` (next 30 scaled days) — across *all* series combined into one big `X` / `Y` tensor pool. This is the "60 in -> 30 out" architecture named in the task: the network never sees the raw series identity as an explicit feature (no categorical series embedding); its only input per time step is the single scaled sales value, and cross-series structure is captured only via the shared scaling-then-windowing procedure feeding one shared network. Training uses Adam optimizer, MSE loss, a 90/10 train/validation split (`torch.utils.data.random_split` , seeded), and early stopping: after each epoch it evaluates validation loss, and if it fails to improve by at least `1e-5` for 5 consecutive epochs, training stops early and the best-validation-loss weights (saved via `state_dict()`) are reloaded at the end.

`LSTMModel.predict` requires `history` (raises `ValueError` if `None` — the one model in this group that hard-requires it functionally, not just per the interface docstring). For each series in `future` , it pulls that series' full history, uses its stored scaler (or computes a fresh min-max scaler from that history if the series was never seen during `fit` — the unseen-series fallback), scales the last `seq_len` values as the seed sequence, then **recursively/iteratively** calls the network: each forward pass produces `horizon` (30) predicted values, appended to the running sequence, and the loop continues (using its own predictions as further input, "roll forward in horizon-sized chunks") until enough predicted values exist to cover all of that series' rows in `future` , truncating to exactly `len(g)` . Final output is un-scaled back to original units and clipped at zero (`np.clip(..., 0, None)`) since sales can't be negative. **Key Code Sections Explained:** - `_Seq2Seq.forward` — `x, _ = lstm(x)` chained across `self.lstms` , then `self.head(x[:, -1, :])` : this is the direct-multi-horizon mechanism — only the last time step's final-layer hidden state feeds the linear head, and that head alone determines all 30 output values simultaneously (no per-step decoding loop inside the network itself). - `LSTMModel._scale` / `_unscale` — per-series min-max scaling using the stored `(min, max)` tuple with a `1e-9` epsilon to avoid divide-by-zero for constant series; essential because raw sales magnitude varies hugely across series and a shared network needs comparable input ranges to learn a single set of weights that generalizes. - `LSTMModel.fit` 's windowing loop (`for i in range(len(ys) - self.seq_len - self.horizon + 1)`) — this is literally where the "60 in, 30 out" supervised training pairs are constructed; note it requires each series to have at least `seq_len + horizon` (90) data points or it contributes zero training examples for that series (and if *no* series has enough history, `fit` raises `RuntimeError("Not enough history...")`). - Early stopping block (`best_val` , `patience` , threshold `1e-5` , patience limit `5`) — standard technique to prevent overfitting on a fixed epoch budget; reloads `best_state` at the end so the returned model reflects its best validation checkpoint, not necessarily the final epoch's weights. - `LSTMModel.predict` 's recursive rollout (`while len(out) < len(g): ... seq.extend(chunk)`) — this is the key nuance flagged in the task: even though the *network* is architecturally direct (one shot to 30 steps), the *model class* behaves recursively when the caller needs more predictions than one 30-day chunk, by feeding its own prior chunk's output back in as if it were real history for the next chunk. This compounds forecast error across chunks for spans longer than `horizon` . **How To Use/Call This File:** Never imported at module load time by `model_factory.py` (which imports `src.models.base_model` , `baseline_models` , `statistical_models` , `tree_models` eagerly at the top, but not `lstm_model`). Instead, `model_factory.create_models()` only does `from src.models.lstm_model import LSTMModel` inside the `if m.get("lstm", {}).get("enabled", False):` branch, wrapped in `try/except ImportError` — if `torch` isn't installed, LSTM is silently skipped with a warning log rather than crashing the whole factory. When enabled, it's used through the same `BaseModel.fit` / `.predict(future, feature_cols, history)` interface as every other model, except it is the one model that will actively error (`ValueError`) if `predict` is called without `history` . **Important Notes:** Per-series scaling means the same trained network weights are reused across series, but each series' inputs/outputs are transformed by that series' own min/max — an implicit assumption that within-series relative dynamics (shape) generalize across series even though absolute scale doesn't. The recursive rollout for spans beyond `horizon` is a source of compounding error not present in the other models' forecasting (ARIMA/ExpSmoothing use each library's own genuine multi-step `.forecast(h)` , not a naive recursive re-feed of point-predictions). Unseen-series handling at predict time (auto-computing a fresh scaler from that series' own history) is more graceful than ARIMA/ExpSmoothing (which would `KeyError`) but still requires that series to have at

least `seq_len` history points, or the input sequence will be shorter than expected with no explicit guard/error for that case in the code shown. `feature_cols` is accepted by both `fit` / `predict` for interface compliance but never actually used — the LSTM works purely off the raw `TARGET_COL` sequence, not engineered features.

FILE: SRC/MODELS/MODEL_FACTORY.PY

Type: Factory **Purpose:** Reads the run configuration's `models` section and instantiates exactly the enabled models with their configured hyperparameters, returning a flat `list[BaseModel]` ready for training. **Why Created (Reasoning):** Centralizes all "which of the seven models are turned on, and with what hyperparameters" decision-making in one place, so the training pipeline doesn't need to know about config structure or individual model constructors at all — it just calls `create_models(config)` and iterates the result. This also isolates the one genuinely optional/heavy dependency (`torch` for LSTM) behind a deferred import + `try/except ImportError`, so the rest of the platform (baselines, tree models, statistical models) works even on machines without PyTorch installed. **Dependencies:** `src.logger.get_logger`; `src.models.base_model.BaseModel`; `src.models.baseline_models.{LinearRegressionModel, MovingAverageModel}`; `src.models.statistical_models.{ARIMAModel, ExpSmoothingModel}`; `src.models.tree_models.{LightGBMModel, RandomForestModel, XGBoostModel}`; and, lazily inside the function body only, `src.models.lstm_model.LSTMModel`. **Key Concepts: Factory pattern** — a single function (`create_models`) that maps a config dict to a list of constructed model objects, hiding each model's constructor signature from the caller. Per-model `enabled` flags in config, with differing defaults (`moving_average` / `linear_regression` default to enabled `True`; `lightgbm` / `xgboost` / `random_forest` / `arima` / `exponential_smoothing` / `lstm` default to disabled `False`). Deferred/guarded import for the one optional heavy dependency (LSTM/torch). **Detailed Explanation:** `create_models(config)` pulls `m = config["models"]`, a global `seed` (defaulting to 42), and `horizon` from `config["forecasting"]["forecast_horizon"]` (used only for constructing `LSTMModel`, since it's the only model whose constructor needs to know the forecast horizon explicitly — the others infer horizon implicitly from however many rows are in `future` at predict time). It then builds up a `models: list[BaseModel]` by checking each model's config sub-dict for `enabled`, and appending a constructed instance when true.

The two baselines (`MovingAverageModel`, `LinearRegressionModel`) are checked first and default to enabled if the config key is entirely absent — reflecting their role as always-on sanity-check baselines. The three tree models are handled uniformly via a loop over `(key, cls)` pairs `[("lightgbm", LightGBMModel), ("xgboost", XGBoostModel), ("random_forest", RandomForestModel)]`: for each, it reads that model's config sub-dict, and if enabled, strips out the `enabled` key itself and passes everything else in the sub-dict as `params` to the constructor along with the global `seed`. ARIMA and ExpSmoothing are checked next, both defaulting to disabled, with `ARIMAModel` additionally reading `auto_order` / `seasonal` sub-keys. Finally, LSTM is checked last — only if `models.lstm.enabled` is `True` does it attempt `from src.models.lstm_model import LSTMModel` inside a `try/except ImportError`; on success it constructs `LSTMModel(m["lstm"], horizon=horizon, seed=seed)` (passing the *entire* `lstm` config sub-dict, including its own `enabled` key, directly as `params` — unlike the tree-model loop, which explicitly strips `enabled` out before passing `params`, this is a minor inconsistency: `LSTMModel.__init__` reads specific keys out of `params` via `.get(...)` so the stray `enabled` key is simply ignored rather than causing an error). If the import fails, a warning is logged and LSTM is silently omitted from the roster rather than crashing.

At the end, it logs the list of enabled model names (`[x.name for x in models]`) for visibility, and returns the assembled list. This function is almost certainly called once per training run by whatever orchestrates training (e.g., a `train.py` script or pipeline runner elsewhere in the codebase, outside this file group), which then loops over the returned list calling `.fit` / `.predict` on each via the common `BaseModel` interface. **Key Code Sections Explained:** - `create_models(config) -> list[BaseModel]` — the single factory function; its full body is the entirety of the file's logic, no classes, just conditional construction. - The tree-model loop (`for key, cls in (("lightgbm", LightGBMModel), ...)`) — demonstrates the factory reducing three near-identical enable/construct blocks (that would otherwise be copy-pasted three times, once per tree model) to one loop, since all three tree wrappers share the same `(params, seed)` constructor signature. - `params = {k: v for k, v in cfg.items() if k != "enabled"}` — strips the factory's own bookkeeping key (`enabled`) out of what actually gets passed as hyperparameters to the model constructor, so estimator constructors don't receive a stray unexpected `enabled=True/False` kwarg. - The LSTM branch's `try/except ImportError` — the critical piece making `torch` an optional dependency for the whole platform: only *this* branch, only when LSTM is explicitly enabled in config, ever attempts to import `torch`-dependent code. - `logger.info("Models enabled: %s", [x.name for x in models])` — relies on each model's class-level `name` attribute (`"MovingAverage"`, `"LinearRegression"`, `"LightGBM"`, `"XGBoost"`, `"RandomForest"`, `"ARIMA"`, `"ExpSmoothing"`, `"LSTM"`) for a human-

readable roster summary in logs. **How To Use/Call This File:** This is the entry point external code should use to get a runnable model roster — callers pass the parsed run `config` dict (matching `config.yaml`'s structure) and get back a `list[BaseModel]`, then drive each one through `.fit(train, feature_cols)` / `.predict(future, feature_cols, history)` per the `BaseModel` contract, with no need to import any individual model class themselves (except `base_model.BaseModel` for typing, if desired). **Important Notes:** Enabling/disabling defaults are inconsistent by design (baselines default on, everything else defaults off) — a config that omits the `models` section entirely, or omits individual model keys, silently determines the roster via these mixed defaults, which is worth knowing when debugging "why didn't model X train." The LSTM `params` dict passed to `LSTMModel.__init__` includes the `enabled` key (unlike the tree-model path, which strips it) — harmless given how `LSTMModel.__init__` reads params via `.get()`, but a minor asymmetry between the two code paths. `create_models` performs no validation on hyperparameter values themselves (e.g., malformed `lstm_units` or negative windows) — any such validation, if it exists, must live in config loading/validation code outside this group.

Group 5: Training, Cross-Validation & Evaluation

INTRO: WHY THESE FILES BELONG TOGETHER, AND READING ORDER

This group is the heart of DemandIQ's "does this actually work" guarantee: it decides how models are tested (`cross_validator.py`), how the whole train → compare → refit → test → register flow is orchestrated (`train_pipeline.py`), and how "good" is defined numerically (`metrics.py`). None of the three is useful alone — `train_pipeline.py` calls into both of the others on every fold, and the two `__init__.py` files are empty package markers that just make `src.training` and `src.evaluation` importable packages (no re-exports, no logic).

Read them in this order:

1. **`src/evaluation/metrics.py`** — smallest, no dependencies on the rest of the codebase, defines the vocabulary (MAPE/RMSE/MAE/R²/SMAPE) everything else reports in.
2. **`src/training/cross_validator.py`** — defines the fold layout (`Fold`, `make_folds`) that every model is trained/validated against. Still self-contained (only needs a DataFrame with a date column).
3. **`src/training/train_pipeline.py`** — the orchestrator. Ties `cross_validator` + `metrics` + `model_factory` + `mlflow_utils.tracker` together into the full train/select/refit/test/register flow described in ARCHITECTURE.md.

The two `__init__.py` files need no dedicated reading time — see the brief note below.

FILE: SRC/TRAINING/INIT.PY AND SRC/EVALUATION/INIT.PY

Type: Package marker (empty) **Purpose:** Turn `src/training/` and `src/evaluation/` into importable Python packages (`src.training.cross_validator`, `src.training.train_pipeline`, `src.evaluation.metrics`). **Why Created (Reasoning):** Required by Python's package import system — without an `__init__.py`, these directories can't be imported as `src.training.*` / `src.evaluation.*` in the way `train_pipeline.py` (`from src.training.cross_validator import make_folds`) and others do. **Dependencies:** None — both files are empty. **Key Concepts:** No re-exports, no `__all__`, no package-level logic; purely structural. **Detailed Explanation:** Both files were read and are empty. They carry no code, so there is nothing to explain functionally — their only role is enabling the dotted-module import paths used throughout the codebase (e.g. `src.training.train_pipeline`, `src.evaluation.metrics`). This is a common minimal pattern in this repo: package `__init__.py` files are left empty unless a package genuinely needs a shared re-export. **Key Code Sections Explained:** N/A (no code). **How To Use/Call This File:** Never imported directly; Python imports it implicitly whenever anything under `src.training` or `src.evaluation` is imported. **Important Notes:** If someone later wants a convenience re-export (e.g. `from src.training import run_training`), it would go here — but nothing in the codebase currently relies on that, so keep them empty.

FILE: SRC/TRAINING/CROSS_VALIDATOR.PY

Type: Core Logic (utility module — pure function + dataclass, no side effects) **Purpose:** Compute walk-forward cross-validation fold boundaries (expanding train window, sliding validation window) plus a held-out final test window, over a DataFrame's unique dates. **Why Created (Reasoning):** Naive shuffled/random k-fold CV would let a model train on future dates and validate on past ones, leaking information a production model would never have at prediction time (ARCHITECTURE.md calls this out explicitly: "Shuffled CV leaks the future"). This module exists specifically to produce date-ordered, non-overlapping folds that mirror how a real forecasting model is retrained over time. **Dependencies:** `dataclasses.dataclass`, `pandas`, `src.constants.DATE_COL`, `src.logger.get_logger`. No dependency on `model_factory.py` or `mlflow_utils/tracker.py` — this module is pure data-splitting logic with zero knowledge of models or tracking. **Key Concepts:** `Fold` dataclass (`fold`, `train_mask`, `val_mask` — boolean masks aligned to the DataFrame's index); `make_folds(df, n_folds=3) -> (list[Fold], test_mask)`; module constants `VAL_FRAC=0.10`, `TEST_FRAC=0.10`, `STEP_FRAC=0.05`; the private helper `at(frac)` that maps a fraction of the timeline to an actual date. **Detailed Explanation:** `make_folds` first collects the sorted list of unique dates in the input DataFrame (`df[DATE_COL].unique()`, sorted) — critically, folds are computed purely from date *position* in this unique timeline, not from row position, so every series/`series_id` in the long-format data splits at exactly the same calendar dates. `n` is the number of unique dates, and `at(frac)` returns the date sitting at that fraction of the timeline (clamped to the last index).

The test window is fixed first: `test_start = at(1 - TEST_FRAC)`, i.e. the last 10% of dates, and `test_mask` flags every row on or after that date. This window is carved out before any fold is built and is never touched by fold construction, guaranteeing it stays untouched through all of CV — it only gets used once, at the very end of `train_pipeline.py`.

Then, for `i` in `range(n_folds)`, the code computes `val_end_frac = (1 - TEST_FRAC) - (n_folds - 1 - i) * STEP_FRAC` and `val_start_frac = val_end_frac - VAL_FRAC`. For the default `n_folds=3` this produces exactly the layout documented in the module docstring: fold 1 trains on `[0, 70%)` and validates on `[70%, 80%)`; fold 2 trains on `[0, 75%)` and validates on `[75%, 85%)`; fold 3 trains on `[0, 80%)` and validates on `[80%, 90%)`. Each fold's `train_mask` is simply "every row dated before the validation window start" (`df[DATE_COL] < v0`) — so train always starts at the very beginning of the timeline and expands forward — while `val_mask` is `[v0, v1)`. This is what makes it "expanding window, sliding validation": train never shrinks or moves, it only grows fold over fold, while the validation slice slides forward in `STEP_FRAC` (5%) increments up to the edge of the test window. **Key Code Sections Explained:** - **`at(frac)`** (lines 39-40): converts a fraction of the unique-date timeline into a concrete `pd.Timestamp`, via `dates.iloc[min(int(n * frac), n - 1)]`. The `min(..., n-1)` clamp prevents an index-out-of-range when `frac` rounds up to exactly 1.0. - **Test window carve-out** (lines 42-43): `test_start = at(1 - TEST_FRAC)` and `test_mask = df[DATE_COL] >= test_start` — this is computed once, outside the fold loop, and returned separately from `folds` so callers (`train_pipeline.py`) can never accidentally train or validate any fold against it. - **Fold loop** (lines 46-56): the core walk-forward logic. `val_end_frac` walks backward from `(1 - TEST_FRAC)` in `STEP_FRAC` steps as `i` increases, so the last fold's validation window sits immediately before the test window and earlier folds' windows sit progressively earlier. `train_mask = df[DATE_COL] < v0` is why train is "expanding": it is always "everything before this fold's validation start," which necessarily grows as `v0` moves later across folds. - **Why shuffled k-fold would leak (reasoning, not code):** an sklearn-style shuffled `KFold` assigns rows to folds independently of date, so a model could easily be trained on rows from October and asked to validate on rows from August — i.e., trained using information from the future relative to what it's being scored on. That is impossible in production (a real forecast can only use data available up to "today"). By deriving every mask from date position (`df[DATE_COL] < v0` / `>= v0 & < v1`), `make_folds` guarantees `train_max < val_min` for every fold, so a model never sees a future date during training that it is later scored against. **How To Use/Call This File:** `train_pipeline.run_training()` calls `folds, test_mask = make_folds(features, config["forecasting"]["cv_folds"])` once per training run, then iterates `for fold in folds: train = features[fold.train_mask]; val = features[fold.val_mask]` for every model, and separately does `trainval = features[~test_mask]; test = features[test_mask]` for the final refit/test step. No other module in the codebase calls `make_folds`. **Important Notes:** Fold boundaries are computed on **unique dates**, not rows, which is essential for a long-format multi-series DataFrame (`date, series_id, ...`) — otherwise a DataFrame with more rows for later dates (e.g. more active series over time) would skew the fractional split. `STEP_FRAC` (5%) is smaller than `VAL_FRAC` (10%), so consecutive folds' validation windows overlap by design (fold 1 validates `[70,80)`, fold 2 validates `[75,85)`) — this is intentional (more folds without shrinking each validation window below a usable size), not a bug. The function will raise nothing for degenerate/small `n`, but very small date ranges could produce empty or overlapping-with-test folds since `at()` only clamps the upper index, not fold ordering.

FILE: SRC/TRAINING/TRAIN_PIPELINE.PY

Type: Orchestrator **Purpose:** Run every enabled model through the identical walk-forward CV folds, log each run to MLflow, select the best model by mean MAPE (RMSE tiebreak), refit that model on train+val, evaluate it once on the held-out test window, register/promote it in the MLflow model registry, and persist all comparison/summary artifacts to disk. **Why Created (Reasoning):** The project's core promise is a fair, leakage-free comparison across 7 heterogeneous model types (baseline, tree, statistical, LSTM) with a single, reproducible selection rule. This module is the one place that owns that end-to-end decision process so `main.py` and the retraining orchestrator don't have to duplicate it. **Dependencies:** `numpy`, `pandas`, `time`; `src.constants` (`BEST_MODEL_META_PATH`, `BEST_MODEL_PATH`, `COMPARISON_PATH`, `DATE_COL`, `FEATURE_IMPORTANCE_PATH`, `TARGET_COL`); `src.evaluation.metrics.evaluate`; `src.features.feature_engineering.feature_columns`; `src.logger.get_logger`; **`src.mlflow_utils.tracker`** (`tracker.setup`, `tracker.log_run`, `tracker.register_best`); **`src.models.model_factory.create_models`**; `src.training.cross_validator.make_folds`; `src.utils.serialization` (`save_json`, `save_pickle`). **Key Concepts:** `run_training(features, config) -> dict` (the single public entry point); `_model_params(model)` (private helper to extract loggable hyperparameters); per-fold loop building a long `rows` list of per-model/per-fold metrics; `avg` — a per-model mean-metrics DataFrame sorted by `["mape", "rmse"]` (this sort *is* the selection rule: MAPE primary, RMSE tiebreak) with a `rank` column; the refit-on-`trainval` / evaluate-once-on-`test` step; the `summary` dict persisted to `BEST_MODEL_META_PATH`. **Detailed Explanation:** `run_training` starts by computing feature columns (`feature_columns(features)`), building the CV folds and test mask via `make_folds(features, config["forecasting"]["cv_folds"])`, turning on MLflow tracking (`tracker.setup(config)`, which no-ops if disabled in config), and instantiating every enabled model via `create_models(config)` (baseline, tree, statistical, and — if torch is installed and enabled — LSTM models, per `model_factory.py`).

The main nested loop is `for model in models: for fold in folds:` — every model is fit on exactly the same `train_mask` / `val_mask` slices for a given fold, so the comparison across models is apples-to-apples. For each (model, fold) pair it times `model.fit(train, fcols)` and `model.predict(val, fcols, history=train)`, computes metrics via `evaluate(val[TARGET_COL].to_numpy(), preds)` from `metrics.py`, appends timing info, logs a row, and — if MLflow is enabled — calls `tracker.log_run(...)` with hyperparameters, dataset sizes, and horizon as params and the metric dict as MLflow metrics. Any exception during a given model/fold is caught, logged, and skipped (`logger.error(..., exc_info=True)`) rather than aborting the whole run — a single bad model config doesn't take down the comparison of the other six.

After the loop, if literally every model failed on every fold, it raises `RuntimeError("Every model failed on every fold — check the logs")`. Otherwise it builds the `comparison` DataFrame from all logged rows, saves it to `COMPARISON_PATH` (`reports/model_comparison.csv`), and computes `avg` = the per-model mean of `mape, rmse, mae, r2_score, smape, training_time_seconds, inference_time_ms`, **sorted by** **`["mape", "rmse"]`** — this single line is the concrete implementation of "MAPE primary, RMSE tiebreak" model selection. `best_name = avg.index[0]` is simply the first row after that sort. The winning model object is then refit from scratch on `trainval = features[~test_mask]` (everything up through validation, i.e. all folds' train+val combined) and evaluated exactly once via `evaluate(...)` against `test = features[test_mask]` — the window that no fold ever touched during CV. The fitted model plus its feature-column list is pickled to `BEST_MODEL_PATH` (`models/best_model.pkl`), feature importances (if the model exposes `feature_importance()`) go to `FEATURE_IMPORTANCE_PATH`, and if MLflow is on, `tracker.register_best(...)` registers the model and promotes it to Production only if it beats the current Production version's test MAPE by the configured margin (>2%), otherwise it lands in Staging. Finally a `summary` dict (best model name, avg CV MAPE, test metrics, full comparison table, registry info, timestamp, feature/row counts) is saved to `BEST_MODEL_META_PATH` and returned to the caller. **Key Code Sections Explained:** - **`_model_params(model)` (lines 23-28):** pulls `model.est.get_params()` when the wrapped model exposes a scikit-learn-style estimator (`est`), filtering to JSON/MLflow-loggable scalar types (`int, float, str, bool`, non-None) — statistical/ baseline models without an `est` attribute simply log no hyperparameters. - **The train/eval loop over models × folds (lines 40-68):** this is "train all models on all folds." Every model sees the identical `fold.train_mask` / `fold.val_mask` from `cross_validator.make_folds`, so no model gets an easier or harder split than another. `infer_ms` is per-row inference latency (`(time.time()-t0)/max(len(val),1)*1000`), guarding against division-by-zero on an empty validation slice. - **Model selection (lines 73-82):** `avg = comparison.groupby("model")[...].mean(). sort_values(["mape", "rmse"])` then `best_name = avg.index[0]` — this is exactly "pick the best by mean MAPE, RMSE tiebreak" from the architecture doc; `rank` is assigned purely for the persisted comparison report, not used for selection logic itself. - **Refit-and-test (lines 83-89):** `best_model.fit(trainval, fcols)` re-trains the winning model on all pre-test data (folds' train+val combined) rather than reusing any fold-specific fitted model, then scores it exactly once on `test` — the previously untouched final 10% window from

`cross_validator`. This single test-set evaluation is the number used for registry promotion decisions. - **Persistence + registration (lines 91-117):** `save_pickle({"model": best_model, "feature_cols": fcols}, BEST_MODEL_PATH)` bundles the model with the exact feature column list it was trained on (needed later for consistent inference); `tracker.register_best(...)` (in `mlflow_utils/tracker.py`) handles the Staging/Production promotion gate; registration failures are caught and logged rather than crashing training, since a working `best_model.pkl` is still valuable even if MLflow registration fails. **How To Use/Call This File:** `main.py` calls `summary = run_training(features, config)` as one step of the full pipeline (after `build_features`, before `generate_forecast`). `src/retraining/orchestrator.py` calls the same `run_training(features, config)` inside its `retrain()` flow after rebuilding features from scratch, then writes a JSONL retraining-event log using the returned summary. Both callers treat `run_training` as the single entry point into this module — nothing else in `train_pipeline.py` is meant to be imported externally. **Important Notes:** Per-fold exceptions are swallowed (logged, not raised), so a model that crashes on every fold simply won't appear in `comparison` / `avg` — it silently loses rather than blocking the run; only a *total* wipeout across all models raises. The refit step re-fits the winning model from scratch on `trainval`, it does **not** reuse any of the fold-fitted model instances, so the model that gets pickled/registered was never actually evaluated during CV — CV only picks *which model class/config* wins, the final artifact is a fresh fit. MLflow logging and registration are both optional/best-effort (`mlflow_on` flag from `tracker.setup`, try/except around `register_best`) so the pipeline still works end-to-end with MLflow disabled or misconfigured.

FILE: SRC/EVALUATION/METRICS.PY

Type: Utility (pure functions) **Purpose:** Compute the five forecast accuracy metrics used throughout the platform — MAPE, RMSE, MAE, R², SMAPE — and bundle them into one dict via `evaluate()`. **Why Created (Reasoning):** Every fold, every model, and the final test evaluation all need a single, consistent definition of "how good is this forecast" so results are comparable across models and folds; centralizing the metric math in one module prevents subtly different MAPE/RMSE implementations creeping into different model classes. **Dependencies:** `numpy`; `sklearn.metrics` (`mean_absolute_error`, `mean_squared_error`, `r2_score`). No dependency on any other DemandIQ module — this file is a leaf. **Key Concepts:** `mape(y_true, y_pred)`, `smape(y_true, y_pred)` (both hand-rolled, both return a 0-1 fraction), `evaluate(y_true, y_pred) -> dict` (returns MAE, RMSE, R² from sklearn plus MAPE/SMAPE from the local functions, with MAPE/SMAPE converted to percentages and rounded to 4 decimals). **Detailed Explanation:** This module is deliberately small and dependency-light: two custom metric functions (`mape`, `smape`) plus one aggregator (`evaluate`) that also leans on scikit-learn for the three "standard" regression metrics. `evaluate` is the only function called from outside this module (by `train_pipeline.run_training`) — `mape` and `smape` are still exported and used directly by the test suite, but production code only ever calls `evaluate`.

The two custom functions exist because MAPE and SMAPE need special handling that generic regression-metric libraries don't provide for a demand-forecasting context: real sales data frequently contains true zeros (no sales that day for a given series), and a percentage-error metric is undefined (division by zero) on those rows. **Key Code Sections Explained:** - **`mape(y_true, y_pred)` (lines 6-12) — how zero-sales rows are handled:** it builds `mask = y_true != 0` and computes `np.mean(np.abs((y_true[mask]-y_pred[mask])/y_true[mask]))` **only over the masked (nonzero-actual) rows** — rows where the true value is 0 are dropped from the MAPE calculation entirely rather than causing a divide-by-zero or being clamped/imputed. If every row in the batch has a zero actual (`not mask.any()`), it returns `nan` rather than crashing. This directly matches ARCHITECTURE.md's documented decision: "Zero-sales rows are skipped in MAPE (SMAPE reported for that regime)." - **`smape(y_true, y_pred)` (lines 15-20) — the complementary metric for the zero regime:** computes `denom = (|y_true| + |y_pred|) / 2` and `ratio = |y_true - y_pred| / denom`, but explicitly special-cases `denom == 0` (which happens when both actual and predicted are exactly zero) to contribute `0.0` to that row instead of `nan` or a divide error — i.e., "predicted no sales, had no sales" is treated as a perfect (zero-error) prediction for that row, rather than being excluded like it is in MAPE. This is exactly why SMAPE is reported alongside MAPE: it stays well-defined even for the all-zero rows MAPE has to drop. - **`evaluate(y_true, y_pred)` (lines 23-31) — the five metrics:** - **MAE** (`mean_absolute_error`): mean absolute error, same units as sales — simple, robust to outliers relative to RMSE. - **RMSE** (`sqrt(mean_squared_error(...))`): root mean squared error — penalizes large misses more than MAE; used as the tiebreak when two models have near-identical MAPE. - **R²** (`r2_score`): fraction of variance in actual sales explained by the predictions; can be negative for a model worse than predicting the mean. - **MAPE**: `mape(y_true, y_pred) * 100`, rounded to 4 decimals — the platform's *primary* selection metric (scale-independent, interpretable as "average % off"), computed only over nonzero-actual rows as described above. - **SMAPE**: `smape(y_true, y_pred) * 100`, rounded to 4 decimals — reported as the complementary metric that stays defined over the zero-sales regime

MAPE excludes. **How To Use/Call This File:** `src/training/train_pipeline.py` imports and calls `evaluate(val[TARGET_COL].to_numpy(), preds)` once per (model, fold) during CV, and again once for the final held-out test evaluation. No other production module calls into `metrics.py` directly; `tests/test_metrics.py` imports `evaluate`, `mape`, and `smape` directly to unit-test the metric math itself. **Important Notes:** MAPE and SMAPE are computed as 0-1 fractions internally and only multiplied by 100 inside `evaluate()` — calling `mape()` / `smape()` directly (as the tests do) yields a fraction, not a percentage, which is a common source of confusion when reading test assertions like `abs(mape(y, p) - 0.075) < 1e-9`. `r2_score` and the sklearn error metrics have no special zero-handling — only the two hand-rolled functions (`mape`, `smape`) treat zero-actual rows specially.

TEST COVERAGE: WHAT INVARIANTS ARE ACTUALLY PROVEN

`tests/test_cv.py` (2 tests, both against a synthetic 200-day single-series DataFrame) is the proof of the walk-forward guarantee described above: - `test_walk_forward_order_and_no_overlap` asserts, for every one of the 3 folds: `train_max < val_min` (strict temporal ordering — training data never contains a date at or after any validation date), `not (train_mask & val_mask).any()` (train and validation are strictly disjoint), and `not (val_mask & test_mask).any()` (validation never overlaps the held-out test window). It also asserts fold train-set sizes are non-decreasing and the last fold's train set is strictly larger than the first (`sizes == sorted(sizes) and sizes[0] < sizes[-1]`) — this is the concrete proof that train windows expand rather than slide or shrink. - `test_test_window_is_last_10pct` asserts the test mask selects roughly 10% of the 200 rows (`15 <= test_mask.sum() <= 25`) and that every test-window date is strictly after every non-test-window date (`df.loc[test_mask,"date"].min() > df.loc[~test_mask,"date"].max()`) — i.e., the test window really is a contiguous trailing slice of the timeline, not a scattered sample.

`tests/test_metrics.py` (3 tests) proves the metric correctness claims documented above: - `test_mape_skips_zeros` uses `y=[0,100,200]`, `p=[50,110,190]` and asserts `mape(y,p) ≈ 0.075` — this is the mean of `|100-110|/100=0.10` and `|200-190|/200=0.05` (average 0.075), with the `y=0` row's `(0-50)/0` excluded entirely, proving the masking behavior rather than, say, a zero being clamped to a small epsilon. - `test_smape_zero_both` asserts `smape([0,0],[0,0]) == 0.0` exactly, proving the `denom == 0 -> 0.0` special case (rather than `nan`) fires correctly. - `test_evaluate_keys_and_perfect_fit` asserts `evaluate(y,y)` returns exactly the five expected keys (`mae, rmse, r2_score, mape, smape`) and that a perfect prediction (`y_pred == y_true`) yields `mape == 0`, `rmse == 0`, `r2_score == 1` — the sanity check that every metric agrees "no error" means zero/one in the expected direction.

HOW THIS GROUP FITS THE LARGER PIPELINE

`main.py` and `src/retraining/orchestrator.py` are the only two callers of `run_training()`. Both follow the same sequence: load/clean data → build features → `run_training(features, config)` → (main.py additionally calls `generate_forecast` and drift detection afterward). Inside `run_training`, `cross_validator.make_folds` and `evaluation.metrics.evaluate` are invoked repeatedly (once per model per fold, then once more for the final test evaluation), while `src.models.model_factory.create_models` and `src.mlflow_utils.tracker` (`setup`, `log_run`, `register_best`) are the two external subsystems this orchestrator wires together with the CV/metrics logic covered here. The practical artifacts this group produces — `reports/model_comparison.csv`, `reports/feature_importance.csv`, `models/best_model.pkl` + `best_model_meta.json`, and MLflow's registry state — are exactly the "training" row of ARCHITECTURE.md's Artifacts Contract table, which the dashboard (a pure reader of that contract) later displays.

Group 6: Experiment Tracking, Drift Monitoring & Automated Retraining

These three concerns — MLflow tracking/registry, drift + performance monitoring, and the retraining orchestrator — are grouped together because they form a single feedback loop, not three independent features. Tracking (`mlflow_utils/tracker.py`) records what every training run did and gate-keeps promotion to Production. Monitoring (`monitoring/drift_detector.py`, `monitoring/performance_monitor.py`) watches the *live* production model against fresh data to decide whether it's still good enough. Retraining (`retraining/orchestrator.py`) is the glue that reads the monitoring verdict, re-runs the whole pipeline, and

hands the result back to the tracker for a new promotion decision. Together they implement exactly the "Retraining state machine" from ARCHITECTURE.md: `manual button OR drift≥3 features OR MAPE +15% → retrain() → full pipeline rerun → register if better → JSONL event log`.

Recommended reading order: `mlflow_utils/tracker.py` first (it defines the run-logging and promotion vocabulary the rest of the system assumes), then `monitoring/drift_detector.py`, then `monitoring/performance_monitor.py` (the two signals that feed triggers), and finally `retraining/orchestrator.py` (which consumes both monitoring modules and calls back into `tracker.py` indirectly via `train_pipeline.py`). The two `__init__.py` files (`src/mlflow_utils/__init__.py`, `src/monitoring/__init__.py`) and `src/retraining/__init__.py` are empty package markers — noted below but not given full sections since there is no logic to explain.

Note on empty files: `src/mlflow_utils/__init__.py`, `src/monitoring/__init__.py`, and `src/retraining/__init__.py` were all read and are empty (zero bytes) — they exist solely so Python treats `mlflow_utils/`, `monitoring/`, and `retraining/` as importable packages. There is no package-level re-export or side effect to document.

FILE: SRC/MLFLOW_UTILS/TRACKER.PY

Type: Core Logic / Integration Utility **Purpose:** Wraps the MLflow tracking and Model Registry APIs so the rest of the codebase logs runs and promotes models through one small, config-driven interface instead of calling the `mlflow` SDK directly everywhere. **Why Created (Reasoning):** The project needs an auditable experiment history (every model × every CV fold, with params/metrics) and a way to decide, mechanically rather than by hand, when a newly trained model should replace the one in Production. Centralizing this in one module means `train_pipeline.py` doesn't need to know MLflow's API surface, and the promotion rule (the >2% MAPE gate from ARCHITECTURE.md) lives in exactly one place. **Dependencies:** `mlflow`, `mlflow.tracking.MlflowClient` (both third-party), `src.logger.get_logger`. Consumed by `src/training/train_pipeline.py` (`tracker.setup`, `tracker.log_run`, `tracker.register_best`) and by `dashboard/server.py` (`tracker.get_model_versions` — confirmed via grep of `dashboard/server.py` line 18/201). **Key Concepts:** `mlflow.start_run` context manager; `MlflowClient` for registry operations (`create_model_version`, `search_model_versions`, `transition_model_version_stage`); "enabled" no-op pattern driven by `config["mlflow"]["enabled"]`; Staging vs Production model-version stages; `test_mape` stashed as a **tag string** on each model version (not a registry-native metric) so it can be re-read later for comparison. **Detailed Explanation:** `setup(config)` points the global MLflow client at the configured tracking URI (`sqlite:///mlflow.db` per config.yaml) and experiment name, returning `False` immediately if `mlflow.enabled` is `False` in config — every other function in this file is written to be skippable so the whole tracking layer is optional. `log_run(...)` is called once per (model, CV fold) pair during training: it opens a run named `{model_name}_fold{fold}`, logs the hyperparameters plus `model_name` / `cv_fold` as params, logs only the numeric entries of the metrics dict (guarding against non-numeric values crashing `mlflow.log_metrics`), sets any tags passed in (e.g. the `run_tags` block from config.yaml: `environment`, `project`), and logs any extra artifact file paths. It returns the run id, though callers in `train_pipeline.py` do not currently use the return value.

`register_best(...)` is the promotion gate. It opens one more MLflow run to log the winning model's summary metrics (`avg_cv_mape`, `test_mape`) and upload the pickled model file as an artifact, then registers a new model version against the registry name from config (creating the registered model itself on first use via `_ensure_registered`). It then searches all existing versions of that registered model, filters to ones currently in the `Production` stage (excluding the just-created version), and takes the **minimum** `test_mape` among them (parsed back out of the `test_mape` tag string, since MLflow doesn't store custom numeric metrics per registry version) as `prod_mape` — defaulting to `inf` if there is no production version yet. `get_model_versions(config)` is a read-only, dashboard-facing query: it re-points the tracking URI, lists all versions of the registry model, and returns a flat list of dicts (version, stage, model_type, test_mape, created) for the UI, swallowing any exception into a warning + empty list so a broken/missing MLflow store doesn't crash the dashboard.

This file is the terminus of the training pipeline (`run_training` in `train_pipeline.py` calls `tracker.setup` once at the start, `tracker.log_run` inside the per-fold loop, and `tracker.register_best` once after the final held-out test evaluation) and the source of truth the dashboard reads model version/stage info from. **Key Code Sections Explained:** - `setup(config)` (lines 13-20): the enable/disable switch. If `mlflow.enabled` is falsy, every downstream `tracker.*` call in `train_pipeline.py` is gated behind the `mlflow_on` boolean it returns, so training still runs and produces `best_model.pkl` even with MLflow entirely off. - `log_run(model_name, fold, params, metrics, tags, artifacts)` (lines 23-33): one run per model/fold. Note the metrics

dict is filtered with `isinstance(v, (int, float))` before calling `mlflow.log_metrics` — this protects against a metrics dict that might contain non-numeric summary fields. - `register_best(config, model_path, model_name, avg_mape, test_mape)` (lines 36-69) — **this is the >2% MAPE promotion gate**: after registering the candidate as a new model version (tagged with its own `test_mape`), it computes `prod_mape` as the best (lowest) `test_mape` tag among current Production versions. The promotion condition is `promote = test_mape < prod_mape * (1 - threshold) or not prod` — i.e., promote if there is no existing Production model at all, OR the new candidate's test MAPE is more than `threshold` (default 0.02 = 2%) fractionally lower than the current production's test MAPE. If promoted, the new version is transitioned to `"Production"` with `archive_existing_versions=True`, which auto-archives the old production version(s); if not promoted, it's transitioned to `"Staging"` and the previous production model is left untouched and still serving. - `_ensure_registered(client, name)` (lines 72-77): idempotent "create registered model if it doesn't already exist" — catches whatever exception `get_registered_model` raises (typically `RestException` for a 404) rather than checking a specific exception type. - `get_model_versions(config)` (lines 80-93): dashboard read path; wraps everything in try/except so registry/DB issues degrade to an empty list plus a logged warning instead of a 500 to the frontend. **How To Use/Call This File:** `src/training/train_pipeline.py` imports `from src.mlflow_utils import tracker` and drives the whole lifecycle: `tracker.setup(config)` once, `tracker.log_run(...)` inside the nested model/fold loop guarded by `if mlflow_on:`, and `tracker.register_best(...)` once after refit-and-test, wrapped in its own try/except so a registry failure doesn't crash the training run (only logs an error and leaves `registry_info = {}`). `dashboard/server.py` imports `get_model_versions` directly (`from src.mlflow_utils.tracker import get_model_versions`, used at line 201) to populate a models/versions view. **Important Notes:** (1) `test_mape` is stored only as a string tag on the model version, not a first-class registry metric — `register_best` re-parses it back with `float(v.tags.get("test_mape", "inf"))`, so if that tag is ever missing on a version it silently treats that version as having infinite (i.e., worst possible) MAPE rather than raising. (2) The `mv` returned by `create_model_version` is only produced `if _ensure_registered(...)` — since `_ensure_registered` always returns `True`, `mv` is effectively always set, but the conditional-expression style (`... if ... else None`) means a future change to `_ensure_registered` returning `False` would leave `mv = None` and the subsequent `mv.version` access would raise `AttributeError`; this is a latent fragility, not a bug today. (3) There is no explicit handling in this file for MLflow being entirely uninstalled (unlike Evidently in drift_detector.py) — `mlflow` and `mlflow.tracking` are hard imports at module load, so if the package is missing, importing `tracker` itself fails; the "optional" behavior only covers the *tracking_uri is disabled in config* case, not *mlflow isn't installed*.

FILE: SRC/MONITORING/DRIFT_DETECTOR.PY

Type: Core Logic **Purpose:** Detects statistical drift between the training-era distribution and a recent window of incoming data, using a Kolmogorov-Smirnov test on the top-N most important features plus the target, and optionally renders an Evidently HTML report. **Why Created (Reasoning):** A production forecasting model degrades silently as the real-world input distribution shifts (new promo patterns, seasonality changes, etc.). This module gives the retraining orchestrator a concrete, thresholded signal — number of drifted features — to trigger on, rather than relying purely on downstream error metrics which may lag behind the underlying cause. **Dependencies:** `pandas`, `scipy.stats.ks_2samp` (third-party); `src.constants` (`DATE_COL`, `DRIFT_HTML_PATH`, `DRIFT_REPORT_PATH`, `FEATURE_IMPORTANCE_PATH`, `TARGET_COL`); `src.logger.get_logger`; `src.utils.serialization.save_json`. Optionally, inside a try/except, `evidently` (new API: `evidently.Report`, `evidently.presets.DataDriftPreset`) and its legacy equivalents (`evidently.report.Report`, `evidently.metric_preset.DataDriftPreset`). Called by `src.retraining.orchestrator.check_triggers` and directly by `dashboard/server.py`. **Key Concepts:** Two-sample KS test (`ks_2samp`) as the statistical core; reference/current split by a rolling cutoff date (`recent_window_days`); `top_features(n)` reading `reports/feature_importance.csv` (written by training) to decide *which* features to monitor; a three-level status enum (`LOW` / `MEDIUM` / `HIGH`) derived from drift counts with the target feature weighted specially; graceful, try/except-wrapped optional HTML reporting via Evidently with dual old/new API support. **Detailed Explanation:** `top_features(n=5)` reads the feature-importance CSV produced by the training pipeline and returns the top-N feature names by importance rank (the CSV's row order, since it's read with `index_col=0` and simply `.head(n)` is taken — importance is assumed pre-sorted by whoever wrote `feature_importance.csv`, i.e., `best_model.feature_importance()` in `train_pipeline.py`). If the file doesn't exist yet (e.g., no training run has happened), it returns an empty list, and `detect_drift` falls back to a hardcoded `["lag_7", "rolling_mean_7"]` pair.

`detect_drift(features, config)` is the main entry point. It reads `drift_detection` config (`recent_window_days` default 30, `p_value_threshold` default 0.05, `features_to_monitor` default 5 — passed as `n` into `top_features`), determines the monitored columns (top features that actually exist in the current feature frame, plus the target column always appended), and splits the full feature DataFrame into `reference` (everything up to `max(date) - recent_window_days`) and `current` (everything after) using the max date present in the data as the anchor — not wall-clock "today." If either side has fewer than 10 rows, it returns early with `status: "UNKNOWN"` and an empty features list, since a KS test on tiny samples is unreliable. Otherwise, for every monitored column it runs `ks_2samp(reference[c].dropna(), current[c].dropna())` , records the KS statistic, p-value, both means, and a boolean `drift` flag (`p < p_thresh`). It counts how many features drifted (`n_drifted`) and separately checks whether the *target* column itself drifted. The overall `status` is `"HIGH"` if the target drifted AND at least 3 features drifted overall, `"MEDIUM"` if at least 2 drifted (regardless of target), else `"LOW"` . The full report dict (timestamp, row counts, n_drifted, status, a canned recommendation string, and the per-feature results) is persisted to `DRIFT_REPORT_PATH` via `save_json` and returned. Finally it calls `_evidently_report` as a side effect for the optional HTML artifact.

This module is consumed two ways: by `retraining/orchestrator.py` 's `check_triggers` , which reads `n_drifted` off the same features list using its own threshold (`drift_feature_count_trigger` , default 3) — note this is a *different* count/threshold pairing than the KS module's own internal `status` classification, so a report tagged "MEDIUM" internally is not necessarily what triggers retraining; and directly by `dashboard/server.py` 's drift endpoint, which calls `detect_drift` on demand for a "check drift now" dashboard action. **Key Code Sections Explained:** - `top_features(n)` (lines 18-22): purely reads a CSV artifact; has no fallback for corrupt/malformed CSVs beyond whatever pandas raises. - `detect_drift(features, config)` (lines 25-75) — **the KS-test mechanism**: builds the monitored column set as `[top-5 important features that exist in this data] + [TARGET_COL]` , splits by a date cutoff computed from the *data's own max date* (not `pd.Timestamp.now()`), and for each column runs `scipy.stats.ks_2samp` comparing the reference and current distributions (NaNs dropped per-column independently before the test). A feature is flagged drifted when its two-sample KS p-value falls below `p_value_threshold` (default 0.05) — the classic "reject the null hypothesis that both samples come from the same distribution" framing. The `status` classification (`HIGH` / `MEDIUM` / `LOW`) is a bespoke severity heuristic layered on top of the raw counts, giving special weight to target drift. - `_evidently_report(reference, current, cols)` (lines 78-97) — **the optional Evidently HTML report**: first tries the modern Evidently API (`from evidently import Report` ; `from evidently.presets import DataDriftPreset` ; `Report([DataDriftPreset()]).run(current[cols], reference[cols])` , then `.save_html(...)` on the returned snapshot). If that whole block raises anything (import error because Evidently isn't installed, or an API mismatch), it falls into a second try that imports the pre-0.7 legacy API instead (`evidently.report.Report` , `evidently.metric_preset.DataDriftPreset` , constructed with `metrics=[...]` and run via keyword args `reference_data=` / `current_data=`). If *that* also fails, it logs a warning with the original exception message and returns silently — drift detection's core KS results are entirely unaffected either way, since `_evidently_report` is called after the JSON report is already saved and its own return value is discarded. **How To Use/Call This File:** Two call sites confirmed by code: `src/retraining/orchestrator.py` 's `check_triggers(features, config)` calls `detect_drift(features, config)` directly to get the current drift report and decide whether to add a "Data drift on N features" trigger reason. `dashboard/server.py` imports `detect_drift` locally inside its drift-check endpoint (`from src.monitoring.drift_detector import detect_drift` , confirmed near line 251) to expose a manual "run drift check" dashboard action, reading the `features.parquet` artifact and returning the report JSON directly to the frontend. **Important Notes:** If Evidently is not installed, `_evidently_report` 's first `try` raises an `ImportError` on `from evidently import Report` , is caught, the legacy import is also caught by the same absence, and the function just logs `"Evidently report skipped: <ImportError message>"` — this matches the README's troubleshooting entry ("Evidently HTML missing → KS-test drift core still runs; Evidently report is optional") and ARCHITECTURE.md's stated rationale ("scipy KS is stable across environments and versions; Evidently renders the pretty HTML report when importable"). Also worth flagging: `config["drift_detection"]["framework"]` (set to `'evidently'` in config.yaml) and `statistical_test: 'ks'` are never actually read by this file — `detect_drift` unconditionally always runs the KS test and unconditionally always attempts the Evidently report; those two config keys appear to be descriptive/documentation-only rather than functional switches (confirmed: this file only reads `recent_window_days` , `p_value_threshold` , and `features_to_monitor` from the `drift_detection` config block). Edge case: with fewer than 10 rows on either side of the cutoff, the function returns `{"status": "UNKNOWN", "features": []}` without writing a report file at all — a caller checking `n_drifted` on that dict via `.get("features", [])` safely gets `0` .

FILE: SRC/MONITORING/PERFORMANCE_MONITOR.PY

Type: Core Logic **Purpose:** Recomputes MAPE on a recent window of ground-truth data using the currently deployed best model, and compares it against the MAPE recorded when that model was trained/tested, flagging degradation beyond a configured threshold. **Why Created (Reasoning):** Drift detection (KS test on feature distributions) is a leading indicator, but the thing that actually matters is whether the model's real-world accuracy has degraded. This module closes that loop by directly re-scoring the deployed model on fresh labeled data and comparing it to its own training-time baseline — the second of the two automatic retraining triggers described in ARCHITECTURE.md's state machine. **Dependencies:** `pandas`; `src.constants` (`BEST_MODEL_META_PATH`, `BEST_MODEL_PATH`, `DATE_COL`, `TARGET_COL`); `src.evaluation.metrics.evaluate`; `src.logger.get_logger`; `src.utils.serialization` (`load_json`, `load_pickle`). Called by `src.retraining.orchestrator.check_triggers` and directly by `dashboard/server.py`. **Key Concepts:** Loading the persisted best-model bundle (`{"model", "feature_cols"}`) and its metadata JSON (which holds the original `test_metrics`); a recent-vs-history split by date, mirroring `drift_detector.py`'s pattern; relative degradation ratio `(recent_mape - baseline_mape) / baseline_mape`; a config-driven degradation threshold (`retraining.mape_degradation_threshold`, default 0.15 = 15%). **Detailed Explanation:** `check_performance(features, config)` first guards on both `BEST_MODEL_PATH` and `BEST_MODEL_META_PATH` existing — if either is missing (no training run has happened yet), it returns `{"status": "NO_MODEL"}` immediately. Otherwise it loads the pickled model bundle and the JSON metadata sidecar, reads the degradation threshold from `config["retraining"]["mape_degradation_threshold"]` and reuses `config["drift_detection"]["recent_window_days"]` (the same window size as drift detection, rather than defining its own separate window config key) to split `features` into `recent` (after the cutoff) and `history` (before/at the cutoff, passed as context to the model's predict method). If `recent` has fewer than 5 rows, it returns `{"status": "INSUFFICIENT_DATA"}`.

Given enough recent data, it calls `bundle["model"].predict(recent, bundle["feature_cols"], history=history)` to produce one-step-ahead predictions, scores them with `evaluate(...)` (the shared metrics module used everywhere else in the pipeline, giving MAPE/RMSE/MAE/R^2/SMAPE), and pulls the baseline MAPE out of `meta["test_metrics"]["mape"]` — the held-out test MAPE recorded at the time this model was trained and registered. `degradation` is computed as a fractional increase: `(recent_mape - baseline) / max(baseline, 1e-9)` (the `1e-9` floor guards divide-by-zero if the original baseline MAPE was exactly 0). `degraded` is `True` when `degradation > threshold`. The result dict reports `status` (`"DEGRADED"` or `"OK"`), both MAPE values, the degradation and threshold as percentages, a timestamp, and all the other recent-window metrics prefixed `recent_*` (via dict-comprehension merge). It logs at `warning` level if degraded, `info` otherwise.

This module has no persistence step of its own (unlike `drift_detector.py`, which writes `DRIFT_REPORT_PATH`) — its return value is only ever passed in-memory to whatever called it; the dashboard endpoint returns it straight to the frontend and `check_triggers` folds it into the combined trigger-decision dict without writing a separate performance-report file to disk. **Key Code Sections Explained:** - The existence guard (lines 17-18): `if not (BEST_MODEL_PATH.exists() and BEST_MODEL_META_PATH.exists()): return {"status": "NO_MODEL"}` — this is what lets the dashboard/orchestrator call this function safely even before the first training run has ever completed. - The recent/history split (lines 24-26) mirrors `drift_detector.py`'s cutoff logic exactly (`features[DATE_COL].max() - pd.Timedelta(days=window)`), but crucially reuses the *drift* config's window size rather than having its own — so changing `drift_detection.recent_window_days` in config.yaml also changes the performance-check window. - **The MAPE comparison and >15% degradation alert** (lines 30-39): `preds = bundle["model"].predict(recent, bundle["feature_cols"], history=history)` re-scores the exact same production model artifact against the freshest labeled rows available; `metrics = evaluate(...)` computes the current MAPE the same way training/testing did; `baseline = meta["test_metrics"]["mape"]` pulls the number that was true at deployment time; `degradation = (metrics["mape"] - baseline) / max(baseline, 1e-9)` expresses "how much worse, proportionally, is the model doing now" and `degraded = degradation > threshold` (threshold default 0.15) is the literal implementation of the "+15%" arrow in ARCHITECTURE.md's retraining state-machine diagram. **How To Use/Call This File:** `src/retraining/orchestrator.py`'s `check_triggers(features, config)` calls `check_performance(features, config)` and checks `perf.get("status") == "DEGRADED"` to add a `"MAPE degraded X% (threshold Y%)"` trigger reason. `dashboard/server.py` imports `check_performance` (`from src.monitoring.performance_monitor import check_performance`, confirmed near line 19/265) to expose a manual "check performance" dashboard action against the current artifacts. **Important Notes:** Because this reuses `drift_detection.recent_window_days` rather than having a dedicated config key, the "recent window" for drift and for performance monitoring cannot be tuned independently through config.yaml as it currently stands — this is worth calling out as a coupling, not a bug. Also note `check_performance` requires `BEST_MODEL_META_PATH` to contain `test_metrics.mape` — this metadata is written by `train_pipeline.py`'s `run_training`

(via `save_json` / `save_pickle` around `BEST_MODEL_PATH` / `BEST_MODEL_META_PATH`, evidenced by those constants being imported and used together in `train_pipeline.py`); if that JSON schema ever changes, this file would raise a `KeyError` rather than degrading gracefully (unlike `NO_MODEL` / `INSUFFICIENT_DATA`, there's no guard around the `meta["test_metrics"]["mape"]` access).

FILE: SRC/RETRAINING/ORCHESTRATOR.PY

Type: Orchestrator **Purpose:** Evaluates whether an automatic retrain should fire (based on drift + performance signals) and, when invoked, runs the entire pipeline end-to-end from raw data through registration, appending a JSONL audit event for every retraining run. **Why Created (Reasoning):** This is the piece that actually closes the feedback loop described in ARCHITECTURE.md's retraining state machine — something has to (a) decide, from the two monitoring signals, whether retraining is warranted, and (b) actually execute a full pipeline rerun and record what happened, in a form both the manual dashboard button and a future scheduler could call. **Dependencies:** `json`, `time` (stdlib); `pandas`; `src.constants.RETRAIN_LOG_PATH`; `src.logger.get_logger`; `src.monitoring.drift_detector.detect_drift`; `src.monitoring.performance_monitor.check_performance`. Inside `retrain()`, lazily imported (to avoid circular imports, per the code's own comment) `src.data_loading.loader.load_data`, `src.features.feature_engineering.build_features`, `src.preprocessing.preprocessor.clean`, `src.training.train_pipeline.run_training`, `src.validation.validator.run_validation`. Called by `dashboard/server.py`. **Key Concepts:** `check_triggers` as a pure decision function combining two independent signals; `retrain()` as the full pipeline re-execution (load → validate → clean → build_features → run_training → log event); an append-only JSONL event log (`RETRAIN_LOG_PATH`) as the audit trail; local/deferred imports specifically to break a circular-import chain. **Detailed Explanation:** `check_triggers(features, config)` is the decision layer. It reads the `retraining` config block, calls both `detect_drift(features, config)` and `check_performance(features, config)` (running both monitoring checks fresh, every time this is called — there is no caching), and builds a `reasons` list: if the number of drifted features (summed from `drift.get("features", [])`) is `>= drift_feature_count_trigger` (default 3), it appends a drift reason string; if `perf.get("status") == "DEGRADED"`, it appends a MAPE-degradation reason string. It returns `should_retrain` (simply `bool(reasons)` — true if either or both fired), the `reasons` list, and the raw `drift` / `performance` dicts for the caller (e.g., dashboard) to display in full.

`retrain(config, trigger="manual")` is the actual execution path. Its five pipeline-stage imports are deliberately done *inside the function body* rather than at module level — the docstring-adjacent comment explains this avoids a circular import with the main-pipeline modules (those modules likely import things that, transitively, would import back into `retraining` or a sibling package at load time; this is a workaround, not an architectural preference). It times the whole run (`t0 = time.time()`), then executes the identical stage sequence `main.py` presumably runs at startup: `load_data(config)` → `run_validation(raw)` (aborting with a `ValueError` if `report["valid"]` is false, embedding the schema errors in the exception message) → `clean(raw, config)` → `build_features(cleaned, config)` → `run_training(features, config)` (which internally does CV across all models, refits the winner, evaluates on the test window, and — as covered in `tracker.py` above — registers/promotes via the MLflow >2% gate). It assembles an `event` dict capturing trigger name, data volume, model/feature counts, CV fold count from config, the winning model name, its avg CV MAPE and test MAPE, whatever `registry_info` `run_training` produced (version/stage/etc., or `{}` if registration failed or MLflow was disabled), and total duration, then appends that event as one JSON line to `RETRAIN_LOG_PATH` (opened in append mode, UTF-8, `json.dumps(event, default=str)` to handle any non-JSON-native values like numpy scalars or Timestamps) followed by a newline. `read_retrain_log()` is the paired reader: returns `[]` if the log file doesn't exist yet, otherwise parses every non-blank line as JSON into a list of dicts, for the dashboard's retrain-history view. **Key Code Sections Explained:** - **`check_triggers(features, config)` — the trigger-check logic** (lines 16-31): two independently-computed booleans folded into one list of human-readable reasons; note that `check_triggers` runs both drift and performance checks unconditionally every time it's called — it doesn't itself gate on `config["retraining"]["trigger_mode"]` (which is never read anywhere in this file or referenced by name in the codebase, per grep — the config key exists in config.yaml with values `manual | automatic | hybrid` but there is no code path in `orchestrator.py`, `dashboard/server.py`, or `main.py` that branches on it). In practice, calling `check_triggers` always computes both signals and always returns whatever reasons apply; what differs by "mode" would presumably be whether something calls `check_triggers` automatically on a schedule versus only the dashboard's manual button calling `retrain()` directly — but no scheduler/cron code was found in this codebase, so "automatic"/"hybrid" trigger_mode currently has no enforcing code path. - **`retrain(config, trigger)` — the full retrain() flow** (lines 34-70): reload-everything-from-scratch semantics (not incremental)

— `load_data` re-reads/re-downloads/re-generates raw data, `run_validation` re-validates it and can abort the whole retrain, then clean → build_features → run_training re-executes the *entire* CV/train/select/test/register sequence, matching ARCHITECTURE.md's "full pipeline rerun on all data" description exactly. The abort path (`raise ValueError(...)` on invalid data) means a caller (the dashboard endpoint) needs to handle that exception — the dashboard code wraps `retrain(...)` in a try/except per the grep at `dashboard/server.py` lines 240-244. - **The JSONL event log format** (lines 54-67): one flat JSON object per line, fields = `timestamp` (ISO string via `pd.Timestamp.now().isoformat()`), `trigger` (the string passed in — `"manual"` by default, or e.g. `"manual (react dashboard)"` as passed from `dashboard/server.py`), `data_points` , `n_features` , `cv_folds` (read fresh from config, not from the training summary), `best_model` , `avg_cv_mape` , `test_mape` , `registry` (nested dict — version/stage/run_id/previous_prod_mape from `tracker.register_best` , or `{}`), and `duration_seconds` . Appending (`"a"` mode) rather than rewriting makes this a genuine audit trail — every retrain, whether it promoted a new model or not, leaves a permanent row. - `read_retrain_log()` (lines 73-77): trivial JSONL reader; will raise `json.JSONDecodeError` if any line in the file is corrupted/partially written (e.g., a crash mid-write), since there's no per-line try/except — this is a plausible edge case if the process is killed while `retrain()` 's `open(...).write(...)` is in flight, though a single `f.write` of a short line is unlikely to be interrupted mid-line on most filesystems. **How To Use/Call This File:** `dashboard/server.py` imports all three public functions (`from src.retraining.orchestrator import check_triggers, read_retrain_log, retrain` , confirmed at line 20). The `/api/retrain-log` GET endpoint returns `read_retrain_log()` directly. The `/api/retrain` POST endpoint (the dashboard's manual "Retrain Now" button) calls `retrain(config, trigger="manual (react dashboard)")` . A separate endpoint (near line 274) calls `check_triggers(features, config)` to show the user, without retraining, whether automatic conditions currently would fire. `main.py` was checked and does not call `check_triggers` or `retrain` at all — the initial pipeline run described in README.md ("python main.py") is a separate one-shot path that presumably calls `run_training` directly rather than going through this orchestrator (the orchestrator's `retrain()` exists specifically for *re*-training after the fact). **Important Notes:** The most important ambiguity to flag explicitly: **`config["retraining"]["trigger_mode"]` (`manual | automatic | hybrid`) is not read by any code in this repository.** `check_triggers` always evaluates both drift and performance regardless of mode, and nothing was found that periodically calls `check_triggers` or auto-invokes `retrain()` — the only invocation of `retrain()` found is the dashboard's manual button. So today, despite the config comment implying three selectable modes, the system behaves as "manual-trigger-only, with a separate read-only endpoint that reports whether automatic-style conditions are currently met" — a human (or a future scheduler this codebase doesn't yet include) must still decide to click retrain even when `check_triggers` reports `should_retrain: True` . This should be treated as a documented gap, not an inferred behavior. Separately: `retrain()` 's data-invalid abort (`ValueError`) means a bad retraining run does *not* get logged to the JSONL file at all (the log-append happens after training, near the end of the function) — so failed/aborted retrain attempts are invisible in `read_retrain_log()` 's history, only successful full runs are recorded.

Group 7: Forecast API & Dashboard Backend

These two files are the seam between the offline ML pipeline and the React frontend. `src/api/forecast_api.py` is the last pipeline stage — it turns a trained `best_model.pkl` into `reports/forecast.csv` . `dashboard/server.py` is the FastAPI layer that the frontend actually talks to: per the Artifacts Contract in ARCHITECTURE.md, it is a pure reader of pipeline output files (parquet/CSV/JSON artifacts written by earlier stages) plus two POST routes that trigger real work (`retrain()` , `detect_drift()`). `dashboard/helpers.py` does not exist in this repo (confirmed via glob `dashboard/*.py` → only `server.py`), so there is no separate helpers module to document.

Read order: `src/api/forecast_api.py` first (it's the producer of `reports/forecast.csv` , one of the artifacts the dashboard serves), then `dashboard/server.py` (the consumer/API surface), then skim `frontend/src/api.js` + `frontend/src/useApi.js` to see how the frontend calls these routes.

FILE: SRC/API/FORECAST_API.PY

Type: Core Logic (pipeline stage — forecast generation)

Purpose: Produces the next- `horizon` -periods forecast per series from the saved best model and writes it to `reports/forecast.csv` , which the dashboard later serves via `/api/forecast` and `/api/forecast/{series_id}` .

Why Created (Reasoning): Training produces a model that scores historical rows; something has to turn that model into an actual future forecast the dashboard can chart. Because different model families need fundamentally different forecasting strategies (single-step feature models vs. models that natively emit a whole horizon), this module centralizes that branching logic in one place instead of scattering it across model classes.

Dependencies: `numpy`, `pandas`; `src.constants` (`BEST_MODEL_PATH`, `DATE_COL`, `FORECAST_PATH`, `SERIES_COL`, `TARGET_COL`); `src.features.feature_engineering.build_features`; `src.logger.get_logger`; `src.models.lstm_model.LSTMModel`; `src.models.statistical_models.ARIMAModel`, `ExpSmoothingModel`; `src.utils.serialization.load_pickle`.

Key Concepts: - `_DIRECT_MODELS = (ARIMAModel, ExpSmoothingModel, LSTMModel)` — the tuple that decides which forecasting strategy runs. - `_future_rows(clean, horizon, freq)` — builds blank future date rows per series, carrying static columns (`item_id`, `store_id`, `sell_price`) and zeroing `promotion_flag`. - `generate_forecast(clean, config, model_bundle=None)` — the public entrypoint; returns a `[date, series_id, forecast]` DataFrame and writes `FORECAST_PATH`. - Recursive vs. direct multi-step forecasting (see below).

Detailed Explanation: `generate_forecast` loads the persisted model bundle (`{"model", "feature_cols"}`) unless one is passed in directly, reads the configured `forecast_horizon`, and picks a strategy based on the model's type. For direct models (ARIMA, exponential smoothing, LSTM) it builds all future date rows up front for every series via `_future_rows` and calls `model.predict(future, fcols, history=clean)` once — these models already know how to emit a full horizon (the LSTM does this via its seq2seq direct multi-horizon output per the ARCHITECTURE.md design decision; ARIMA/ExpSmoothing via their own multi-step forecast methods). For every other model (the feature-based ones: Moving Average, Linear Regression, LightGBM, XGBoost, Random Forest) it forecasts recursively: it appends one blank future row per series, regenerates the full feature set with `build_features` (so lag/rolling features see the just-predicted value), predicts that single step, clips negative predictions to 0, fills the row's target with the prediction, appends it to the working history, and repeats `horizon` times. This is exactly the leakage-consistent-but-compounding recursive approach documented in ARCHITECTURE.md ("day t+2 sees day t+1's forecast as `lag_1`").

After either path, the output is trimmed to `[date, series_id, forecast]`, clipped at 0, rounded to 2 decimals, and written to `FORECAST_PATH` (`reports/forecast.csv`) — the artifact `dashboard/server.py`'s `/api/forecast` routes read. This module is invoked once at the end of `main.py`'s pipeline run (and again inside `retrain()`'s full pipeline rerun), never on a live request path — the dashboard never calls this module directly, only reads its output file.

Key Code Sections Explained: - **`_DIRECT_MODELS` branch (recursive vs. direct multi-step):** This is the central design point of the file. - *Direct* (`isinstance(model, _DIRECT_MODELS)` → ARIMA, ExpSmoothing, LSTM): build every future row for the whole horizon at once, call `model.predict` a single time with `history=clean` so the model can use its own internal state (per-series ARIMA fit / per-series LSTM scaling, per ARCHITECTURE.md). No feature engineering is re-run per step because these models don't consume the ~40 engineered features — they operate on the raw series/history directly. - *Recursive* (everything else — the tree/linear/baseline feature models): loop `horizon` times; each iteration adds exactly one future row, calls `build_features(..., save=False)` to regenerate lag/rolling/decomposition features against the growing `work` DataFrame (so the freshly-appended prediction becomes visible as `lag_1`, `rolling_mean_*`, etc. on the next iteration), predicts only the last row per series (`feats.groupby(SERIES_COL).tail(1)`), clips to non-negative, and folds the result back into `work`'s target column before the next loop iteration. This deliberately trades off error compounding (each step's noisy prediction feeds the next) for reusing the same leakage-safe feature pipeline used at training time. - **`_future_rows`:** Generates the s calendar skeleton for future dates using `pd.date_range(last_date, periods=horizon+1, freq=freq)[1:]` (drops the known last date), carries forward static per-series attributes, and leaves `TARGET_COL` as NaN — the value later filled in by prediction. - **Bundle loading:** `model_bundle or load_pickle(BEST_MODEL_PATH)` lets callers (like `retrain()`) pass an in-memory freshly-trained bundle without a round trip through disk, while the default pipeline path loads the persisted `models/best_model.pkl`.

How To Use/Call This File: Not called by the dashboard or frontend at all — it's a pipeline-internal function called from `main.py` (end of the pipeline run) and from the retraining orchestrator (`src/retraining`) after a successful retrain. The frontend only ever sees its output indirectly, through `dashboard/server.py`'s `/api/forecast` and `/api/forecast/{series_id}` GET routes, which read the `reports/forecast.csv` this module writes.

Important Notes: If `models/best_model.pkl` doesn't exist, `load_pickle(BEST_MODEL_PATH)` will fail before this function can run — this is the root cause of the dashboard's "No pipeline artifacts" / "run the pipeline first" errors documented in the README troubleshooting table, since `reports/forecast.csv` never gets created. The recursive path's compounding error is a known, documented trade-off (not a bug) per ARCHITECTURE.md. There is no confidence interval logic here — the dashboard computes its own approximate ±1.96σ band client-side (in `server.py`) from historical diff-std, not from this module.

FILE: DASHBOARD/SERVER.PY

Type: API Server (FastAPI, dev-only)

Purpose: Serves the React dashboard: exposes GET routes that read the pipeline's on-disk artifacts (parquet/CSV/JSON) as JSON, plus POST routes that trigger retraining and drift detection.

Why Created (Reasoning): The pipeline (`main.py`) and the monitoring/ retraining modules produce files and can run expensive operations (retraining, drift checks), but the React frontend needs an HTTP/JSON interface. This file is that thin translation layer — explicitly scoped as "local dev only... not a model serving layer" (see its module docstring and ARCHITECTURE.md) so the pipeline and UI stay decoupled via the artifacts contract rather than a tightly coupled in-process call.

Dependencies: `sys`, `pathlib.Path` (to add the repo root to `sys.path`); `pandas`; `fastapi` (`FastAPI`, `HTTPException`, `UploadFile`), `fastapi.middleware.cors.CORSMiddleware`, `fastapi.responses.HTMLResponse`; `src.constants` (as `C`, for artifact paths); `src.mlflow_utils.tracker.get_model_versions`; `src.monitoring.performance_monitor.check_performance`; `src.retraining.orchestrator` (`check_triggers`, `read_retrain_log`, `retrain`); `src.monitoring.drift_detector.detect_drift` (imported lazily inside the route); `src.utils.config_loader.load_config`; `src.utils.serialization.load_json`.

Key Concepts: - `app = FastAPI(title="DemandIQ API")` with permissive CORS (`allow_origins=["*"]`, all methods/headers) — dev convenience, no auth. - Small helper functions: `_json(path_name)`, `_csv(path_name)` — look up a path by name off `src.constants` and load it only if it exists, else `None`. - `_clean_nan(obj)` — recursively converts float `NaN` to `None` since raw `NaN` isn't valid JSON and FastAPI's default encoder chokes on it. - `_require(df, name)` — raises `HTTPException(404, ...)` with a "not found — run the pipeline first" message when an artifact is missing; this is the literal source of the README's "Dashboard shows 'No pipeline artifacts'" troubleshooting entry. - Every GET route is a pure reader of one or more files under `C.<PATH>`; none of them execute pipeline logic.

Detailed Explanation: The file inserts the repo root onto `sys.path` so `src.*` imports resolve when run via `uvicorn dashboard.server:app`. It then builds one FastAPI app with wide-open CORS (acceptable only because this is explicitly a local dev server) and defines a set of GET routes, each reading one artifact type: model/drift status summary, cleaned-data overview with validation report and a 50-row sample, the list of series IDs, per-series EDA (history + optional seasonal decomposition + ACF/PACF, using `statsmodels` best-effort wrapped in `try/except` so missing dependencies or too-short series degrade gracefully to `None`), model comparison metrics grouped and ranked by MAPE, whole-portfolio and per-series forecast payloads (with a naive ±1.96×σ confidence band computed from the trailing 90-day diff standard deviation), MLflow run/version info, drift report JSON, an optional Evidently HTML drift report, and the retraining event log.

Two POST routes break the "pure reader" pattern by design (per the Architecture doc's explicit callout): `/api/retrain` calls `src.retraining.orchestrator.retrain(config, trigger="manual (react dashboard)")`, and `/api/drift-check` calls `src.monitoring.drift_detector.detect_drift(features, config)` directly against the current `features.parquet`. Two further GET routes (`/api/performance-check`, `/api/triggers`) also execute monitoring logic live rather than reading a cached report — they call `check_performance` and `check_triggers` from the monitoring/retraining modules on demand using the current `features.parquet`. `/api/upload` is a bare file-drop endpoint: it writes the uploaded bytes to `data/raw/uploaded.csv` and returns instructions telling the user to manually edit `config.yaml` — no automatic pickup.

Key Code Sections Explained: - **`GET /api/status`** — reads `BEST_MODEL_META_PATH` (JSON) and `DRIFT_REPORT_PATH` (JSON), and checks whether `CLEAN_DATA_PATH` parquet exists. Returns a dashboard summary: whether the pipeline has run, best model name, test/avg-CV MAPE, trained-at timestamp, feature count, drift status, and series count. All fields are `None` when the corresponding artifact is absent (no exception raised here, unlike the `_require`-guarded routes). - **`GET /api/data-overview`** — requires `CLEAN_DATA_PATH` via `_require` (raises 404 otherwise); reads the validation report JSON, computes per-column

missing-value percentages, and returns a 50-row NaN-cleaned sample of the cleaned data. - **`GET /api/forecast`** **and** **`GET /api/forecast/{series_id}`** — both require `FORECAST_PATH` (CSV) and `CLEAN_DATA_PATH` (parquet); they join the last 120 days of history per series with the forecast rows, and derive a confidence band as `1.96 * std(diff(sales over trailing 90 days))` — a simple heuristic, not a model-derived interval. `/api/forecast` additionally returns the full series list, model metadata, and top-15 feature importances for the default (first) series; `/api/forecast/{series_id}` returns the same history/forecast shape for one explicit series and 404s if that series isn't in the forecast file. - **`POST /api/retrain`** — the manual-retrain action button's target. Loads `config.yaml` via `load_config()`, then calls `retrain(config, trigger="manual (react dashboard)")` from `src.retraining.orchestrator`, wrapping any exception into an `HTTPException(500, str(e))`. This is the one route that runs the *entire* pipeline rerun (per ARCHITECTURE.md's retraining state machine), synchronously, blocking the HTTP request until it completes. - **`POST /api/drift-check`** — the manual drift-check action button. Requires `FEATURES_PATH` (features.parquet) to exist, otherwise 404s with "run the pipeline first". Lazily imports `detect_drift` from `src.monitoring.drift_detector` inside the function body (not at module top-level, unlike the other monitoring imports) and calls `detect_drift(features, config)` directly, returning its result live — distinct from `GET /api/drift`, which only reads the cached `DRIFT_REPORT_PATH` JSON written by a previous run.

How To Use/Call This File: The frontend's `frontend/src/api.js` creates one shared axios client: `export const api = axios.create({ baseURL: "http://localhost:8000/api" })`. `frontend/src/useApi.js` wraps `api.get(path)` in a data-fetching hook used throughout the app's pages. Actions are called directly with `api.post(...)`: `frontend/src/pages/Overview.jsx` calls `api.post("/retrain")` (Retrain button), `api.post("/drift-check")` (Check Drift button), and `api.post("/upload", form)` (CSV upload), while `frontend/src/pages/DriftMonitoring.jsx` calls `api.get("/performance-check")`. Because CORS is wide open (`allow_origins=["*"]`), the Vite dev server on `localhost:5173` can call `localhost:8000` with no extra configuration.

Important Notes: No authentication or authorization anywhere — acceptable only because this is explicitly a local, single-user dev server (documented in both the module docstring and README's Quick Start). Nearly every GET route will 404 with "... not found — run the pipeline first" until `python main.py` has run at least once and produced the artifacts listed in ARCHITECTURE.md's Artifacts Contract table — this is the concrete mechanism behind the README's "Dashboard shows 'No pipeline artifacts'" troubleshooting row. `/api/retrain` runs synchronously and can take as long as the full pipeline (data load → validate → clean → features → train 7 models × folds → register → forecast → monitor), so the frontend request will block/time out on a slow machine — no background job/polling mechanism exists. `/api/drift/html` depends on the optional Evidently report and 404s if it wasn't generated (per README: "KS-test drift core still runs; Evidently report is optional"). `/api/upload` does not wire the uploaded file into the pipeline automatically — it only returns instructions for the user to edit `config.yaml` by hand and retrain.

FILE: DASHBOARD/HELPERS.PY

Not present in this repository. `Glob dashboard/*.py` returns only `dashboard/server.py`; there is no separate helpers module — all reader/helper logic (`_json`, `_csv`, `_clean_nan`, `_require`) is defined inline in `server.py` itself (see above).

Group 8: React Frontend Dashboard

DIRECTORY TREE FOUND

```
frontend/src/components/
  ChartCard.jsx
  InfoTooltip.jsx
  Loading.jsx
  MetricCard.jsx
  Sidebar.jsx
  StatusBadge.jsx

frontend/src/pages/
  DataOverview.jsx
  DriftMonitoring.jsx
  Eda.jsx
  Forecasting.jsx
  MlflowTracking.jsx
  ModelComparison.jsx
  Overview.jsx

frontend/src/assets/
  hero.png
  react.svg
  vite.svg
```

OVERALL APP STRUCTURE

DemandIQ's dashboard is a single-page React app (Vite-built, React 19) that is a pure read/act client over the FastAPI dev server (`dashboard/server.py`, `http://localhost:8000/api/...`, documented in `README.md`). It has no build-time coupling to the Python pipeline — it only ever talks to that API over HTTP.

Bootstrapping. `index.html` is the Vite HTML shell: one `<div id="root">` and a module script pointing at `src/main.jsx`. `main.jsx` mounts `<App/>` into that div, wrapped in React's `<StrictMode>` (double-invokes effects/renders in dev to catch impurities) and react-router-dom's `<BrowserRouter>` (enables client-side, history-API-based routing so page navigation doesn't reload the document).

Routing. `App.jsx` is the top-level layout: a fixed `Sidebar` plus a `<main>` panel that renders one of seven pages via `<Routes>/<Route>`, matched on `react-router-dom` v7 path patterns (`/`, `/data`, `/eda`, `/models`, `/forecast`, `/mlflow`, `/drift`). Each route maps 1:1 to a page component in `src/pages/`, and the sidebar's nav links (`Sidebar.jsx`) are the only UI for switching between them — there's no nested/dynatic routing beyond that.

Data fetching pattern. Every page follows the same shape: call the custom `useApi(path)` hook (in `src/useApi.js`) to GET a JSON endpoint on mount (and whenever `deps` change), get back `{ data, error, loading, reload }`, then render one of three states — `Loading`, `ErrorBox`, or the real content (with `Empty` as a fourth "success but nothing there" state). Mutating actions (POST /retrain, POST /drift-check, POST /upload, GET /performance-check) are fired imperatively from event handlers using the shared `api` axios instance directly (not through `useApi`), with local `useState` flags for their own in-flight/disabled state and a lightweight home-grown "toast" (just a `<div>` + `setTimeout`) for one-shot feedback.

Shared config/utility layer. `src/api.js` is the single axios client (`baseURL: http://localhost:8000/api`) — every network call in the app goes through this one instance. `src/colors.js` exports a fixed 8-color `PALETTE` (referencing CSS custom properties defined in `index.css`, so colors flip automatically with light/dark theme) and `seriesColors(names)`, which assigns colors to series names by **sorting the names first** — this guarantees a given series always gets the same color regardless of what subset of series is currently displayed. `src/metricInfo.js` is a static dictionary of plain-language metric explanations (`METRIC_INFO`) and short display labels (`METRIC_LABEL`) keyed by backend metric name (`mape`, `rmse`, `ks_statistic`, etc.) — it decouples "what a metric is called on screen" and "how it's explained in the (i) tooltip" from every individual page, so adding/renaming a metric is a one-file change. `index.css` defines the entire visual system as CSS custom properties (`:root` for light, an `@media (prefers-color-scheme: dark)` block for dark) plus every reusable class the components/pages use (`.card`, `.badge`, `.metric-card`, `.section`, `.grid-*`, `.btn`, `.toast`, spinner keyframes, Recharts override selectors) — there is no CSS-in-JS or CSS modules; all styling is global classnames plus a handful of inline `style={{}}` one-offs for layout tweaks.

RECOMMENDED READING ORDER

1. `frontend/index.html` → `frontend/src/main.jsx` → `frontend/src/App.jsx` (entry chain and routing)
2. `frontend/src/api.js` → `frontend/src/useApi.js` (how every page gets data)
3. `frontend/src/colors.js`, `frontend/src/metricInfo.js`, `frontend/src/index.css` (shared config/styling every page/component leans on)
4. `frontend/src/components/*` (small, composed everywhere — read before the pages that use them)
5. `frontend/src/pages/Overview.jsx` (busiest page: read + 3 write actions + upload)
6. Remaining pages in nav order: `DataOverview.jsx` → `Eda.jsx` → `ModelComparison.jsx` → `Forecasting.jsx` → `MlflowTracking.jsx` → `DriftMonitoring.jsx`
7. Config/tooling: `vite.config.js`, `package.json`, `.oxlintrc.json`

FILE: FRONTEND/INDEX.HTML

Type: Config / Entry Point **Purpose:** The single static HTML page Vite serves; provides the `#root` mount node and loads the app's JS entry as an ES module. **Why Created (Reasoning):** Every Vite + React SPA needs exactly one HTML shell; this is it, with the page `<title>` set to "DemandIQ" and a favicon reference. **Dependencies:** `/src/main.jsx` (loaded as `<script type="module">`), `/favicon.svg` (referenced, not read here). **Key Concepts:** Standard Vite SPA bootstrap — no server-rendered content, everything below `#root` is injected by React at runtime. **Detailed Explanation:** This file never changes at runtime; Vite's dev server and production build both serve it as-is (with asset URLs rewritten at build time). It sets the document language, charset, a responsive viewport meta tag, and the tab title — none of the actual application markup lives here, it's all produced by React once `main.jsx` runs. **Key Code Sections Explained:** `<div id="root"></div>` is the single DOM node React takes over; `<script type="module" src="/src/main.jsx">` is what triggers app bootstrap in the browser. **How To Use/Call This File:** Never edited for feature work; Vite reads it to know the entry point and to inject dev-mode HMR client code. **Important Notes:** No `<noscript>` fallback or meta description — this is an internal tool, not a public marketing page.

FILE: FRONTEND/VITE.CONFIG.JS

Type: Config **Purpose:** Vite build/dev-server configuration; registers the official React plugin (Babel-based Fast Refresh + JSX transform). **Why Created (Reasoning):** Needed so `.jsx` files compile and Hot Module Replacement works during `npm run dev`. **Dependencies:** `vite`, `@vitejs/plugin-react`. **Key Concepts:** Plugin-based build config; no custom aliasing, proxying, or env handling configured — deliberately minimal. **Detailed Explanation:** This is close to the default Vite React template config: one `plugins: [react()]` entry and nothing else. There's no dev-server proxy to the FastAPI backend; instead `api.js` hardcodes the backend's absolute URL (`http://localhost:8000`), so CORS must be enabled on the FastAPI side (confirmed indirectly by the fact the dashboard works from a different origin/port, `5173` vs `8000`). **Key Code Sections Explained:** `defineConfig({ plugins: [react()] })` — the only line of substance; enables JSX/TSX transform and React Refresh. **How To Use/Call This File:** Read automatically by `vite` / `vite build` / `vite preview` (the npm scripts in `package.json`); not imported by app code. **Important Notes:** No path aliases, no build target overrides, no env var exposure config — if the backend host/port ever needs to change per environment, this file and `api.js` would need to introduce `import.meta.env` support, which doesn't exist yet.

FILE: FRONTEND/PACKAGE.JSON

Type: Config **Purpose:** Declares the frontend's npm scripts and dependency versions. **Why Created (Reasoning):** Standard Node/npm project manifest; pins the exact library set the dashboard is built on. **Dependencies:** Runtime: `axios` (HTTP client), `react` / `react-dom` (v19), `react-router-dom` (v7), `recharts` (v3, charting). Dev: `@vitejs/plugin-react`, `vite` (v8), `oxlint` (linter), TypeScript type packages for React (`@types/react*`, used for editor intellisense even though the codebase is plain JSX, not TSX). **Key Concepts:** `"type": "module"` — the package is ESM-only. Scripts: `dev` (vite dev server), `build` (production bundle), `lint` (oxlint), `preview` (serve built output locally). **Detailed Explanation:** No test runner, no CSS framework, no state-management library (Redux/Zustand/etc.) and no data-fetching library (React Query/SWR) are listed — all data fetching is hand-rolled in `useApi.js`, confirming the app intentionally stays dependency-light. `"private": true` prevents accidental `npm publish`. **Key Code Sections Explained:** The `dependencies` vs `devDependencies` split is the standard one — everything needed at runtime in the browser bundle vs. everything only needed to build/lint locally. **How To Use/Call This File:**

`npm install` reads it to fetch dependencies; `npm run dev|build|lint|preview` invoke the corresponding script. **Important Notes:** Uses `oxlint` (a fast Rust-based linter) instead of ESLint — configured further in `.oxlintrc.json`. Version pins use `^` (caret) ranges, so minor/patch updates can drift.

FILE: FRONTEND/.OXLINTRC.JSON

Type: Config **Purpose:** Configuration for the `oxlint` linter run via `npm run lint`. **Why Created (Reasoning):** Enforces React-specific correctness rules cheaply and fast (oxlint is Rust-based, much faster than ESLint) without pulling in the full ESLint + plugin toolchain. **Dependencies:** None (declarative JSON), references its own JSON schema from `node_modules`. **Key Concepts:** Rule configuration only — two rules enabled: `react/rules-of-hooks` (error) and `react/only-export-components` (warn, with `allowConstantExport: true`). **Detailed Explanation:** `react/rules-of-hooks` is the most important rule here — it statically catches violations of React's hooks rules (e.g. calling `useState` / `useEffect` conditionally or in loops), which is exactly the kind of bug that's easy to introduce in the hand-rolled `useApi` pattern this codebase leans on heavily. `react/only-export-components` (warn) nudges toward keeping component files exporting only components (relevant to Fast Refresh reliability), but `allowConstantExport: true` permits the small constant exports seen in files like `Sidebar.jsx` (`LINKS`) or `StatusBadge.jsx` (`MAP`) without warning. **Key Code Sections Explained:** `plugins: ["react", "oxc"]` activates the React-aware and core oxc rule sets that the two configured rules come from. **How To Use/Call This File:** Picked up automatically by `oxlint` when `npm run lint` runs. **Important Notes:** Very minimal config — most rules are left at oxlint's defaults; no Prettier/formatting integration is configured here.

FILE: FRONTEND/SRC/MAIN.JSX

Type: Entry Point **Purpose:** Boots the React application: creates the root, wraps `App` in `StrictMode` and `BrowserRouter`, and mounts it to `#root`. **Why Created (Reasoning):** Required glue between `index.html`'s empty div and the actual component tree; also the single place global CSS (`index.css`) is imported so it applies app-wide. **Dependencies:** `react` (`StrictMode`), `react-dom/client` (`createRoot`), `react-router-dom` (`BrowserRouter`), `./index.css`, `./App.jsx`. **Key Concepts:** React 19's `createRoot` API (replaces legacy `ReactDOM.render`); `StrictMode` for double-invoking renders/effects in dev to surface side-effect bugs; `BrowserRouter` for HTML5 history-based client routing. **Detailed Explanation:** This is the standard Vite React-Router bootstrap: exactly one `createRoot(...).render(...)` call, nothing else. Because `BrowserRouter` wraps `<App/>` here rather than inside `App.jsx`, `App.jsx` itself can freely use routing hooks/components (`Routes`, `Route`, `NavLink` in `Sidebar`) without needing to also own the router setup. **Key Code Sections Explained:** `createRoot(document.getElementById("root")).render(...)` — finds the DOM node from `index.html` and mounts the tree; the JSX tree order (`StrictMode` > `BrowserRouter` > `App`) determines that routing context is available everywhere in `App` and below, while double-render checks from StrictMode apply to the whole tree. **How To Use/Call This File:** Loaded directly by `index.html`'s module script; not imported anywhere else. **Important Notes:** `StrictMode` causes intentional double-invocation of function bodies/effects in development only — irrelevant in production builds, but a reason to expect `useApi`'s effect to fire twice in dev.

FILE: FRONTEND/SRC/APP.JSX

Type: Entry Point / Layout + Router **Purpose:** Defines the app's shell layout (sidebar + main content area) and the client-side route table mapping each URL path to a page component. **Why Created (Reasoning):** Central place to declare "what pages exist and what they're called" — keeps `Sidebar.jsx`'s nav links and this route table as the only two places page paths are defined. **Dependencies:** `react-router-dom` (`Routes`, `Route`), `./components/Sidebar`, and all seven page components from `./pages/*`. **Key Concepts:** Declarative routing (`<Routes><Route path=... element=.../></Routes>`); component composition (layout wraps swappable page content); no lazy-loading/code-splitting — all pages are imported eagerly. **Detailed Explanation:** The layout is a flex row (`.app-shell` in `index.css`): a fixed-width sticky `Sidebar` on the left, and a `<main className="main">` on the right that renders whichever page matches the current route. There are exactly seven routes, one per dashboard section, each a flat top-level path (no nested routes, no route params in the URL — series selection within a page, e.g. Eda/Forecasting, is handled via component state and query-less API paths like `/eda/:sid`, not the URL). **Key Code Sections Explained:** The `<Routes>` block is a straight 1:1 table: `/` → `Overview`, `/data` → `DataOverview`, `/eda` → `Eda`, `/models` → `ModelComparison`, `/forecast` → `Forecasting`, `/mlflow` → `MlflowTracking`, `/drift` → `DriftMonitoring`. Adding a new dashboard page requires exactly two edits: a new `<Route>` here and a new link entry in `Sidebar.jsx`'s `LINKS` array. **How To**

Use/Call This File: Rendered by `main.jsx` inside `BrowserRouter`; not used anywhere else. **Important Notes:** No 404/catch-all route is defined — navigating to an unmatched path renders nothing inside `<main>` (sidebar still shows). No route-level auth/guarding since this is a local dev tool.

FILE: FRONTEND/SRC/API.JS

Type: Utility **Purpose:** Creates and exports the single axios instance every page/action uses to talk to the FastAPI backend. **Why Created (Reasoning):** Centralizes the backend base URL in one place so it isn't duplicated across 7+ page files; also the natural place to add interceptors/auth headers later if ever needed. **Dependencies:** `axios`. **Key Concepts:** Axios instance configuration (`axios.create`) rather than using the global `axios` object directly. **Detailed Explanation:** This is a two-line file: `export const api = axios.create({ baseURL: "http://localhost:8000/api" })`. Every `useApi(path)` call and every direct `api.get/post(...)` call in the page files is relative to this baseURL, so callers only ever specify the path suffix (e.g. `/status`, `/retrain`, `/eda/${sid}`). The base URL is hardcoded to localhost:8000, matching the README's documented dev workflow (`uvicorn dashboard.server:app --port 8000`). **Key Code Sections Explained:** `axios.create({ baseURL: ... })` — the only meaningful call; no default headers, no interceptors, no timeout configured. **How To Use/Call This File:** Imported as `{ api }` by `useApi.js` and by every page that fires a POST/GET action directly (`Overview.jsx`, `DriftMonitoring.jsx`). **Important Notes:** Hardcoded absolute URL means this won't work unmodified against a deployed/non-localhost backend — there's no environment-variable-driven base URL (no `import.meta.env.VITE_API_URL`), a known limitation for anything beyond local dev.

FILE: FRONTEND/SRC/USEAPI.JS

Type: Hook **Purpose:** A single reusable custom hook that GETs a JSON endpoint and exposes `{ data, error, loading, reload }`, refetching when `path` or any value in `deps` changes. **Why Created (Reasoning):** Every page needs the identical loading/error/data lifecycle for its main fetch; this hook eliminates repeating that `useState` x3 + `useEffect` boilerplate in all seven pages, and standardizes error message extraction from axios errors. **Dependencies:** `react` (`useEffect`, `useState`, `useCallback`), `./api` (`api`). **Key Concepts:** Custom hook composition; `useCallback` to memoize the fetch function so `useEffect`'s dependency array is stable; conditional fetching (`path` can be `null`/falsy to skip fetching, used by `Eda.jsx`/`Forecasting.jsx` before a series id is known); manual refetch via a returned `reload` function. **Detailed Explanation:** Internally it holds three pieces of state — `data`, `error`, `loading` — starting as `null`, `null`, `true`. `reload`, built with `useCallback` keyed only on `path` (an intentional `eslint-disable` on the exhaustive-deps rule), performs the actual `api.get(path)` call: sets `loading=true`/`error=null` before the request, `setData` on success, `setError` on failure (preferring `err.response?.data?.detail` — FastAPI's standard error body shape — falling back to `err.message`), and always `setLoading(false)` in `finally`. If `path` is falsy, it skips fetching entirely and just flips `loading` to `false` — this is what lets `Eda`/`Forecasting` call `useApi(active ? ... : null)` before a series id is resolved without triggering a request to `/eda/undefined`. **Key Code Sections Explained:** `useEffect(reload, [reload, ...deps])` is the trigger: because `reload` only changes identity when `path` changes, and `deps` is spread in alongside it, callers can pass extra reactive values (e.g. `[active]` in `Eda.jsx`) to force a refetch even when the `path` string itself is derived from that same value — in practice `path` already encodes the change (e.g. `` `/eda/${active}` ``) so `deps` is mostly redundant/defensive here, but it's the mechanism that would matter if a caller ever passed a static path with an external trigger. **How To Use/Call This File:** Called as `const { data, error, loading, reload } = useApi("/status")` (no deps) or `useApi(active ? \`/forecast/${active}\` : null, [active])` (conditional path + extra dep) in every page component. **Important Notes:** No caching, dedup, or abort-on-unmount logic — a fast-changing `deps` value (e.g. rapid series switching) can theoretically let a stale response arrive after a newer request and overwrite fresher data (no race-guard / AbortController); acceptable for this dashboard's manual, human-paced interaction pattern but not something to import into a busier app without adding request cancellation.

FILE: FRONTEND/SRC/COLORS.JS

Type: Utility **Purpose:** Provides a fixed 8-slot color palette (as CSS variable references) and a `seriesColors(names)` function that deterministically assigns one color per series name. **Why Created (Reasoning):** Charts that show multiple series/models (Model Comparison bars, EDA lines) need consistent, distinguishable colors that don't shift when the underlying list is filtered — assigning by array index alone would repaint colors whenever the visible subset changes. **Dependencies:** None (pure JS); relies on CSS custom properties (`--series-1` .. `--series-8`) defined in `index.css`. **Key Concepts:** Pure/deterministic mapping function; using CSS variables as color values so charts automatically follow the light/dark theme without any JS-side theme logic. **Detailed Explanation:** `PALETTE` is a plain array of 8 `var(--series-N)` strings. `seriesColors(names)` copies and

alphabetically sorts the input array first, then walks the sorted list assigning `PALETTE[i % PALETTE.length]` to each name in a `{name: colorVar}` map — sorting before assigning is the key trick: it guarantees a series named `"Store_1"` always lands in the same palette slot no matter which other series are currently present in `names`, since its alphabetical rank among any subset containing it tends to stay stable (documented directly in the file's own comment: "a filtered chart never repaints survivors"). **Key Code Sections Explained:** `sorted.forEach((n, i) => { map[n] = PALETTE[i % PALETTE.length]; })` — the modulo wraps around if there are more than 8 series, so uniqueness isn't guaranteed beyond 8 distinct series (colors then repeat). **How To Use/Call This File:** `ModelComparison.jsx` calls `seriesColors(avg.map(r => r.model))` once per render to build a model→color lookup used both in the bar chart `<Cell fill=...>` and in the legend swatch in the table. **Important Notes:** Sort is alphabetical/lexicographic on raw strings — not numeric-aware, so `"Store_10"` sorts before `"Store_2"`; not an issue for the current model-name use case but worth remembering if reused for numerically-named series.

FILE: FRONTEND/SRC/METRICINFO.JS

Type: Utility (static config) **Purpose:** Central dictionary of plain-language explanations (`METRIC_INFO`) and short display labels (`METRIC_LABEL`) for every metric shown anywhere in the dashboard. **Why Created (Reasoning):** Metric names from the backend (`mape`, `ks_statistic`, `quality_score`, etc.) are technical; every page that shows a metric needs both a human label and an on-hover explanation, and hardcoding these per-page would duplicate and risk inconsistent wording across pages. **Dependencies:** None — plain exported objects. **Key Concepts:** Data-driven UI text (component code stays generic — `METRIC_LABEL[key] || key` — while all copy lives in one config file). **Detailed Explanation:** `METRIC_INFO` maps ~16 metric keys to one-to-two-sentence explanations written for a non-ML audience (e.g. MAPE explained as "average forecast error as a percent of actual sales. Lower is better."), consumed by `InfoTooltip` to render its hover popover. `METRIC_LABEL` maps a subset of those same keys (plus a couple more, like `rank`, `outlier_pct`) to short display strings used as table headers/metric-card labels (e.g. `r2_score` → `"R²"`, `training_time_seconds` → `"Training time (s)"`). **Key Code Sections Explained:** Both are flat object literals keyed by the exact metric field names returned by the FastAPI backend's JSON responses — this coupling means adding a new metric to a backend response and wanting it labeled/explained in the UI requires adding one key here, nothing else. **How To Use/Call This File:** `InfoTooltip` reads `METRIC_INFO[metricKey]`; `MetricCard` reads `METRIC_LABEL[metricKey]`; `ModelComparison.jsx` and others read both directly for table headers and chart tooltips. **Important Notes:** If a metric key used in a page isn't present in `METRIC_INFO`, `InfoTooltip` simply renders nothing (no tooltip button appears) — a silent, non-breaking fallback rather than showing "undefined".

FILE: FRONTEND/SRC/INDEX.CSS

Type: Styling **Purpose:** The app's entire visual system — CSS custom properties for theming (light + dark), the flex/grid layout primitives, and every reusable class used across components and pages. **Why Created (Reasoning):** With no CSS framework or CSS-in-JS library installed, this hand-written stylesheet is the sole source of visual styling; keeping it as design tokens (CSS variables) lets colors/spacing be reused consistently and lets dark mode be a pure CSS media-query concern with zero JS. **Dependencies:** None (plain CSS); consumed implicitly by every component/page via className strings. **Key Concepts:** CSS custom properties (design tokens) scoped to `:root`; `prefers-color-scheme` media query for automatic dark mode; utility-ish layout classes (`.grid-2/3/4`, `.section`) alongside semantic component classes (`.card`, `.badge`, `.metric-card`). **Detailed Explanation:** The token set covers surfaces/text/borders, 8 chart series colors, and 4 status colors (good/warning/serious/critical) — both defined twice, once for light (`:root`) and once overridden for dark (`@media (prefers-color-scheme: dark) { :root:not([data-theme="light"]) {...} }`), meaning the app respects the OS/browser theme automatically with no manual toggle in the UI. Layout-wise, `.app-shell` is the flex row (sidebar + main), `.sidebar` is a sticky full-height column, and `.grid` / `.grid-2/3/4` are CSS Grid helpers used throughout the pages for stat-tile and chart rows, collapsing to one column under 1000px. Component-level classes cover everything visual: cards, metric tiles, info-icon/tooltip popover, tables, buttons (including a `.btn-primary` variant and disabled state), badges (4 status variants), a legend, empty/error states, a spinner (plus a `.spinner-dark` variant for use on light buttons), and a fixed-position toast — plus three Recharts-specific overrides at the bottom (`.recharts-cartesian-axis-tick text`, `.recharts-cartesian-grid line`, `.recharts-default-tooltip`) that retint Recharts' own generated SVG/DOM elements to match the token palette instead of Recharts' default black/white. **Key Code Sections Explained:** The dark-mode block is the trickiest part — it only applies within a `prefers-color-scheme: dark` media query AND only when `:root` does NOT have `data-theme="light"` set, implying the app was built with room for a manual light/dark override attribute even though no such toggle currently exists in any component (the `data-theme` attribute is never set by app JS today). **How To Use/Call This File:** Imported once, globally, in `main.jsx` (`import`

"./index.css"); every component/page references its classes by string in className. **Important Notes:** Because styling is global classnames (not CSS Modules/scoped), classnames must stay unique and consistently used; there's no build-time guarantee a typo'd classname will be caught — it will simply render unstyled.

FILE: FRONTEND/SRC/COMPONENTS/INFOTOOLTIP.JSX

Type: UI Component **Purpose:** A small circular "(i)" button that reveals a one-line plain-language explanation on hover or keyboard focus. **Why Created (Reasoning):** Metrics like MAPE/KS-statistic/quality_score aren't self-explanatory to a non-ML dashboard user; this gives every metric label an optional inline explainer without cluttering the layout, reusing metricInfo.js copy. **Dependencies:** react (useState), ../metricInfo (METRIC_INFO). **Key Concepts:** Local component state (open) driving conditional rendering; accessibility via onFocus/onBlur in addition to onMouseEnter/onMouseLeave so keyboard users can trigger it too; aria-label and role="tooltip" for screen readers. **Detailed Explanation:** Takes either a metricKey (looked up in METRIC_INFO) or a direct text override (used by pages showing ad-hoc explanations not in the shared dictionary, e.g. ACF/PACF descriptions in Eda.jsx). If neither resolves to content, the component renders null entirely — no empty button appears. Otherwise it renders a <button> with class info-icon (styled as a small bordered circle in index.css) and, when open is true, a absolutely positioned above the button showing the resolved text. **Key Code Sections Explained:** const content = text || METRIC_INFO[metricKey]; — text wins over metricKey lookup, letting callers override the shared copy per-instance. The four event handlers (onMouseEnter/Leave, onFocus/Blur) all just toggle the same open boolean, unifying mouse and keyboard interaction into one state variable. **How To Use/Call This File:** <InfoTooltip metricKey="mape" /> or <InfoTooltip text="custom explanation" />; used directly in pages (DataOverview, ModelComparison, DriftMonitoring, Eda) and indirectly via MetricCard and ChartCard, which forward their own tooltip/metricKey props into an internal InfoTooltip. **Important Notes:** Purely CSS-positioned popover (no portal/positioning library) — could clip or overlap if used near a viewport edge; not an issue in this dashboard's fixed layout widths.

FILE: FRONTEND/SRC/COMPONENTS/METRICCARD.JSX

Type: UI Component **Purpose:** Renders a single stat tile: a label (with optional info tooltip), a large value, and an optional sub-line. **Why Created (Reasoning):** Nearly every page opens with a row of KPI tiles (test MAPE, series count, quality score, etc.); this component avoids re-writing that markup/CSS per page. **Dependencies:** ./InfoTooltip, ../metricInfo (METRIC_LABEL). **Key Concepts:** Prop-driven presentational component with sensible fallbacks (label || METRIC_LABEL[metricKey] || metricKey); nullish coalescing (value ?? "—") to show an em-dash placeholder instead of blank/undefined. **Detailed Explanation:** Accepts metricKey (used both for the label lookup and passed straight through to InfoTooltip), an optional explicit label override, value, an optional sub caption line, and an optional tooltip text override. It wraps everything in a .card.metric-card div matching the shared card styling from index.css. **Key Code Sections Explained:** {label || METRIC_LABEL[metricKey] || metricKey} is a three-level fallback chain: explicit label wins, then the shared dictionary label, then the raw key itself as a last resort — meaning any metric works even if it's missing from metricInfo.js, just without a friendly name. **How To Use/Call This File:** <MetricCard metricKey="test_mape" value={\ ${x.toFixed(2)}%`} /> — used in Overview, DataOverview, ModelComparison (indirectly, via inline metric rows), Forecasting, MlflowTracking. **Important Notes:** value is entirely pre-formatted by the caller (e.g. .toFixed(2) + %` concatenation happens in the page, not here) — this component does no number formatting itself.

FILE: FRONTEND/SRC/COMPONENTS/STATUSBADGE.JSX

Type: UI Component **Purpose:** Renders a colored pill badge for a status string (drift severity, MLflow registry stage, performance status), mapping known values to a color class + icon. **Why Created (Reasoning):** Several different status vocabularies (LOW/MEDIUM/HIGH, OK/DEGRADED, Production/Staging/Archived) all need the same visual treatment (good=green, warning=yellow, critical=red, neutral=grey); one lookup table + component avoids duplicating that logic per status type. **Dependencies:** None beyond React itself (no imports besides implicit JSX runtime). **Key Concepts:** Lookup-table dispatch pattern (const m = MAP[status] || fallback) instead of a chain of if/else or switch statements. **Detailed Explanation:** MAP is a flat object keyed by every known status string across all the vocabularies the app uses, each mapped to { cls, icon }. The component itself is a two-line function: look up MAP[status], fall back to a neutral bullet icon if unrecognized, and render <span className={\ badge ${m.cls}`}>{icon}{status ?? "unknown"}. **Key Code Sections Explained:** The fallback MAP[status] || { cls: "badge-neutral", icon: "•" } means any status value not explicitly listed (typos, new

backend statuses not yet added here) degrades gracefully to a neutral grey badge rather than crashing or rendering blank. **How To Use/Call This File:** — used in Overview (drift status), MlflowTracking (registry stage), DriftMonitoring (drift status). **Important Notes:** Status matching is exact-string and case-sensitive ("Production" vs "production" would not match) — relies on the backend consistently returning one of the exact strings in MAP`.

FILE: FRONTEND/SRC/COMPONENTS/CHARTCARD.JSX

Type: UI Component **Purpose:** A card wrapper around any chart: adds a title, optional info tooltip, and a sized container div (Recharts' `ResponsiveContainer` needs an explicit-height parent). **Why Created (Reasoning):** Every chart in the app (Eda's trend lines, ModelComparison's bar charts, Forecasting's composed chart) needs identical card chrome + a correctly-sized wrapper div; centralizing it avoids repeating that div/height boilerplate per chart. **Dependencies:** `./InfoTooltip`. **Key Concepts:** `children`-as-render-target composition pattern (the chart itself, e.g. a full `<ResponsiveContainer><LineChart>...` tree, is passed in as `children`); a `height` prop with a default (320) since Recharts requires a parent with a concrete pixel height for `ResponsiveContainer` to compute percentage sizing. **Detailed Explanation:** Renders a `.card` div; if a `title` is given, shows a `.section-title` row with the title text and an optional `InfoTooltip` (passed as `tooltip` text); below that, a plain `<div style={{ width: "100%", height }}>` that hosts whatever chart markup was passed as `children`. **Key Code Sections Explained:** `<div style={{ width: "100%", height }}>{children}</div>` — this fixed-height div is what makes Recharts' `<ResponsiveContainer>` (used inside every chart page) resize correctly; without an ancestor with a real height, `ResponsiveContainer` would collapse to 0 height. **How To Use/Call This File:** `<ChartCard title="MAPE (%) — lower is better" tooltip={...} height={340}><ResponsiveContainer>...</ResponsiveContainer></ChartCard>` — used in `Eda`, `ModelComparison`, `Forecasting`. **Important Notes:** If `title` is falsy, no title row renders at all (not even empty space) — used by callers that want a bare chart card.

FILE: FRONTEND/SRC/COMPONENTS/LOADING.JSX

Type: UI Component **Purpose:** Three tiny presentational components covering the app's three non-happy-path states: `Loading` (spinner + "Loading..."), `ErrorBox` (red error message), and `Empty` (grey "nothing here" placeholder with an overridable message). **Why Created (Reasoning):** Every page's data-fetch lifecycle (via `useApi`) needs to render something for loading/error/empty-data cases; consolidating these three into one file keeps that boilerplate a single, consistent import across all seven pages. **Dependencies:** None beyond React (implicit JSX runtime); relies on `.empty-state`, `.error-state`, `.spinner`/`.spinner-dark` classes from `index.css`. **Key Concepts:** Multiple named exports from one file (no default export) — a small violation of the strict "one component per file" convention, justified here because all three are trivial and always used together; children-as-default-content pattern in `Empty` (`children = "Nothing here yet — run the pipeline first."`). **Detailed Explanation:** `Loading` renders a small spinning circle (`.spinner.spinner-dark`, animated via the `@keyframes spin` in index.css) next to "Loading..." text inside an `.empty-state` container. `ErrorBox` takes an `error` prop (any value — page callers pass the string extracted by `useApi`) and stringifies it into a red `.error-state` box prefixed with "Couldn't load this: ". `Empty` accepts optional `children` as the message, defaulting to a generic "run the pipeline first" hint, letting callers customize (e.g. `<Empty>No retraining events yet.</Empty>`). **Key Code Sections Explained:** `export function Loading()`, `export function ErrorBox({ error })`, `export function Empty({ children = "..." })` — three independent named exports, imported together as `import { Loading, ErrorBox, Empty } from "../components/Loading"` in every page. **How To Use/Call This File:** Standard pattern at the top of every page: `if (loading) return <Loading />; if (error) return <ErrorBox error={error} />;`, then further down, wherever a list/table might be empty: `{!list.length ? <Empty>...</Empty> : <table>...}`. **Important Notes:** `ErrorBox` does `String(error)` — since `useApi` already reduces axios errors down to a string message before setting `error` state, this is mostly a no-op safety net, not double-stringification of an object.

FILE: FRONTEND/SRC/COMPONENTS/SIDEBAR.JSX

Type: UI Component **Purpose:** The persistent left-hand navigation panel, listing all seven dashboard sections as `NavLink`s with icons. **Why Created (Reasoning):** Central, single place declaring the human-facing nav (label + emoji icon + path) for every page, kept alongside (but separate from) `App.jsx`'s route table. **Dependencies:** `react-router-dom` (`NavLink`). **Key Concepts:** Data-driven rendering (`LINKS.map(...)` instead of seven hand-written `<NavLink>` elements); `NavLink`'s render-prop `className` function to apply an `"active"` class based on the current route; the `end` prop to prevent the root `/` link from matching every other path as a prefix. **Detailed Explanation:** `LINKS` is an array of `{ to, label, icon, end? }` objects, one per route defined

in `App.jsx` (must stay in sync manually — no shared single source of truth between the two files). Rendering maps over `LINKS`, producing a `<NavLink>` per entry whose `className` prop is a function receiving `{ isActive }` and returning `"active"` or `""`, which `index.css`'s `.sidebar nav a.active` selector then styles (tinted background + colored text). **Key Code Sections Explained:** `end: true` is set only on the `/` (Overview) entry — without it, `NavLink` would treat `/` as a prefix match and mark it active on every route (since every path starts with `/`); all other links don't need `end` since their paths are more specific and don't prefix-collide with each other. **How To Use/Call This File:** Rendered once by `App.jsx` as `<Sidebar />`, outside the `<Routes>` switch, so it persists across all page navigations. **Important Notes:** Adding a page requires updating both `LINKS` here and the `<Route>` list in `App.jsx` — there is no single shared route config, so the two must be kept manually consistent.

FILE: FRONTEND/SRC/PAGES/OVERVIEW.JSX

Type: Page Component **Purpose:** The dashboard's home page (`/`): shows top-line status (production model, test MAPE, drift status, series count), exposes the three primary write actions (retrain, drift check, CSV upload) plus a CSV forecast download, and lists retraining history. **Why Created (Reasoning):** Needed a single "at a glance + take action" landing page so a user doesn't have to hop between pages to trigger the platform's core operational actions (retrain, drift check). **Dependencies:** `react` (`useRef`, `useState`), `../api` (`api`), `../useApi`, `../components/MetricCard`, `../components/StatusBadge`, `../components/Loading` (`Loading`, `ErrorBox`, `Empty`). **Key Concepts:** Two parallel `useApi` calls (`/status`, `/retrain-log`) for read data; local `useState` for three independent async-action-in-flight flags (`retraining`, `checking`) plus a `toast` message; `useRef` to reference the file input (declared but not strictly required given the `onChange` handler already receives the event); manual `reload()` calls after a mutation to refresh dependent GET data; `window.open` for a non-SPA file download. **Detailed Explanation:** On load, it fetches `/status` (drives the four top KPI tiles and the "last trained" line) and `/retrain-log` (drives the history table) via `useApi`. Three actions are wired to buttons: `retrain()` POSTs `/retrain`, shows a toast summarizing the new best model/MAPE/duration, and reloads both `status` and `log`; `checkDrift()` POSTs `/drift-check` and toasts the returned status + recommendation, reloading `status` only; `upload()` reads the selected file from a file input, wraps it in a `FormData`, POSTs `/upload`, and toasts the server's response message. A fourth action, `downloadForecast()`, isn't an API call through axios at all — it just opens `http://localhost:8000/api/forecast` in a new tab, letting the browser handle the CSV download natively. **Key Code Sections Explained:** `showToast = (msg) => { setToast(msg); setTimeout(() => setToast(null), 6000); }` — the entire "toast notification" implementation: no queue, no stacking, a new toast simply replaces whatever's showing and clears itself after 6 seconds. Each action function follows the same try/catch/finally shape: set an in-flight flag true, call the API, toast success or the error's `detail` / `message`, and reset the flag in `finally` regardless of outcome — guaranteeing buttons don't get stuck disabled on failure. **How To Use/Call This File:** Rendered by `App.jsx` at route `/` (also the sidebar's "Overview" link, marked `end: true`). **Important Notes:** Loading/error short-circuit only covers the `/status` fetch (`if (loading) return <Loading/>; if (error) return <ErrorBox/>`) — the retraining log's own loading state is ignored (it just shows `Empty` until data arrives, no dedicated spinner for that second fetch). The file upload has no client-side validation of file type/size beyond the `accept=".csv"` hint on the input; all validation is left to the backend's error response.

FILE: FRONTEND/SRC/PAGES/DATAOVERVIEW.JSX

Type: Page Component **Purpose:** Route `/data` — shows dataset shape (rows, series count, date range), the automated data-quality/validation report, missing-values-by-column, and a raw sample table. **Why Created (Reasoning):** Gives a data-quality first look before a user trusts any downstream model/forecast — surfaces the same validation report the Python pipeline (`src/validation/`) produces. **Dependencies:** `../useApi`, `../components/MetricCard`, `../components/InfoTooltip`, `../components/Loading`. **Key Concepts:** Single `useApi("/data-overview")` call; derived local variables (`report`, `missingEntries`) computed from `data` on every render (no memoization needed given cheap computation); a small local (non-exported) helper component `Row` defined at the bottom of the same file for a label/value flex row, kept file-local since it's only used here. **Detailed Explanation:** Top KPI row shows row count (locale-formatted with `.toLocaleString()`), series count, date range string, and quality score (from the nested `validation_report`). Below that, a two-column grid: missing-values-by-column table (built from `Object.entries(data.missing_pct || {})`, showing `Empty` if there are none) and a validation report summary (schema errors joined into one string or "Valid ✓", duplicate row count, date gap count, negative sales count, and outlier count/percentage with its own `InfoTooltip`). Finally, a raw sample table renders the first 20 rows of `data.sample`, deriving column headers dynamically from `Object.keys(data.sample[0])` so it doesn't need to know the schema in advance. **Key Code Sections Explained:** The local `Row` component (`function Row({ label, value })`) is a tiny label/value flex-justify-between row reused 5 times in the validation report card — kept as a plain unexported function rather than promoted to `components/` since nothing else uses it. The sample table's header generation (`Object.keys(data.sample[0]).map(...)`) means it's entirely

schema-agnostic — works for any CSV columns the backend echoes back. **How To Use/Call This File:** Rendered by `App.jsx` at `/data`. **Important Notes:** If `data.sample` is empty, `data.sample[0]` is `undefined` and the header row's `.map` is skipped via the `&&` guard (`{data.sample[0] && Object.keys(...)}`) — avoids a crash but would also render an empty-looking table with no `Empty` fallback message for that specific case (unlike the missing-values table, which does have one).

FILE: FRONTEND/SRC/PAGES/EDA.JSX

Type: Page Component **Purpose:** Route `/eda` — lets a user pick one series from a dropdown and view its historical trend, seasonal decomposition (trend/seasonal/residual), and ACF/PACF autocorrelation bar charts. **Why Created (Reasoning):** Surfaces the exploratory-analysis outputs (decomposition, autocorrelation) that the Python pipeline's EDA step would otherwise only produce as notebook output — makes them interactively browsable per series. **Dependencies:** `react` (`useState`), `recharts` (`LineChart`, `Line`, `BarChart`, `Bar`, `XAxis`, `YAxis`, `CartesianGrid`, `Tooltip`, `ResponsiveContainer`), `../useApi`, `../components/ChartCard`, `../components/Loading` (`Loading`, `ErrorBox`). **Key Concepts:** Two chained `useApi` calls — one to list available series (`/series`), a second, conditional one for the selected series' EDA payload (`/eda/${active}`, or `null` until a series is resolved); derived "active series" state (`sid || series?.[0]`) so the first series is auto-selected without an extra effect; conditional rendering of chart sections based on whether the backend payload includes `decomposition` / `acf_pacf` (defensive against partial data, e.g. a series too short to decompose). **Detailed Explanation:** `series` (list of series ids) loads first; `active` derives to either the user's manual selection (`sid`) or the first entry of `series` as a default. The second `useApi` call is keyed on `active` both in its path and its `deps` array, so switching the `<select>` triggers a new fetch. Once `data` arrives: a full-width line chart shows `history` (date vs sales); if `data.decomposition` exists, three side-by-side line charts render `trend`, `seasonal`, `residual` (title-cased from the object key, `connectNulls` set since decomposition series often have leading/trailing NaNs); if `data.acf_pacf` exists, two bar charts render ACF and PACF values, converting each raw array of numbers into `{lag: i, value: v}` objects since Recharts needs objects, not primitives. **Key Code Sections Explained:** `const active = sid || series?.[0];` — this is the "auto-select first item" pattern used again in `Forecasting.jsx`; it means the select's initial render (before user interaction) already has a real series and immediately fires the EDA fetch. `data.acf_pacf[k].map((v, i) => ({ lag: i, value: v }))` is a one-line data-shape adapter turning the backend's plain number array into Recharts-compatible records. **How To Use/Call This File:** Rendered by `App.jsx` at `/eda`. **Important Notes:** Loading/error states here are rendered inline (`{loading && <Loading/>}` / `{error && <ErrorBox/>}`) rather than as early returns, because the `<select>` dropdown (built from the separately-loaded `series` list) needs to keep rendering even while the second, series-specific fetch is in flight — an early return would hide the dropdown during every series switch.

FILE: FRONTEND/SRC/PAGES/MODELCOMPARISON.JSX

Type: Page Component **Purpose:** Route `/models` — a table of every trained model's average CV metrics (with rank) plus four bar charts comparing MAPE, RMSE, training time, and inference time across models. **Why Created (Reasoning):** Directly surfaces the pipeline's core "7 models on identical folds" comparison (per README) so a user can see why one model was promoted over the others. **Dependencies:** `recharts` (`BarChart`, `Bar`, `XAxis`, `YAxis`, `CartesianGrid`, `Tooltip`, `ResponsiveContainer`, `Cell`, `LabelList`), `../useApi`, `../components/ChartCard`, `../components/InfoTooltip`, `../components/Loading` (`Loading`, `ErrorBox`), `../metricInfo` (`METRIC_INFO`, `METRIC_LABEL`), `../colors` (`seriesColors`). **Key Concepts:** A local factory function (`bar(metricKey, title)`) that generates a chart-card JSX block for any metric column, avoiding four near-identical copy-pasted chart blocks; per-model consistent coloring via `seriesColors`; Recharts' `<Cell>` for per-bar coloring and `<LabelList>` for on-bar value labels. **Detailed Explanation:** `useApi("/model-comparison")` returns `{ average: [...] }`, one row per model with all metric columns plus a `rank`. `seriesColors(avg.map(r => r.model))` builds a stable model→color map used both for bar `<Cell>` fills and for the small colored swatch preceding each model name in the table. The `bar(metricKey, title)` helper is called four times (MAPE, RMSE, training time, inference time) each producing a `<ChartCard>` wrapping a Recharts `<BarChart>` with value labels rendered directly above each bar (`LabelList`) and per-bar coloring via mapped `<Cell>` elements. The metrics table above the charts iterates a fixed `METRIC_COLS` array (`mape, rmse, mae, r2_score, smape, training_time_seconds, inference_time_ms`) to build both the header row (with an `InfoTooltip` per column) and each data row. **Key Code Sections Explained:** `const bar = (metricKey, title) => (<ChartCard .../>)` — a function returning JSX, not a separate component; deliberately kept as a closure inside the page component (not hoisted to `components/`) since it captures `avg` and `colors` from the enclosing scope rather than taking them as explicit props. `{avg.map((r) => <Cell key={r.model} fill={colors[r.model]} />)}` inside `<Bar>` is the Recharts idiom for per-datapoint

coloring (as opposed to one uniform `fill` on `<Bar>` itself). **How To Use/Call This File:** Rendered by `App.jsx` at `/models`. **Important Notes:** `METRIC_COLS` is a fixed list hardcoded in this file (not derived from the API response), so if the backend adds a new metric to `/model-comparison`, this file must be edited to surface it in the table.

FILE: FRONTEND/SRC/PAGES/FORECASTING.JSX

Type: Page Component **Purpose:** Route `/forecast` — shows the winning model's test metrics, its top-15 feature importances, and per-series history + 30-day forecast (with a 95% confidence band) plus a forecast data table and CSV download. **Why Created (Reasoning):** The primary "so what will actually happen" page of the dashboard — turns the pipeline's forecast API output into a chart non-technical stakeholders can read. **Dependencies:** `react` (`useState`), `recharts` (`ComposedChart`, `Line`, `Area`, `XAxis`, `YAxis`, `CartesianGrid`, `Tooltip`, `ResponsiveContainer`, `BarChart`, `Bar`, `Legend`), `../useApi`, `../components/ChartCard`, `../components/MetricCard`, `../components/Loading` (`Loading`, `ErrorBox`, `Empty`). **Key Concepts:** An optimization to avoid a redundant network request: the first series' history+forecast is already embedded in the initial `/forecast` payload (`fc.default_series`), so `useApi` for a specific series (`/forecast/${active}`) is only triggered when `active` differs from that default; `ComposedChart` mixing an `Area` (confidence band) with two `Line`s (actual vs forecast) on one chart; data reshaping to merge two separate arrays (`history`, `forecast`) into one array Recharts can plot on a shared X axis. **Detailed Explanation:** The page first fetches `/forecast`, which returns overall `best_model`, `test_metrics`, `feature_importance`, the list of `series`, and a `default_series` (history+forecast for `series[0]`, precomputed server-side to save a round trip). `active` is either the user-selected `sid` or `fc.series[0]`. `seriesFetch` is a second `useApi` call whose path is `null` (skip fetch) unless the user has picked a series other than the default — this is the "avoid redundant fetch" optimization. `view` then resolves to either `fc.default_series` (when viewing the default) or `seriesFetch.data` (otherwise). `merged` builds the ComposedChart's single dataset by mapping `view.history` into `{date, actual}` records and `view.forecast` into `{date, forecast, band: [lower, upper]}` records, concatenated into one array — Recharts plots `actual` and `forecast` as separate lines that naturally don't overlap in date range, and `band` as a two-value array `Area` (min/max range fill) only present on the forecast portion. **Key Code Sections Explained:** `const seriesFetch = useApi(active && fc && active !== fc.series[0] ? \` /forecast/${active}` : null, [active]);` `is the key conditional-fetch line — three conditions must all hold to actually fetch: an active series is selected, the main forecast payload has loaded, and that active series isn't the one already embedded as` default_series`. The` merged `array's` band: [f.lower, f.upper] `is Recharts' documented way of feeding a min/max pair per point to draw a shaded range rather than a single line. **How To Use/Call This File:** Rendered by` App.jsx `at` /forecast`. **Important Notes:**` While seriesFetch `is in flight (after switching to a non-default series),` view `becomes` undefined `momentarily since` seriesFetch.data `starts` null `— handled by around the chart+table section, so switching series shows a brief loading state exactly where the chart was, while the top KPI/feature-importance sections (which don't depend on` view`) stay rendered continuously.

FILE: FRONTEND/SRC/PAGES/MLFLOWTRACKING.JSX

Type: Page Component **Purpose:** Route `/mlflow` — shows the current best MLflow run, the model registry's version history (with stage badges), and a raw table of all experiment runs. **Why Created (Reasoning):** Exposes MLflow tracking/registry data (per README's "every fold logged... best model registered, promoted to Production") inside the same dashboard rather than requiring a separate `mlflow ui` tab for a quick glance. **Dependencies:** `../useApi`, `../components/MetricCard`, `../components/StatusBadge`, `../components/Loading` (`Loading`, `ErrorBox`, `Empty`). **Key Concepts:** Single `useApi("/mlflow")` call; schema-agnostic table rendering for the runs list (deriving headers from `Object.keys(data.runs[0])`, same pattern as `DataOverview`'s sample table); numeric-vs-other formatting inline (`typeof v === "number" ? v.toFixed(3) : (v ?? "—")`). **Detailed Explanation:** `best` (`data.best_run`) drives a top KPI row (model name, avg CV MAPE, test MAPE) and, if a registry version exists, a one-line "registered as version N — stage: " card. Below that, a registry-versions table (sorted descending by version number client-side: `[...data.versions].sort((a,b) => b.version - a.version)`) shows version, model type, test MAPE, stage badge, and a locale-formatted created timestamp. A final table dumps every experiment run's raw fields generically — this table's shape is entirely driven by whatever keys the first run object has, so it automatically adapts if the backend adds/removes logged params or metrics. **Key Code Sections Explained:** `[...data.versions].sort((a, b) => b.version - a.version)` — spreads into a new array before sorting to avoid mutating the state array in place (a good practice since `data` is React state from `useApi`). The runs table's cell renderer, `typeof v === "number" ? v.toFixed(3) : (v ?? "—")`, is what makes the fully-generic table still look reasonably formatted rather than showing raw floats with 15 decimal digits. **How To Use/Call This File:** Rendered by `App.jsx` at `/mlflow`. **Important Notes:** The

footer line pointing to `mlflow ui --backend-store-uri sqlite:///mlflow.db` (port 5000) is a static hint, not a live link into an embedded iframe (unlike `DriftMonitoring`'s Evidently iframe) — the full MLflow UI is intentionally left as a separate tool, not embedded.

FILE: FRONTEND/SRC/PAGES/DRIFTMONITORING.JSX

Type: Page Component **Purpose:** Route `/drift` — shows the latest drift report (overall status, per-feature KS-test results), an optional embedded Evidently HTML report, an on-demand model performance/degradation check, and retraining history. **Why Created (Reasoning):** Surfaces the pipeline's drift-detection and performance-degradation monitoring (per README: KS test on top features, MAPE degradation alerting, automated retraining triggers) as an actionable page rather than a background-only mechanism. **Dependencies:** `react` (`useState`), `../api` (`api`), `../useApi`, `../components/StatusBadge`, `../components/InfoTooltip`, `../components/Loading` (`Loading`, `ErrorBox`, `Empty`). **Key Concepts:** Two parallel `useApi` reads (`/drift`, `/retrain-log`); a third data source (`/performance-check`) fetched imperatively via `api.get` (not `useApi`) inside a button click handler, with manual `useState` for its result (`perf`) and in-flight flag (`checking`); a toggleable embedded `<iframe>` for the Evidently HTML report, only mounted in the DOM when `showReport` is true (lazy-mount pattern — avoids loading the iframe's content until requested). **Detailed Explanation:** The main drift report (`drift` from `useApi("/drift")`) drives an overall-status card (status badge + recommendation + drifted-feature count + last-checked timestamp) and a per-feature KS-test table (feature name, reference/current means, KS statistic, p-value — each numeric column paired with an `InfoTooltip` explaining it — and a stable/drift badge per row). `runPerfCheck()` is a manual click-triggered `GET /performance-check` (not wrapped by `useApi` since it's an on-demand check, not something to auto-fetch on mount); its result renders one of three badge variants depending on `perf.status` (`DEGRADED` critical, `OK` good, anything else neutral), each interpolating `recent_mape` / `baseline_mape` / `degradation_pct`. The Evidently report toggle button flips `showReport`; only when true does the `<iframe src="http://localhost:8000/api/drift/html">` get rendered, meaning the (potentially large) HTML report is never fetched until the user explicitly asks to see it. **Key Code Sections Explained:** `if (!drift?.status) return <Empty>No drift report yet — run a drift check from the Overview page.</Empty>;` — a distinct guard beyond the usual loading/error checks: even after a successful, non-error fetch, if the backend simply has no drift report yet (e.g. pipeline never ran a drift check), the page shows a targeted empty-state message pointing the user to the Overview page's "Check drift" action rather than showing a confusingly empty table. The `runPerfCheck` handler catches failures into the same `perf` state shape (`{ status: "ERROR", message: ... }`) rather than a separate error state, so the same conditional-rendering block below handles both success and failure outcomes uniformly (falling into the neutral-badge branch for unrecognized statuses like `"ERROR"`). **How To Use/Call This File:** Rendered by `App.jsx` at `/drift`; cross-references the "Overview" page's drift-check action in its own empty-state copy. **Important Notes:** The Evidently iframe points at a same-origin-adjacent absolute URL (`localhost:8000`, different port than the frontend's `5173`) — relies on the backend serving that HTML report with permissive framing/CORS; if the Evidently report was never generated server-side, the iframe will simply show whatever error page the backend returns for that route (no client-side check that the report exists before rendering the iframe).

Group 9: Test Suite

OVERVIEW

DemandIQ's `tests/` directory follows a one-file-per-`src/`-concern layout: `test_cv.py` tests `src/training/cross_validator.py`, `test_features.py` tests `src/features/feature_engineering.py`, `test_metrics.py` tests `src/evaluation/metrics.py`, and `test_validation.py` tests `src/validation/validator.py`. `tests/__init__.py` is empty — it exists only so `tests` is importable as a package (no shared fixtures or config live there; there is no `conftest.py`).

There is no test runner magic beyond plain pytest, no shared fixtures, no mocking framework, and no parametrization anywhere in these four files — each test builds its own small DataFrame inline with a module-level `_df()` helper and asserts directly on outputs. This is a narrow, surgical test suite that targets the four correctness properties the README calls out as most load-bearing: (1) MAPE/SMAPE/evaluate metric math, (2) leakage-safety of engineered features, (3) walk-forward CV fold boundaries never overlapping or peeking into the future, and (4) schema/outlier validation logic.

What's covered: `src/evaluation/metrics.py`, `src/features/feature_engineering.py` (partially — only `build_features` / `feature_columns`, not the holiday/business/hierarchical helpers individually), `src/training/cross_validator.py` (only `make_folds`, not the orchestrator), `src/validation/validator.py` (only `validate_schema` and `detect_outliers`, not `quality_metrics` or `run_validation`).

What's untested (explicit gaps): There is no test file at all for `src/models/` (base_model, baseline_models, tree_models, statistical_models, lstm_model, model_factory — none of the 7 models have a unit test), `src/mlflow_utils/` (tracking/registry/promotion logic is untested), `src/monitoring/` (KS-test drift detection and performance-degradation alerting are untested), `src/retraining/` (trigger checks, retraining pipeline, event log are untested), `src/api/` (forecast generation, recursive/direct multi-step logic is untested), `src/data_loading/` (Kaggle downloader, loader, synthetic fallback are untested), and `src/preprocessing/` (frequency detection, gap reindexing, imputation are untested). `src/training/train_pipeline.py` (the orchestrator that calls `make_folds` and drives training) is also untested — only the fold-splitting primitive is covered. Given the README's emphasis on MLflow promotion gating (">2% test MAPE improvement") and drift-triggered retraining, these are the highest-value gaps if the suite were to be expanded.

Suggested reading order: `test_metrics.py` (simplest, no DataFrame fixtures, pure functions) → `test_validation.py` (introduces DataFrame-based testing, still single-concern) → `test_cv.py` (introduces the `Fold` dataclass and boolean-mask reasoning) → `test_features.py` (most involved — depends on understanding lag/rolling semantics and leakage rules established by the other three).

FILE: TESTS/INIT.PY

Type: Test **Purpose:** Empty package marker for the `tests` directory; makes `tests` importable as a Python package so `python -m pytest tests` and `from tests.x import y`-style imports resolve consistently. **Why Created (Reasoning):** Not a behavioral test — it exists purely for Python packaging/import mechanics. Its presence guards against import-path ambiguity between `tests/test_*.py` files run directly vs. via package-qualified imports. **Dependencies:** None. **Key Concepts:** Python package `__init__.py` convention. **Detailed Explanation:** The file is completely empty. It contains no fixtures, no `conftest`-style setup, no shared constants. All four test files are self-contained and import directly from `src.*`, so this file does no real work beyond making `tests` a discoverable package. **Key Code Sections Explained:** N/A — no code. **How To Use/Call This File:** Not invoked directly; pytest discovers it as part of collecting the `tests` package. **Important Notes:** No `conftest.py` exists anywhere in the suite — every test file duplicates its own tiny `_df()` builder rather than sharing fixtures. That's a minor duplication but keeps each test file fully self-contained and easy to read in isolation.

FILE: TESTS/TEST_CV.PY

Type: Test **Purpose:** Tests `src/training/cross_validator.py`'s `make_folds()` function, verifying the walk-forward cross-validation split produces correctly ordered, non-overlapping train/validation/test windows. **Why Created (Reasoning):** This test exists to guard against the single most dangerous bug class in time-series ML: temporal leakage in cross-validation splits. If `make_folds` ever let a validation window overlap training data, or let training data creep past the validation window's start, every downstream metric (MAPE, model comparison, MLflow-registered "best model") would be silently invalid despite looking correct. This test is the CV-side counterpart to `test_features.py`'s feature-side leakage checks — together they are the project's leakage-safety net. **Dependencies:** Imports `make_folds` from `src.training.cross_validator`. Exercises the `Fold` dataclass (`fold`, `train_mask`, `val_mask` fields) indirectly through its return value. Uses `pandas` to build a synthetic 200-row daily date series with a `sales` column. **Key Concepts:** Plain `assert` statements (no pytest-specific fixtures or parametrize decorators). Boolean-mask arithmetic on pandas Series (`&`, `.any()`, `.sum()`). A local `_df(n=200)` helper function (not a pytest fixture) constructs the test data. **Detailed Explanation:** `test_walk_forward_order_and_no_overlap` builds a 200-day synthetic series and calls `make_folds(df, n_folds=3)`. For each of the 3 returned folds it asserts: the maximum date in the training mask is strictly before the minimum date in the validation mask (temporal order — no future data in training); the train and validation masks never overlap (`(train_mask & val_mask).any()` is False); and the validation mask never touches the held-out test mask. It also asserts that training set sizes grow monotonically across folds (expanding-window CV) and that the last fold has strictly more training rows than the first.

`test_test_window_is_last_10pct` checks that the reserved test window (returned as `test_mask` alongside the folds) contains roughly 10% of the data (between 15 and 25 rows out of 200) and that every date in the test window is strictly after every date in the non-test window — i.e., the test set is a clean, untouched final holdout that no fold's validation window can leak into.

Together these two tests fully pin down the fold-generation contract described in the module's own docstring (expanding train from 0%, sliding validation window, fixed final 10% test region) without needing to inspect the fraction-based arithmetic (`VAL_FRAC`, `TEST_FRAC`, `STEP_FRAC` constants) directly — the tests are black-box and would still pass if those constants were retuned, as long as the invariants (order, disjointness, expansion, ~10% test) hold. **Key Code Sections Explained:** - `test_walk_forward_order_and_no_overlap`: verifies temporal ordering (train < val), mask disjointness (train ∩ val = ∅, val ∩ test = ∅), and expanding-window growth across the 3 folds. - `test_test_window_is_last_10pct`: verifies the test window size (~10% of rows) and that it is chronologically entirely after the rest of the data. **How To Use/Call This File:** `python -m pytest tests/test_cv.py -v` **Important Notes:** Only `n_folds=3` is exercised — there's no test for `n_folds=1` (edge case) or very small `n` where fold boundaries could collapse or produce empty windows. The test also doesn't verify behavior when the input DataFrame has multiple `series_id` values with different date ranges (the docstring claims "every series splits at the same dates" via a global date index, but with a single-series DataFrame this multi-series behavior is untested). No test exists for `train_pipeline.py`, which is what actually consumes these folds during training.

FILE: TESTS/TEST_FEATURES.PY

Type: Test **Purpose:** Tests `src/features/feature_engineering.py`'s `build_features()` and `feature_columns()` functions, verifying that lag and rolling-window features are computed correctly and, critically, that no feature leaks information from the current or future rows. **Why Created (Reasoning):** This file exists specifically to prove no target leakage — a core architectural claim stated explicitly in the README ("Leakage-safe features … all target-derived features use strictly-past values"). Feature leakage is an insidious bug: a model trained on leaky features can show excellent validation MAPE while being useless in production, because at inference time the "future" values the leaky feature secretly used won't exist yet. This test operationalizes that architectural promise into a checkable assertion instead of leaving it as a documentation claim. **Dependencies:** Imports `build_features` and `feature_columns` from `src.features.feature_engineering`. Exercises `add_lags`, `add_rolling`, `add_temporal`, `add_decomposition`, `add_business`, `add_hierarchical` indirectly (all are called inside `build_features`). Uses `numpy` and `pandas` to build a synthetic two-series (`A`, `B`) dataset with a strictly increasing `sales` sequence (0..99), which makes lag/rolling arithmetic easy to hand-verify. **Key Concepts:** Module-level config dict `CFG` (not a pytest fixture) reused across all three test functions. A local `_df(n=100)` helper builds the synthetic data. Plain `assert` on numpy array equality (`.to_numpy()` comparison) and on scalar values. No parametrize, no mocking. **Detailed Explanation:** `test_lag_is_exactly_previous_value` builds features for series with `sales = 0..99` and checks that `lag_1` at row `t` exactly equals `sales` at row `t-1`, for series "A" specifically (sorted by date first, since `build_features` internally re-sorts by `series_id, date`). This directly validates the shift-based lag implementation (`add_lags` uses `groupby(SERIES_COL)[TARGET_COL].shift(k)`).

`test_rolling_mean_excludes_current_row` checks a specific hand-computed value: for series "A" with `sales = 0..99`, the last row's `rolling_mean_7` must equal 95 — i.e., the mean of values at indices 92 through 98 (the 7 values strictly before index 99), never including index 99 itself. This is the key leakage assertion: it proves `add_rolling`'s `shift(1)` before the rolling window is applied correctly, so a 7-day rolling mean at time `t` never includes `sales[t]`.

`test_no_nans_in_feature_matrix` calls `build_features` and asserts that none of the columns returned by `feature_columns(feats)` contain any NaN values, confirming that the warm-up-row-dropping (`dropna(subset=[f"lag_{max_lag}"])`) plus the final `fillna(0)` in `build_features` leaves a fully dense, NaN-free matrix ready for model training — a practical requirement since most of the 7 models (especially the linear/tree/LSTM models) don't tolerate NaN inputs natively. **Key Code Sections Explained:** - `test_lag_is_exactly_previous_value`: verifies `lag_1` equals the prior row's raw `sales` value, per series. - `test_rolling_mean_excludes_current_row`: verifies `rolling_mean_7` at the final row equals the mean of the 7 rows immediately before it, not including the current row — the core no-leakage assertion. - `test_no_nans_in_feature_matrix`: verifies the final feature matrix (as defined by `feature_columns`) has zero NaNs after warm-up-row dropping and fill. **How To Use/Call This File:** `python -m pytest tests/test_features.py -v` **Important Notes:** The config (`CFG`) enables temporal, decomposition, business, and hierarchical features, but no test independently checks `add_temporal`, `add_decomposition`, `add_business`, or `add_hierarchical` in isolation — only the lag and rolling-mean pieces

get direct numeric verification, and NaN-freedom is checked only in aggregate. There's no leakage test for `rolling_std/min/max`, `trend_component` / `seasonal_component` / `residual_component` (decomposition), or `store_avg_sales` / `product_avg_sales` (hierarchical) — these all use the same `shift(1)`-before-aggregation pattern but aren't individually asserted, so a leakage regression introduced only in those helpers wouldn't be caught. `add_business`'s `sell_price`-dependent columns (`price_norm`, `price_change_pct`) also aren't exercised since the test DataFrame has no `sell_price` column. The holiday-feature try/except fallback path (when the `holidays` package errors) is untested too.

FILE: TESTS/TEST_METRICS.PY

Type: Test **Purpose:** Tests `src/evaluation/metrics.py`'s `mape()`, `smape()`, and `evaluate()` functions, verifying the core forecast-accuracy metric calculations, especially edge-case handling around zero actuals. **Why Created (Reasoning):** MAPE is called out in the README as the platform's primary metric and the one used to gate MLflow model promotion ("promoted to Production only when it beats the incumbent by >2% test MAPE"). A subtly wrong MAPE implementation (e.g., not skipping zero-actual rows, causing division by zero or artificially inflated error) would silently corrupt every model comparison, every promotion decision, and every dashboard number in the system. This test exists to lock down that one arithmetic contract precisely, including its documented zero-handling special case. **Dependencies:** Imports `evaluate`, `mape`, `smape` from `src.evaluation.metrics`. Uses `numpy` arrays as input; `evaluate()` internally depends on `sklearn.metrics` (`mean_absolute_error`, `mean_squared_error`, `r2_score`), which this test indirectly verifies via the perfect-fit assertion. **Key Concepts:** Plain `assert` with floating-point tolerance (`abs(... - expected) < 1e-9`) for the MAPE test — appropriate since MAPE involves float division. No fixtures, no parametrize. **Detailed Explanation:** `test_mape_skips_zeros` passes `y_true = [0, 100, 200]` and checks that the zero-actual row is excluded from the calculation: only the two nonzero rows (100→110, a 10% error, and 200→190, a 5% error) are averaged, giving `(0.10 + 0.05) / 2 = 0.075`. This directly validates the `mask = y_true != 0` logic in `mape()`, which prevents a division-by-zero (or infinite percentage error) when actual demand is zero for a given day/series — a common real-world occurrence in demand data.

`test_smape_zero_both` checks the degenerate case where both actual and predicted are all zero — SMAPE should return exactly `0.0` rather than `NaN` from a 0/0 division, validating the `np.where(denom == 0, 0.0, ...)` guard in `smape()`.

`test_evaluate_keys_and_perfect_fit` checks two things about `evaluate()`: that its return dict has exactly the five expected keys (`mae`, `rmse`, `r2_score`, `mape`, `smape`), and that for a perfect prediction (`y_pred == y_true`), `mape == 0`, `rmse == 0`, and `r2_score == 1` — the three metrics whose "no error" value is unambiguous and easy to assert exactly. `mae` and `smape` aren't asserted for the perfect-fit case, presumably because they're trivially implied by the other checks or considered lower-risk given they're direct wrappers/analogues. **Key Code Sections Explained:** - `test_mape_skips_zeros`: verifies zero-actual rows are excluded from the MAPE average rather than causing a divide-by-zero or corrupting the result. - `test_smape_zero_both`: verifies SMAPE returns 0.0 (not NaN) when both actual and predicted are zero. - `test_evaluate_keys_and_perfect_fit`: verifies `evaluate()`'s output dict shape (exact key set) and correctness for the trivial perfect-prediction case. **How To Use/Call This File:** `python -m pytest tests/test_metrics.py -v` **Important Notes:** No test covers the `mape()` all-zero-actuals case (where `mask.any()` is False and the function should return `float("nan")`) — that's a documented branch in the source (`if not mask.any(): return float("nan")`) with no corresponding test. There's also no test verifying `evaluate()`'s `mape` / `smape` are returned as 0–100 percentages (rounded to 4 decimals) rather than 0–1 fractions, even though the source multiplies by 100 and rounds — the perfect-fit case (0 either way) doesn't distinguish scale. No test exercises `mae` / `r2_score` / `smape` values for a non-trivial (imperfect) prediction, so partial-credit correctness of those three metrics is unverified.

FILE: TESTS/TEST_VALIDATION.PY

Type: Test **Purpose:** Tests `src/validation/validator.py`'s `validate_schema()` and `detect_outliers()` functions, verifying that malformed input data is correctly flagged and that statistical outliers in the sales/target column are correctly identified. **Why Created (Reasoning):** This is the first line of defense in the pipeline (data validation → feature engineering → training, per the README's lifecycle diagram): if bad schema or extreme outliers pass through silently, every downstream stage (feature engineering, model training, drift detection) operates on corrupted assumptions. This test guards against schema-check regressions (e.g., failing to catch a renamed or wrong-typed column) and against the outlier detector failing to flag genuine anomalies (or over-/under-flagging, given the IQR method's real-vs-noise tradeoff called out in the source comment "demand

spikes are often real"). **Dependencies:** Imports `detect_outliers` and `validate_schema` from `src.validation.validator`. Uses `pandas` to build a synthetic 50-row single-series DataFrame with one deliberately injected spike (`10000.0` vs. a baseline of `100.0`). **Key Concepts:** Plain `assert`, list/string containment checks (`any("sales" in e for e in ...)`) to test error-message content without over-constraining exact wording. A local `_df()` helper (no `n` parameter, fixed at 50 rows) builds the shared test data. **Detailed Explanation:** `test_schema_valid` confirms that a well-formed DataFrame (correct `date` / `series_id` / `sales` columns and dtypes) produces an empty error list from `validate_schema` — the "happy path" baseline.

`test_schema_catches_missing_and_wrong_type` covers two negative cases in one test: renaming `sales` to `qty` (a missing required column) must produce an error message containing the string "sales"; and converting the `date` column to `str` dtype (wrong type) must produce an error message containing the string "datetime". Both assertions check message content loosely (substring match via `any(... in e for e in errors)`) rather than exact string equality, keeping the test resilient to minor wording changes while still confirming the correct *column* and *reason* are being reported.

`test_outlier_detected` builds a series of 49 rows at `100.0` plus one final row at `10000.0`, calls `detect_outliers` (default `method="iqr"`), and asserts exactly one outlier is found and that it is the last row (`mask.iloc[-1]` is True). This validates the IQR-based logic (`outside [Q1 - 3*IQR, Q3 + 3*IQR]`) correctly isolates the single injected spike without false-flagging any of the 49 constant-value rows (whose IQR is 0, so any deviation would technically be "outside" — the test confirms the constant baseline rows are stable and don't self-trigger). **Key Code Sections Explained:** - `test_schema_valid`: verifies a correctly-typed DataFrame produces zero schema errors. - `test_schema_catches_missing_and_wrong_type`: verifies a missing column and a wrong dtype each produce a descriptive error naming the offending column/reason. - `test_outlier_detected`: verifies the IQR method flags exactly the one injected spike row and no others. **How To Use/Call This File:** `python -m pytest tests/test_validation.py -v` **Important Notes:** Only the default `method="iqr"` path of `detect_outliers` is tested — the alternative `method="z"` (z-score, `|z| > z_thresh`) branch has no test coverage at all. `quality_metrics()` (missing %, duplicate rows, negative sales, date-gap counting) and `run_validation()` (the full pipeline including the quality-score formula and JSON report writing) are both completely untested by this file — only the two lower-level building blocks are covered. There's also no test for multi-series behavior in `detect_outliers` (the synthetic DataFrame uses a single `series_id`), even though the function groups by `series_id` and computes per-series IQR bounds.

Part 4: Implementation Flow — Building DemandIQ From Scratch

This is the order a developer would sensibly build this system in, given the dependency chain uncovered in Part 3 (each step only depends on what came before it).

Step 1: Project Skeleton & Configuration

What: Establish the package layout, canonical paths, config schema, logging, and serialization helpers before any pipeline logic exists. **Files Created:** `requirements.txt`, `setup.py`, `.env.example`, `src/__init__.py`, `src/constants.py`, `config.yaml`, `src/utils/config_loader.py`, `src/logger.py`, `src/utils/serialization.py` **Why:** Every later module imports `src.constants` for paths/column names, `src.logger.get_logger` for logging, and reads `config.yaml` through `load_config()`. Building this first means every subsequent stage has somewhere to write its artifacts and a validated config dict to read from, with zero rework later. **Technologies Introduced:** Python packaging (`setuptools`), PyYAML, stdlib `logging` / `pickle` / `json`.

Step 2: Data Acquisition

What: Get raw data onto disk and into the canonical long format (`date, series_id, item_id, store_id, sales, sell_price, promotion_flag`), regardless of source. **Files Created:** `src/data_loading/__init__.py`, `src/data_loading/downloader.py`, `src/data_loading/loader.py` **Why:** Nothing downstream can be built or tested without a data source. Building the Kaggle→synthetic fallback first (per ARCHITECTURE.md's "Synthetic fallback" decision) means every later stage can be developed and tested without needing real Kaggle credentials. **Technologies Introduced:** kagglehub, pandas, numpy (synthetic generator).

Step 3: Data Validation

What: Schema + data-quality gate before anything trusts the loaded data. **Files Created:** `src/validation/__init__.py`, `src/validation/validator.py` **Why:** A fail-fast gate here (per ARCHITECTURE.md's pipeline diagram: `load → validate → clean`) prevents silently training on malformed data. Building it right after loading means the rest of the pipeline can assume clean, schema-correct input. **Technologies Introduced:** None new — pure pandas/numpy logic plus the `save_json` helper from Step 1.

Step 4: Preprocessing

What: Per-series frequency detection, gap reindexing to a complete date range, and imputation. **Files Created:** `src/preprocessing/__init__.py`, `src/preprocessing/preprocessor.py` **Why:** Feature engineering (lags, rolling windows) assumes a contiguous per-series date index — this stage guarantees that invariant, producing `data/processed/clean.parquet`. **Technologies Introduced:** Parquet I/O (pyarrow, implicit via pandas).

Step 5: Feature Engineering

What: Build the ~40-column, leakage-safe feature matrix (temporal, lag, rolling, decomposition, business, hierarchical). **Files Created:** `src/features/__init__.py`, `src/features/feature_engineering.py`, `tests/test_features.py` **Why:** This is the model-input contract every one of the 7 models trains against. The leakage-safety test is written alongside the feature code (not

after) because it's the platform's core correctness claim — proving `shift(1)` runs before every rolling/expanding aggregate. **Technologies Introduced:** `holidays` (optional), numpy.

Step 6: Model Abstraction & Implementations

What: A `BaseModel` interface, then all 7 concrete model families behind it. **Files Created:** `src/models/__init__.py`, `src/models/base_model.py`, `src/models/baseline_models.py`, `src/models/statistical_models.py`, `src/models/tree_models.py`, `src/models/lstm_model.py`, `src/models/model_factory.py` **Why:** Building the abstract contract first (Strategy pattern), then baselines (simplest), then statistical, tree, and LSTM (most complex) lets each new model family be added and tested independently without touching training/evaluation code. The factory is built last since it depends on every concrete class existing. **Technologies Introduced:** scikit-learn, LightGBM, XGBoost, statsmodels (ARIMA/ExponentialSmoothing), PyTorch (optional, deferred import).

Step 7: Evaluation Metrics

What: MAPE (primary), RMSE (tiebreak), MAE, R^2, SMAPE, with zero-sales-safe handling. **Files Created:** `src/evaluation/__init__.py`, `src/evaluation/metrics.py`, `tests/test_metrics.py` **Why:** A single, tested definition of "how good is this forecast" must exist before any model comparison can be meaningful — this is a leaf module with no dependency on models or training, so it can be built and tested in isolation. **Technologies Introduced:** scikit-learn's `mean_absolute_error` / `mean_squared_error` / `r2_score`.

Step 8: Walk-Forward Cross-Validation

What: Expanding-window train/validation folds plus a held-out final test window, split by date (not row/shuffle). **Files Created:** `src/training/__init__.py`, `src/training/cross_validator.py`, `tests/test_cv.py` **Why:** This is the leakage-safety net on the CV side (complementing Step 5's feature-side test) — built and proven correct before wiring it into the full training orchestrator, since a bug here would silently invalidate every later metric. **Technologies Introduced:** None new — pure pandas date-mask logic.

Step 9: Experiment Tracking (MLflow)

What: Run logging per model/fold, model registry, and the >2%-MAPE-improvement promotion gate. **Files Created:** `src/mlflow_utils/__init__.py`, `src/mlflow_utils/tracker.py` **Why:** Built before the training orchestrator since `train_pipeline.py` calls directly into it; isolating MLflow's API surface here means the orchestrator never needs MLflow-specific code. **Technologies Introduced:** MLflow, SQLite (as MLflow's backend).

Step 10: Training Orchestrator

What: Wire together model factory + cross-validator + metrics + MLflow tracker into the full train → select → refit → test → register flow. **Files Created:** `src/training/train_pipeline.py` **Why:** This is the capstone of Steps 6–9 — it can only be built once every dependency it orchestrates already exists and is independently correct. **Technologies Introduced:** None new — pure orchestration.

Step 11: Forecast Generation

What: Turn a trained model into an actual future forecast (recursive for feature models, direct for ARIMA/ExpSmoothing/LSTM). **Files Created:** `src/api/__init__.py`, `src/api/forecast_api.py` **Why:** Needs a trained `best_model.pkl` (Step 10's output) to exist conceptually before it can be built; produces `reports/forecast.csv`, the artifact the dashboard later serves. **Technologies Introduced:** None new.

Step 12: Drift & Performance Monitoring

What: KS-test feature/target drift detection (+ optional Evidently HTML report) and recent-window MAPE degradation checking. **Files Created:** `src/monitoring/__init__.py`, `src/monitoring/drift_detector.py`, `src/monitoring/performance_monitor.py` **Why:** Both need a trained model and feature importances (Step 10's outputs) to compare against — natural to build once training exists. **Technologies Introduced:** scipy (`ks_2samp`), Evidently (optional).

Step 13: Retraining Orchestrator

What: Combine the two monitoring signals into trigger logic, and re-run the full pipeline on demand with a JSONL audit log. **Files Created:** `src/retraining/__init__.py`, `src/retraining/orchestrator.py` **Why:** This is the feedback loop's capstone — it calls back into Steps 2–10 (load, validate, clean, features, train) as a group, so it must come after all of them exist. **Technologies Introduced:** None new — stdlib `json` / `time`.

Step 14: Pipeline Entry Point

What: Wire load → validate → clean → features → train → forecast → drift into one runnable script. **Files Created:** `main.py` **Why:** Only meaningful once every stage function it calls (Steps 2–5, 10–12) exists; this is the "does the whole thing actually run end-to-end" integration point. **Technologies Introduced:** `argparse` (CLI flag).

Step 15: Dashboard Backend (FastAPI)

What: A local dev HTTP API reading pipeline artifacts as JSON, plus two POST actions (retrain, drift-check) calling back into `src/`. **Files Created:** `dashboard/server.py` **Why:** Can only be built once the artifacts contract (Steps 2–13's file outputs) is stable — it's a pure reader by design, so it's decoupled from pipeline internals and can be built/iterated independently of them. **Technologies Introduced:** FastAPI, uvicorn, CORS middleware.

Step 16: React Frontend Dashboard

What: A 7-page Vite + React SPA consuming the FastAPI backend. **Files Created:** `frontend/` (Vite scaffold, `index.html`, `vite.config.js`, `package.json`), `src/main.jsx`, `src/App.jsx`, `src/api.js`, `src/useApi.js`, `src/colors.js`, `src/metricInfo.js`, `src/index.css`, `src/components/*`, `src/pages/*` **Why:** Built last since it depends entirely on the backend's API surface (Step 15) being stable; the shared utility layer (`api.js`, `useApi.js`, `colors.js`, `metricInfo.js`) is built before the pages that consume it, and small components (`ChartCard`, `MetricCard`, etc.) before the pages that compose them. **Technologies Introduced:** React 19, Vite, Recharts, axios, react-router-dom, oxlint.

Step 17: Remaining Test Coverage

What: `tests/test_validation.py` (rounds out the four-file test suite alongside Steps 5 and 8's tests). **Files Created:** `tests/__init__.py`, `tests/test_validation.py` **Why:** Placed last in this narrative only because it wasn't required to unblock any other step — in practice it would likely be written alongside Step 3. **Technologies Introduced:** pytest.

Part 5: Execution Summary

How to run the project

```
pip install -r requirements.txt

# 1. Run the full pipeline (downloads M5 from Kaggle if credentials exist,
#    otherwise auto-generates realistic synthetic demand data)
python main.py

# 2. Launch the dashboard API (FastAPI, local dev only — reads pipeline artifacts,
#    exposes retrain/drift actions; not a model-serving layer)
uvicorn dashboard.server:app --reload --port 8000

# 3. Launch the React dashboard (separate terminal)
cd frontend && npm install && npm run dev   # http://localhost:5173

# 4. (optional) MLflow UI
mlflow ui --backend-store-uri sqlite:///mlflow.db
```

Three (or four) separate local processes, no orchestration between them — this is a local development / demo platform, not a production-hardened service (see Part 2 §5 for the full honest assessment: no auth, no containers, no queue).

What happens when you run `python main.py`

1. **Load config** — `config.yaml` is parsed and validated (`load_config`); RNGs (`random` , `numpy` , `torch`) are seeded from `random_seed: 42` .
2. **Load data** — Kaggle M5 download attempted first (if `data.source: kaggle`); falls back to synthetic generation with zero external credentials if it fails. A `data.source: csv` config points at your own file instead.
3. **Validate** — schema + quality-score check; a hard `SystemExit` if the data is structurally invalid. Writes `reports/validation_report.json` .
4. **Clean** — per-series gap reindexing + imputation. Writes `data/processed/clean.parquet` .
5. **Build features** — ~40 leakage-safe features. Writes `data/processed/features.parquet` .
6. **Train** — 7 models × 3 walk-forward folds, every run logged to MLflow; best model (lowest mean MAPE, RMSE tiebreak) refit on train+val and scored once on the held-out test window; registered/promoted to Production if it beats the incumbent by >2% test MAPE. Writes `reports/model_comparison.csv` , `reports/feature_importance.csv` , `models/best_model.pkl` , `models/best_model_meta.json` .
7. **Forecast** — next 30 periods per series from the winning model. Writes `reports/forecast.csv` .
8. **Drift check** — KS test on the top-5 important features + target (if `drift_detection.enabled`). Writes `reports/drift_report.json` (+ optional `reports/drift_report.html` if Evidently is installed).

A full run on the default top-10-series synthetic/M5 subset is designed to complete in well under 15 minutes.

Expected inputs and outputs

Input	Output
`config.yaml` (all tunables)	Every artifact listed below
Kaggle M5 CSVs *or* a user CSV (`date, sales` , + optional ids) *or* nothing (synthetic fallback)	`data/processed/clean.parquet` , `data/processed/features.parquet`
Trained models (in-memory during the run)	`models/best_model.pkl` , `models/best_model_meta.json` , `reports/model_comparison.csv` , `reports/feature_importance.csv` , MLflow runs in `mlflow.db`
Cleaned data + best model	`reports/forecast.csv`
Features + feature importances	`reports/drift_report.json` (+ `.html`)
Dashboard "Retrain" click	Re-runs steps 2–6 above; appends one line to `reports/retraining_log.jsonl`

Common workflows

- **First run**: `python main.py` → `uvicorn dashboard.server:app --port 8000` → `npm run dev` → open `http://localhost:5173` . Every dashboard page will 404 with "run the pipeline first" until `main.py` has completed once.
- **Bring your own data**: set `data.source: 'csv'` and `data.csv_path: 'data/raw/my_sales.csv'` in `config.yaml` (columns: `date, sales` , optionally `series_id` / `item_id` / `store_id`), then re-run `python main.py` .
- **Tune models**: edit any `models.*` block in `config.yaml` (hyperparameters, `enabled` flags) and re-run — no code changes needed.
- **Check for drift / retrain on demand**: use the Overview page's "Check Drift" and "Retrain Now" buttons, which POST to `/api/drift-check` and `/api/retrain` and synchronously re-run the relevant pipeline stages.
- **Inspect experiment history**: `mlflow ui --backend-store-uri sqlite:///mlflow.db` (port 5000 by default) for the full run/registry view beyond what the dashboard's MLflow Tracking page surfaces.
- **Run the test suite**: `python -m pytest tests -q` — covers metrics, feature-leakage-safety, walk-forward CV, and schema/outlier validation (see Part 3, Group 9 for explicit coverage gaps: models, MLflow, monitoring, retraining, and the forecast API are currently untested).

Troubleshooting (from README.md)

Problem	Fix
Kaggle 403 / no credentials	Pipeline auto-falls back to synthetic data; add `~/.kaggle/kaggle.json` + accept the M5 competition rules for real data
Training too slow	Lower `data.n_series` , `models.*.n_estimators` , or `models.lstm.epochs`
Dashboard shows "No pipeline artifacts"	Run `python main.py` first
Evidently HTML missing	KS-test drift core still runs regardless; the Evidently report is optional

Index of Technologies

Languages: Python · JavaScript/JSX

Frameworks & Libraries: pandas · numpy · scikit-learn · LightGBM · XGBoost · statsmodels (ARIMA/ExponentialSmoothing) · PyTorch · MLflow · Evidently · FastAPI · uvicorn · React · Vite · Recharts · axios · react-router-dom

Databases & Data Storage: SQLite (via `mlflow.db`) · Parquet · CSV

APIs & External Services: Kaggle API (via kagglehub)

Build Tools & Build Systems: Vite (build/bundle)

Testing & QA: pytest · oxlint

Development Tools: `config.yaml` (centralized configuration)

Deployment & DevOps: Local-only manual process launch (no Dockerfile/CI/orchestration present)

Package Managers & Dependency Management: pip / `requirements.txt` · npm / `package-lock.json`

Other Utilities: PyYAML · scipy (`ks_2samp`) · holidays · pyarrow · matplotlib

Index of Files

Group 1 — Configuration & Entry Point

`config.yaml` · `main.py` · `src/constants.py` · `src/utils/config_loader.py` · `src/logger.py` · `src/utils/serialization.py` · `setup.py` · `.env.example` · `src/__init__.py`

Group 2 — Data Loading & Validation

`src/data_loading/__init__.py` · `src/data_loading/downloader.py` · `src/data_loading/loader.py` · `src/validation/__init__.py` · `src/validation/validator.py`

Group 3 — Preprocessing & Feature Engineering

`src/preprocessing/__init__.py` · `src/preprocessing/preprocessor.py` · `src/features/__init__.py` · `src/features/feature_engineering.py`

Group 4 — Model Implementations

`src/models/__init__.py` · `src/models/base_model.py` · `src/models/baseline_models.py` · `src/models/statistical_models.py` · `src/models/tree_models.py` · `src/models/lstm_model.py` · `src/models/model_factory.py`

Group 5 — Training, Cross-Validation & Evaluation

`src/training/__init__.py` · `src/training/cross_validator.py` · `src/training/train_pipeline.py` · `src/evaluation/__init__.py` · `src/evaluation/metrics.py`

Group 6 — Experiment Tracking, Drift Monitoring & Automated Retraining

`src/mlflow_utils/__init__.py` · `src/mlflow_utils/tracker.py` · `src/monitoring/__init__.py` · `src/monitoring/drift_detector.py` · `src/monitoring/performance_monitor.py` · `src/retraining/__init__.py` · `src/retraining/orchestrator.py`

Group 7 — Forecast API & Dashboard Backend

`src/api/__init__.py` · `src/api/forecast_api.py` · `dashboard/server.py`

Group 8 — React Frontend Dashboard

`frontend/index.html` · `frontend/vite.config.js` · `frontend/package.json` · `frontend/.oxlintrc.json` · `frontend/src/main.jsx` · `frontend/src/App.jsx` · `frontend/src/api.js` · `frontend/src/useApi.js` · `frontend/src/colors.js` · `frontend/src/metricInfo.js` · `frontend/src/index.css` ·

`frontend/src/components/{ChartCard,InfoTooltip,Loading,MetricCard,Sidebar,StatusBadge}.jsx` · `frontend/src/pages/{Overview,DataOverview,Eda,ModelComparison,Forecasting,MlflowTracking,DriftMonitoring}.jsx`

Group 9 — Test Suite

`tests/__init__.py` · `tests/test_cv.py` · `tests/test_features.py` · `tests/test_metrics.py` · `tests/test_validation.py`

Not covered by any test (explicit gap)

`src/models/*` · `src/mlflow_utils/*` · `src/monitoring/*` · `src/retraining/*` · `src/api/*` · `src/data_loading/*` · `src/preprocessing/*` · `src/training/train_pipeline.py`